Lab 2: Stamp Machine

ME 416
Bryson Medd, Aviva Harman
October 16, 2026

A: Problem 1

A. Offsets from tag centers to stamp discs' centers

To determine these distances, it's necessary to make a few measurements from the OnShape document. These reference the center of curvature of the stamp holder as the origin:

- $r_{tag\ 1} = r_{tag\ 2} = 84.75\ mm$ (distance from origin to each tag's center, in xy plane)
- $r_{inner\ row} = 125.00\ mm$, $r_{outer\ row} = 145.66\ mm$ (radial distance from origin to each stamp's center, in xy plane)
- $\theta_{stamp} = 8.007^o * (n - 1) - 9.604^o$
  - Where n is stamp column number, defined in Figure 2
  - Angles are defined with positive as counterclockwise
  - Zero degrees is defined as the location of the righthand tag (tag 1)
- $\theta_{tag\ 1} = 0^o$, $\theta_{tag\ 2} = 76.870^o$
- $z_{outer\ row\ stamp} = 48.087\ mm$, $z_{inner\ row\ stamp} = 34.087\ mm$
  - Measured from top of disc
- $z_{tag\ 1\ center} = z_{tag\ 2\ center} = 5.4\ mm$

The stamp's frame is defined as shown in Figure 1.

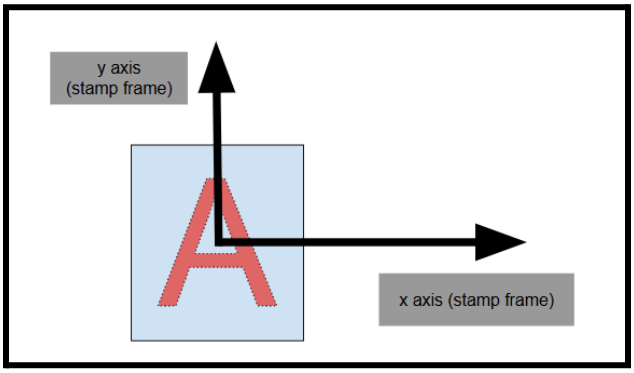


Figure 1: Stamp's local axes.

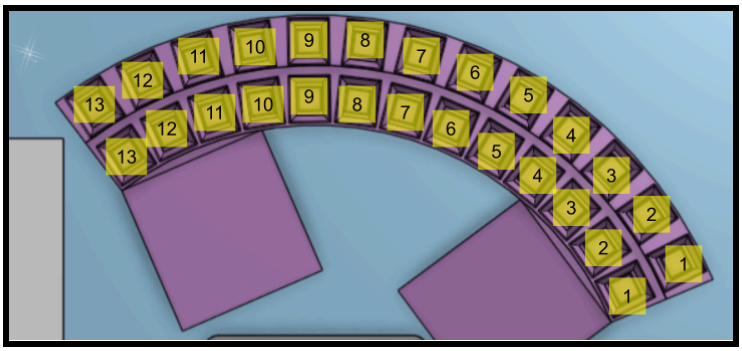


Figure 2: Stamp row number *n*, defined.

The first step in this problem is to define the distance from the stamp to the tag in the stamp holder's reference frame (shown in Figure 3). Here, the origin is the holder's radius of curvature, and the x axis intersects the center of the tag 1 location.

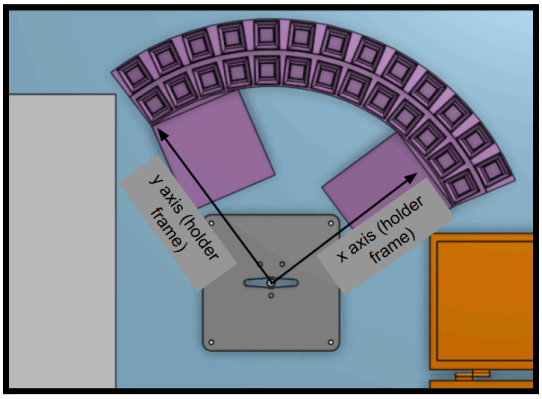


Figure 3: Stamp holder local axes.

The distance from a stamp to a tag in the stamp holder's frame is:

$$\mathbf{d}_{holder\ frame} = \left[r_{tag}cos\theta_{tag} \quad -r_{tag}sin\theta_{tag}\right]^T - \left[r_{stamp}cos\theta_{stamp} \quad -r_{stamp}sin\theta_{stamp}\right]^T$$

Rotating this from the holder's frame to the stamp's frame:

$$\mathbf{d}_{stamp\ frame} = \mathbf{R}\mathbf{d}_{holder\ frame'} \qquad \mathbf{R} = \begin{bmatrix} cos\theta & sin\theta \\ -sin\theta & cos\theta \end{bmatrix}, \qquad \theta = -\left(90 - \theta_{stamp}\right)$$

Finally, a simple z offset is added:

$$\mathbf{d}_{stamp\ frame,\,3d} = \left[\mathbf{d}_{stamp\ frame}{}^T \quad z_{tag\ center} - z_{stamp}\right]^T$$

This defines the offset from the top of a stamp to the center of an AprilTag.

B. Stamp rotation

A stamp picked up when the arm is at yaw ϕ_{pick} and placed when the arm is at ϕ_{press} has been rotated by $\phi_{press} - \phi_{pick}$ from its original orientation in the stamp holder. This is a rotation about the z axis (up).

C. Letter arc

The letters are stamped in an arc because the arm's reach is small and radially symmetric about its origin. The arm doesn't have enough range to travel to points in a straight line, unless the line is very short. This could be fixed by adding a linear slide at the base that moved the whole robot arm in the y-axis.

B: Problem 2

Table 1: Commands vs. outcomes.

Letter	$[U \quad V]^T$ commanded for stamp press (mm)	Joint angles required, $[\theta_1 \quad \theta_2]$ (degrees)	Servo commands required, $[T_1 \quad T_2]$ (μs)	$[U \quad V]^T$ where letter landed (mm)
B	-125.7, 28.9	48.1, -38.9	94.5, 97.5	-130.5, 28
R	-125.7, 43.9	48.0, -38.8	94.6, 97.5	-130, 40
Y	-125.7, 58.9	47.9, -38.7	94.7 97.6	-130, 56
A	-133.1, 22.7	51.0, -56.7	91.4, 77.2	-133, 3

C: Error Budget

From the MATLAB camera calibration, the mean reprojection error is 0.11 pixels. This can be multiplied by the pixel pitch to propagate it to units of mm. After measuring the size of the tag in the image, we obtain that 84 px=50mm, so the pixel pitch is 0.5952 mm/px. This means the error due to reprojection is 0.11 px*0.5952 mm/px=**0.0655 mm**. Next is the tag corner error through **H**. This is represented by $\sqrt{2} \cdot \sigma_c \cdot$mmPerPx / 50 mm, in radians, where σ_c is the corner error in pixels. Estimating σ_c to be 1 px, the corner error in radians is then 0.0168 rad. This can then be multiplied by the radius the arm uses, which is approximately 150 mm (as this is about the distance from the base to the far stamps). Then, tag corner error is 0.0168 rad*150 mm=**2.53 mm**. We accounted for the disc height in our program using $\mathbf{H}_h$, so the remaining uncertainty due to parallax is $\sigma_h \cdot d / Hcam$, where σ_h is the uncertainty of the disc height, d is horizontal distance from where the optical axis hits the table to the point of interest, and H_{cam} is camera height. After measuring approximate values, it's found that d=238 mm and H_{cam} is 555 mm. The value of σ_h is estimated to be 3 mm because the screws attached to the tops of the magnets were at heights varying about this amount, and the stamp holder is assumed to have little height variation. Therefore, parallax

error is **1.29 mm** (in the direction radially outward from the camera nadir). Finally, servo-fit residual error was calculated by Claude and determined to be **3.0 mm** in the x-y plane and **1.0 mm** in the z direction. Propagating these errors, the predicted error is **4.1 mm**. This is very similar to, and slightly smaller than, the errors shown in Section E (4-6 mm). It makes sense that the actual error is larger because there are many factors at play in the physical setup, including arm geometry inconsistencies, lack of thorough data about the servo fit accuracy, servo drag, and inaccuracy of the physical measurements. The most dominant sources of error are tag corner error and servo-fit residual error.

D: Letter Rotations

The later letters are rotated more clockwise than the initial letters. This makes sense because according to our answer in 1b, the letters will be rotated according to their initial orientation plus the change in yaw of the RastiCrane. The letters were ordered from left to right in the stamp holder, and placed to the left of the holder, so it makes sense that the later letters have a larger clockwise rotation because they had a more clockwise position in the stamp holder. The change in yaw of the RastiCrane is similar for all the letters because the letters were ordered from left to right both in the stamp holder and on the paper, so the rotation differences are governed by the stamp positions.

E: Arm Comparison

Table 2: Misses for Bryson's arm (average: 5.29 mm).

Letter	B	R	Y
Miss magnitude (mm)	4.88	5.81	5.19

Table 3: Misses for Aviva's arm (average: 5.29 mm).

Letter	A
Miss magnitude (mm)	19.7

The average error is 5.29 mm for Bryson's arm and 19.7 mm for Aviva's arm. This is because we didn't have time to properly calibrate Aviva's arm.

F: Contribution Statement

Claude was used both to assist with project structure and code. First, we used it to create an outline of important steps based on the assignment document. Then we used it to create multiple Python programs: a servo calibration GUI, arm position control GUI, and the final computer vision + stamping GUI. For each one, we troubleshooted through testing and iterated with Claude to obtain final programs. AI was not used for report writing, but Claude was consulted for guidance on calculating error budget.

G: Appendix

Main program:

"""Stamp holder GUI: webcam photo -> 26 stamp positions -> click a stamp, the arm picks it up with the magnet and puts it back.

python tools/stamp_hover_gui.py [stampholder.json] [--port COM6] [--camera 1]

1. Live view shows whether tags 0, 1 and 3 are seen. Keep the arm out of the shot (Park), then Take photo (or SPACE). The frame goes to photos/webcam_<time>.png and MATLAB runs locateStampHolder on it (about 20 s). Load result... reuses a logs/stampholder_*.json.
2. Click a stamp on the annotated photo. The tool point (EE) picks it up, then puts it back:
   travel lift / retreat / rotate / extend to above the stamp at the safe height
   (if the base doesn't need to turn: lift, then straight across)
   grip straight down until the EE is GRIP_DEPTH_mm below the stamp top
   pull out straight up to the carry height (safe height + the stamp hanging below the EE)
   lower straight down to the slide height (grip height + SLIDE_RAISE_mm, so the
   end plate clears the well rim): the stamp hangs in its well
   slide in along the arm (radially toward the base) by the slide distance, at the
   slide height: the well holds the stamp and the magnet shears off
   clear straight up to the safe height
   Legs that are already done (e.g. the lift when the arm is already high) are skipped.
   Pick up / Put down run the halves on their own (put down = into the selected well).
   While carrying, a click puts the stamp down in the clicked well.
3. Position check: Hover over takes the EE to the hover height above the selected stamp.
   The base jog buttons turn the base alone, the reach jog buttons move the EE in/out along the arm at the same height. Jog until the EE is centred on the stamp; Record offset appends both jogs to logs/stamp_offsets.csv (logs/base_offsets.csv has the older, base-only records).

4. Stamp job tab: with a stamp selected, Run stamp job picks it up, presses it into the ink pad (tag 2 + config/stamp_job.json offset) the set number of times, presses it on the paper spot, puts it back in its well and parks. The presses go down to "approach" above contact at normal speed, then press / press up at the press speed with a dwell at the bottom. Every press (ink and paper) then rocks: the EE goes "rock" mm out along the arm, in, and back to the centre, "rocks" times, before lifting. "Click on photo picks: paper spot" moves the spot. Hover ink / Hover paper check
   the targets. While this tab is open, clicking a stamp (with "Move as soon as a stamp is clicked" ticked) runs the stamp job instead of Pick & return.
   Stamp word does the same for every stamp in the Word entry, in order (e.g. "L3 U7 L3"),

without parking in between: the first letter at the paper spot, each next one the letter spacing further along +V. The whole route is planned before the arm moves. "Click on photo picks: add to word" appends the clicked stamp to the word.

Each leg is one firmware goto, which moves the joints in step, i.e. a straight line in servo space, not in space. A leg whose tool path would bow more than PATH_TOL_mm away from the straight line (FINE_PATH_TOL_mm for the legs at the well) is split at its midpoint until it doesn't.

The safe height starts at the tallest stamp top + SAFE_CLEARANCE_mm. Geometry and the IK are the same as tools/move_to_tag.py (config/arm_bryson.json). Not connected = plan only.
"""

```
import argparse
import csv
import datetime
import json
import math
import queue
import re
import subprocess
import threading
import time
import tkinter as tk
from pathlib import Path
from tkinter import filedialog, messagebox, ttk
from typing import NamedTuple

import cv2
import numpy as np
import serial
import serial.tools.list_ports

from check_stamp_holder import matlabCommand
from ik_gui import BASE, ELBOW, SHOULDER, Kinematics, loadCalibration
from move_to_tag import (DEFAULT_ARM_CONFIG, DEFAULT_CAMERA, DEFAULT_MATLAB,
DEFAULT_SPEED_degps,
                         DEFAULT_WEBCAM_CALIBRATION, PHOTO_DIR, WEBCAM_SIZE_px,
                         WEBCAM_WARMUP_FRAMES, Arm, ReachError, ReachModel, loadArmConfig)
from servo_angle_gui import LOG_DIR, REPO_ROOT, loadJointConfig

HOLDER_CONFIG = REPO_ROOT / "config" / "stamp_holder.json"
JOB_CONFIG = REPO_ROOT / "config" / "stamp_job.json"
LAYOUT_CONFIG = REPO_ROOT / "config" / "layout.json"
```

```
NEEDED_TAGS = ((0, "base tag 0"), (1, "holder tag 1 (left)"), (3, "holder tag 3 (right)"))

GRIP_DEPTH_mm = 15.0            # EE this far below the stamp top to grip it
DEFAULT_SLIDE_mm = 20.0         # slide in along the arm this far to shear the magnet off
DEFAULT_HOVER_mm = 5.0          # Hover over: EE this far above the stamp top
JOG_STEPS_deg = (-1.0, -0.5, 0.5, 1.0)  # base servo jog buttons
REACH_STEPS_mm = (-2.0, -0.5, 0.5, 2.0)  # reach jog buttons (+ = out, away from the base)
SETTLE_s = 0.5                  # pause before the grip / lower legs so the arm stops swinging
SETTLE_BEFORE = {"grip", "lower", "press"}
BASE_OVERSHOOT_deg = 5.0        # base servo deg: every base move ends coming DOWN (clockwise
                        # from above) from at least this far above its target, so the
                        # servo's ~3 deg direction-dependent band lands on the same side
OFFSETS_FILE = LOG_DIR / "stamp_offsets.csv"  # Record offset appends here
SLIDE_RAISE_mm = 8.0            # slide this far above the grip height (rim = stamp top - 10)
SAFE_CLEARANCE_mm = 20.0        # default safe height = tallest stamp top + this
DEFAULT_RETREAT_mm = 90.0       # reach (along the arm) to pull in to before turning the base
PATH_TOL_mm = 2.0               # largest allowed bow of a leg away from its straight line
FINE_PATH_TOL_mm = 1.0          # same, for the legs in and next to the well
FINE_LEGS = {"grip", "pull out", "lower", "slide", "clear", "approach", "press", "rock",
          "press up"}
PATH_SAMPLES = 12
MAX_SPLIT_DEPTH = 4             # up to 16 gotos per leg
SAME_POSE_deg = 0.3             # legs that move no servo more than this are skipped
SAME_BASE_deg = 0.5             # base servo this close to the target: no retreat/rotate needed

DISPLAY_SCALE = 1.5             # 640x480 photo shown at up to 960x720 (less on small screens)
PICK_RADIUS_px = 25             # click within this of a stamp top (display pixels)
LIVE_PERIOD_ms = 60

COLORS = {"lower": "#00dcff", "upper": "#3cff3c", "bad": "#e53935", "selected": "#ffd600",
        "arm": "#ff00ff", "ink": "#ff8c00", "paper": "#4da3ff"}


# ------------------------------------------------------------------ route planning
def armPose(model, theta0, R, W):
    """Pose with the tool point at reach R along the arm (base angle theta0) and height W."""
    t = math.radians(theta0)
    pose = model.solve(R * math.cos(t) - model.ds * math.sin(t),
                       R * math.sin(t) + model.ds * math.cos(t), W)
    pose["R"] = R
    return pose


def poseFromServos(model, s):
```

```
    """(theta0, R, W) of the tool point for servo angles s: the forward kinematics."""
    theta0 = model.baseAngleOfServo(s[BASE])
    _, (rI, zI) = model.kin.fkFromServos(s[SHOULDER], s[ELBOW])
    W = model.zA + zI + model.dz
    return theta0, model.reachFromI(theta0, rI, W), W


def reachOf(model, U, V):
    return math.sqrt(U ** 2 + V ** 2 - model.ds ** 2)


def bow(model, a, b):
    """Largest distance (mm, in the arm's plane) of the servo-space path a -> b from the
    straight line between its ends."""
    (_, Ra, Wa), (_, Rb, Wb) = poseFromServos(model, a), poseFromServos(model, b)
    dR, dW = Rb - Ra, Wb - Wa
    length2 = dR ** 2 + dW ** 2
    worst = 0.0
    for k in range(1, PATH_SAMPLES):
        f = k / PATH_SAMPLES
        _, R, W = poseFromServos(model, [sa + f * (sb - sa) for sa, sb in zip(a, b)])
        u = 0.0 if length2 == 0 else min(1.0, max(0.0, ((R - Ra) * dR + (W - Wa) * dW) / length2))
        worst = max(worst, math.hypot(R - Ra - u * dR, W - Wa - u * dW))
    return worst


def splitLeg(model, startS, end, tol, depth=0):
    """Waypoints (poses) from servo angles startS to the pose end, split until each bows
    less than tol mm. Only end keeps its "event"."""
    if depth >= MAX_SPLIT_DEPTH or bow(model, startS, end["s"]) <= tol:
        return [end]
    t0, R0, W0 = poseFromServos(model, startS)
    try:
        mid = armPose(model, (t0 + end["theta0"]) / 2, (R0 + end["R"]) / 2, (W0 + end["W"]) / 2)
    except ReachError:
        return [end]
    mid["label"] = end["label"]
    mid["slow"] = end.get("slow", False)
    return (splitLeg(model, startS, mid, tol, depth + 1)
            + splitLeg(model, mid["s"], end, tol, depth + 1))


def planRoute(model, startS, legs):
    """legs = [(label, pose)]; drops legs that don't move, splits the rest. Returns poses."""
```

```
    route, s, pending = [], list(startS), None
    for label, pose in legs:
        if max(abs(a - b) for a, b in zip(s, pose["s"])) <= SAME_POSE_deg:
            pending = pose.get("event", pending)        # already there: fire it on the next move
            continue
        pose["label"] = label
        route += splitLeg(model, s, pose, FINE_PATH_TOL_mm if label in FINE_LEGS else
PATH_TOL_mm)
        if pending and route[-1].get("event") is None:
            route[-1]["event"] = pending
        pending = None
        s = pose["s"]
    return route


def endServos(startS, legs):
    """Servo angles at the end of legs (startS if there are none)."""
    return list(legs[-1][1]["s"]) if legs else list(startS)


class Settings(NamedTuple):
    grip: float         # EE below the stamp top when gripping, mm
    slide: float        # slide in along the arm to release, mm
    slideRaise: float   # slide this far above the grip height, mm
    safeW: float        # travel height without a stamp
    retreatR: float     # reach to pull in to before turning the base
    stampHeight: float

    @property
    def carryW(self):
        """Travel height with a stamp: it hangs (stamp height - grip) below the EE."""
        return self.safeW + max(0.0, self.stampHeight - self.grip)


def stampPoint(model, stamp):
    """(U, V, top W, reach R, base angle theta0) of a stamp."""
    U, V, Wtop = stamp["top_UVW_mm"]
    return U, V, Wtop, reachOf(model, U, V), model.solve(U, V, Wtop)["theta0"]


def atStamp(model, U, V, W):
    pose = model.solve(U, V, W)
    pose["R"] = reachOf(model, U, V)
    return pose
```

```
def retreatLegs(model, startS, safeW, retreatR):
    """Lift straight up to safeW, then pull in to retreatR, at the current base angle."""
    theta0, R, W = poseFromServos(model, startS)
    legs = []
    if W < safeW:
        try:
            legs.append(("lift", armPose(model, theta0, R, safeW)))
        except ReachError as err:
            raise ReachError(f"can't lift straight up from here ({err}). Move the arm up first.")
    legs.append(("retreat", armPose(model, theta0, retreatR, safeW)))
    return legs


def travelLegs(model, startS, U, V, travelW, retreatR):
    """To above (U, V) at travelW. If the base has to turn: lift, retreat, rotate, extend.
    If it doesn't (e.g. already above that stamp): lift, then straight across."""
    above = atStamp(model, U, V, travelW)
    if abs(above["s"][BASE] - startS[BASE]) > SAME_BASE_deg:
        legs = retreatLegs(model, startS, travelW, retreatR)
        if above["s"][BASE] > startS[BASE] - BASE_OVERSHOOT_deg:
            # Turning counterclockwise (or only a little clockwise): go past, then come back down.
            legs.append(("overshoot", armPose(model, above["theta0"] + BASE_OVERSHOOT_deg /
model.baseA,
                                              retreatR, travelW)))
        return legs + [
            ("rotate", armPose(model, above["theta0"], retreatR, travelW)),
            ("extend", above),
        ]
    theta0, R, W = poseFromServos(model, startS)
    legs = []
    if W < travelW:
        try:
            legs.append(("lift", armPose(model, theta0, R, travelW)))
        except ReachError as err:
            raise ReachError(f"can't lift straight up from here ({err}). Move the arm up first.")
    return legs + [("extend", above)]


def pickLegs(model, startS, stamp, st):
    """Travel to above the stamp, down until the EE is st.grip below its top, then straight
    up to the carry height. The "gripped" event fires at the bottom."""
    U, V, Wtop, _, _ = stampPoint(model, stamp)
```

```
    grip = atStamp(model, U, V, Wtop - st.grip)
    grip["event"] = "gripped"
    return travelLegs(model, startS, U, V, st.safeW, st.retreatR) + [
        ("grip", grip),
        ("pull out", atStamp(model, U, V, st.carryW)),
    ]


def putLegs(model, startS, stamp, st):
    """Travel to above the well at the carry height, straight down to the slide height
    (st.slideRaise above the grip height), slide in along the arm by st.slide (the well keeps
    the stamp, the magnet shears off), then straight up to the safe height. The "released"
    event fires at the end of the slide."""
    U, V, Wtop, R, theta0 = stampPoint(model, stamp)
    slideW = Wtop - st.grip + st.slideRaise
    slide = armPose(model, theta0, R - st.slide, slideW)
    slide["event"] = "released"
    return travelLegs(model, startS, U, V, st.carryW, st.retreatR) + [
        ("lower", atStamp(model, U, V, slideW)),
        ("slide", slide),
        ("clear", armPose(model, theta0, R - st.slide, st.safeW)),
    ]


def hoverLegs(model, startS, stamp, hover, st):
    """Travel to above the stamp at the safe height, then straight down to hover mm above
    its top."""
    U, V, Wtop, _, _ = stampPoint(model, stamp)
    return travelLegs(model, startS, U, V, st.safeW, st.retreatR) + [
        ("descend", atStamp(model, U, V, Wtop + hover)),
    ]


def pickAndReturnLegs(model, startS, stamp, st):
    pick = pickLegs(model, startS, stamp, st)
    return pick + putLegs(model, endServos(startS, pick), stamp, st)


def parkLegs(model, startS, st, homePose, travelW=None):
    """Retreat at travelW (default the safe height), turn the base to its home angle, then
    go to the firmware's home pose."""
    travelW = st.safeW if travelW is None else travelW
    home = {"s": list(homePose)}
    home["theta0"], home["R"], home["W"] = poseFromServos(model, homePose)
```

```
    return retreatLegs(model, startS, travelW, st.retreatR) + [
        ("rotate", armPose(model, home["theta0"], st.retreatR, travelW)),
        ("home", home),
    ]


class JobSettings(NamedTuple):
    inkDepth: float     # stamp face this far below the ink pad surface, mm
    dabs: int           # presses into the ink
    paperDepth: float   # stamp face this far below the paper surface, mm
    approach: float     # face this far above the surface before pressing, mm
    dwell: float        # wait at the bottom of each press, s
    pressSpeed: float   # servo speed for press / press up, deg/s
    rock: float = 0.0   # each press: rock the EE this far out and in along the arm, mm
    rocks: int = 0      # each press: this many out-and-in rocks at the bottom


def pressHeights(target, depth, st, job):
    """Tool point W (approach, bottom of the press) for pressing the carried stamp's face
    depth mm into the surface at target; the face hangs (stamp height - grip) below the EE."""
    hang = st.stampHeight - st.grip
    return target[2] + hang + job.approach, target[2] + hang - depth


def pressProblem(model, target, depth, st, job):
    """None if the press at target is reachable, else why not (with the lowest W possible)."""
    U, V, _ = target
    approachW, pressW = pressHeights(target, depth, st, job)
    try:
        for W in (approachW, pressW):
            model.solve(U, V, W)
        return None
    except ReachError as err:
        lowest = next((W for W in range(0, 200) if _reachable(model, U, V, W)), None)
        hint = "" if lowest is None else f"; lowest tool W here is ~{lowest}"
        return (f"needs tool W {pressW:.1f} at reach {reachOf(model, U, V):.0f} ({err}){hint}. "
                "Move it further from the base (more -U).")


def _reachable(model, U, V, W):
    try:
        model.solve(U, V, W)
        return True
    except ReachError:
```

```
        return False


def pressLegs(model, startS, target, depth, st, job, event, times=1, rock=0.0, rocks=0):
    """Carrying a stamp: travel to above target = (U, V, surface W) at the carry height,
    down until the stamp face is job.approach above the surface, then `times` slow presses
    to depth below it (dwell at the bottom) and back up. event fires after the last one.
    With rock and rocks, each press rocks at the bottom: the EE moves rock mm out along the
    arm, rock mm in, and back to the centre, rocks times. The end plate stays level, so the
    rocking comes from the stamp tipping on the magnet while the paper holds its face."""
    U, V, surfaceW = target
    approachW, pressW = pressHeights(target, depth, st, job)
    try:
        legs = travelLegs(model, startS, U, V, st.carryW, st.retreatR)
        legs.append(("approach", atStamp(model, U, V, approachW)))
        bottom = atStamp(model, U, V, pressW)
        rocking = []
        for _ in range(rocks if rock > 0 else 0):
            for dR in (rock, -rock):
                rocking.append(armPose(model, bottom["theta0"], bottom["R"] + dR, pressW))
            rocking.append(atStamp(model, U, V, pressW))
    except ReachError as err:
        name = {"inked": "ink pad", "stamped": "paper"}.get(event, event)
        raise ReachError(f"can't reach the {name} press at [{U:.1f}, {V:.1f}] (tool W "
                         f"{pressW:.1f}, reach {reachOf(model, U, V):.1f}): {err}") from None
    for k in range(times):
        press = atStamp(model, U, V, pressW)
        press["slow"], press["dwell"] = True, job.dwell
        legs.append(("press", press))
        for pose in rocking:
            pose = dict(pose, s=list(pose["s"]), slow=True)
            legs.append(("rock", pose))
        up = atStamp(model, U, V, approachW)
        up["slow"] = True
        if k == times - 1:
            up["event"] = event
        legs.append(("press up", up))
    return legs


def letterLegs(model, startS, stamp, ink, spot, st, job):
    """One letter: pick the stamp up, ink it (job.dabs presses), press it on the paper at
    spot, put it back in its well. ink / spot = (U, V, surface W)."""
    legs = pickLegs(model, startS, stamp, st)
```

```
    legs += pressLegs(model, endServos(startS, legs), ink, job.inkDepth, st, job, "inked",
                      job.dabs, rock=job.rock, rocks=job.rocks)
    legs += pressLegs(model, endServos(startS, legs), spot, job.paperDepth, st, job, "stamped",
                      rock=job.rock, rocks=job.rocks)
    return legs + putLegs(model, endServos(startS, legs), stamp, st)


def stampJobLegs(model, startS, stamp, ink, spot, st, job, homePose):
    """The whole job: one letter, then park."""
    legs = letterLegs(model, startS, stamp, ink, spot, st, job)
    return legs + parkLegs(model, endServos(startS, legs), st, homePose)


def stampWordLegs(model, startS, stamps, ink, spots, st, job, homePose):
    """Several letters in one go: letterLegs for each (stamp, spot) in order, straight from
    one to the next, then park once. Each letter's events get its index appended
    ("gripped 0", "stamped 2", ...)."""
    legs = []
    for k, (stamp, spot) in enumerate(zip(stamps, spots)):
        try:
            letter = letterLegs(model, endServos(startS, legs), stamp, ink, spot, st, job)
        except ReachError as err:
            raise ReachError(f"letter {k + 1} ({stamp['name']}): {err}") from None
        for _, pose in letter:
            if pose.get("event"):
                pose["event"] = f"{pose['event']} {k}"
        legs += letter
    return legs + parkLegs(model, endServos(startS, legs), st, homePose)


def parseWord(text, stamps):
    """Stamps named in text (space or comma separated, any case), in order. Raises
    ValueError naming the first unknown name."""
    byName = {s["name"].upper(): s for s in stamps}
    names = [n for n in re.split(r"[\s,]+", text.strip().upper()) if n]
    unknown = next((n for n in names if n not in byName), None)
    if unknown:
        raise ValueError(f"no stamp named {unknown} (names are L1..L13, U1..U13)")
    return [byName[n] for n in names]


def wordSpots(spot, advance, n):
    """Paper spots of n letters: the first at spot, each next one advance mm along +V."""
    U, V, W = spot
```

```
    return [(U, V + k * advance, W) for k in range(n)]


def loadHolder():
    return json.loads(HOLDER_CONFIG.read_text())


def loadJob():
    return json.loads(JOB_CONFIG.read_text())


def baseTagCenter():
    """Tag 0's centre [U, V] from config/layout.json (it defines the world frame)."""
    return json.loads(LAYOUT_CONFIG.read_text())["baseTag"]["centerUV_mm"]


def paperCorner(job):
    """[U, V] of the paper's corner: its edge runs from here toward +V, the paper is at -U."""
    U0, V0 = baseTagCenter()
    return U0 + job["paperEdgeU_mm"], V0 + job["paperCornerV_mm"]


def inkPadPoint(result, job):
    """(U, V, surface W) of the ink pad centre from the ink tag in the result; None if the
    tag wasn't seen (or the result predates other tags being saved)."""
    tags = result.get("tags") or []
    if isinstance(tags, dict):
        tags = [tags]
    tag = next((t for t in tags if t["id"] == job["inkTagId"]), None)
    if tag is None:
        return None
    dU, dV, dW = job["inkOffsetUVW_mm"]
    return tag["centerUV_mm"][0] + dU, tag["centerUV_mm"][1] + dV, tag["height_mm"] + dW


def defaultSafeHeight(holder):
    """Tallest stamp top in the holder CAD (holder on the plate) + SAFE_CLEARANCE_mm."""
    tallest = max(s["seatXYZ_mm"][2] for s in holder["stamps"]) + holder["stampHeight_mm"]
    return math.ceil(tallest + SAFE_CLEARANCE_mm)


def describe(pose):
    s1, s2, s3 = pose["s"]
    event = f"  [{pose['event']}]" if pose.get("event") else ""
```

```
    if pose.get("slow"):
        event = "  (slow)" + event
    return (f"{pose['label']:<8} theta0 {pose['theta0']:7.2f}  reach {pose['R']:6.1f}  "
            f"W {pose['W']:6.1f} mm   s = {s1:6.1f} {s2:6.1f} {s3:6.1f}{event}")


# ------------------------------------------------------------------ stamp results
def stampPixels(result):
    """Raw-photo pixel of each stamp top. Older JSON without topPx: project with the pose
    (no lens distortion, so a few px off near the edges)."""
    return [tuple(s["topPx"]) if "topPx" in s else worldToPixel(result, s["top_UVW_mm"])
            for s in result["stamps"]]


def distortionScale(result, xn):
    """MATLAB's radial model: distorted = undistorted * (1 + k1 r^2 + k2 r^4 ...)."""
    k = np.ravel(result.get("radialDistortion") or [])
    r2 = float(xn @ xn)
    return 1.0 + sum(kk * r2 ** (i + 1) for i, kk in enumerate(k))


def worldToPixel(result, X):
    """Raw-photo pixel of world point X = [U, V, W] (with the lens distortion when the result
    has it; older results project without it, a few px off near the edges)."""
    pose = result["pose"]
    xc = np.array(pose["R"]) @ np.asarray(X, float) + np.ravel(pose["t"])
    xn = xc[:2] / xc[2]
    xn = xn * distortionScale(result, xn)
    K = np.array(pose["K"])
    return K[0, 0] * xn[0] + K[0, 2], K[1, 1] * xn[1] + K[1, 2]


def pixelToWorld(result, px, py, W):
    """[U, V] on the plane at height W seen at raw-photo pixel (px, py)."""
    pose = result["pose"]
    K, R, t = np.array(pose["K"]), np.array(pose["R"]), np.ravel(pose["t"])
    distorted = np.array([(px - K[0, 2]) / K[0, 0], (py - K[1, 2]) / K[1, 1]])
    xn = distorted
    for _ in range(10):                          # undo the radial model by iteration
        xn = distorted / distortionScale(result, xn)
    centre = -R.T @ t
    ray = R.T @ np.array([xn[0], xn[1], 1.0])
    point = centre + (W - centre[2]) / ray[2] * ray
    return float(point[0]), float(point[1])
```

```
def locateWithMatlab(matlab, calibration, photoPath):
    """Run locateStampHolder on the photo. Returns (json path, MATLAB output) or (None, output)."""
    run = subprocess.run([matlab, "-batch", matlabCommand(photoPath, calibration)], cwd=REPO_ROOT,
                         capture_output=True, text=True)
    output = (run.stdout + run.stderr).strip()
    saved = re.search(r"Saved (.+?\.json)", output)
    if run.returncode != 0 or not saved:
        return None, output
    return Path(saved[1].strip()), output


# ---------------------------------------------------------------- GUI
class StampHoverGui:
    def __init__(self, root, args, model):
        self.root = root
        self.args = args
        self.model = model
        self.homePose = [j["home"] for j in model.kin.joints]
        self.arm = None
        self.cap = None
        self.detector = cv2.aruco.ArucoDetector(
            cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_APRILTAG_36h11))
        self.events = queue.Queue()          # callables posted by worker threads
        self.abort = threading.Event()
        self.armBusy = False
        self.matlabBusy = False
        self.live = False
        self.lastFrame = None
        self.result = None
        self.pixels = []
        self.selected = None                 # index into result["stamps"]
        self.armAt = None                    # stamp the arm last picked up or put down
        self.photo = None
        self.holder = loadHolder()
        self.job = loadJob()
        self.inkPad = None                   # (U, V, surface W) from the loaded result
        self.neededTags = NEEDED_TAGS + ((self.job["inkTagId"],
                                          f"ink pad tag {self.job['inkTagId']}"),)

        root.title("Stamp pick and return")
        self._buildConnectionBar()
        body = ttk.Frame(root)
```

```
        body.pack(fill="both", expand=True)
        self.scale = max(1.0, min(DISPLAY_SCALE, (root.winfo_screenheight() - 280) /
WEBCAM_SIZE_px[1],
                                  (root.winfo_screenwidth() - 520) / WEBCAM_SIZE_px[0]))
        w, h = (round(v * self.scale) for v in WEBCAM_SIZE_px)
        self.canvas = tk.Canvas(body, width=w, height=h, background="#202020",
                                highlightthickness=0, cursor="crosshair")
        self.canvas.pack(side="left", padx=6, pady=6)
        self.imageItem = self.canvas.create_image(0, 0, anchor="nw")
        self.canvas.bind("<Button-1>", self.onClick)
        side = ttk.Frame(body)
        side.pack(side="left", fill="both", expand=True, padx=6, pady=6)
        self._buildPhotoPanel(side)
        self._buildMotionPanel(side)
        self._buildLog(side)

        root.bind("<Escape>", lambda _e: self.stop())
        root.bind("<space>", self.onSpace)
        root.protocol("WM_DELETE_WINDOW", self.onClose)
        self.refreshPorts()
        self.pollEvents()
        if args.result:
            self.loadResult(Path(args.result))
        else:
            self.startLive()

    # ---------------------------------------------------------------- layout
    def _buildConnectionBar(self):
        bar = ttk.Frame(self.root, padding=6)
        bar.pack(fill="x")
        ttk.Label(bar, text="Port:").pack(side="left")
        self.portVar = tk.StringVar(value=self.args.port or "")
        self.portBox = ttk.Combobox(bar, textvariable=self.portVar, width=32)
        self.portBox.pack(side="left", padx=4)
        ttk.Button(bar, text="Refresh", command=self.refreshPorts).pack(side="left")
        self.connectButton = ttk.Button(bar, text="Connect", command=self.toggleConnection)
        self.connectButton.pack(side="left", padx=4)
        self.connStatus = ttk.Label(bar, text="Disconnected (plan only)", foreground="#b00020")
        self.connStatus.pack(side="left", padx=8)
        tk.Button(bar, text="STOP (Esc)", command=self.stop, bg="#c62828", fg="white",
                  font=("Segoe UI", 10, "bold"), padx=14).pack(side="right")
        ttk.Button(bar, text="Set", command=self.setSpeed).pack(side="right", padx=4)
        self.speedVar = tk.StringVar(value=f"{self.args.speed:g}")
        ttk.Entry(bar, textvariable=self.speedVar, width=6).pack(side="right")
```

```
    ttk.Label(bar, text="Speed (deg/s):").pack(side="right")

def _buildPhotoPanel(self, side):
    box = ttk.LabelFrame(side, text="Photo", padding=6)
    box.pack(fill="x")
    row = ttk.Frame(box)
    row.pack(fill="x")
    self.liveButton = ttk.Button(row, text="Live view", command=self.startLive)
    self.liveButton.pack(side="left")
    self.photoButton = ttk.Button(row, text="Take photo (Space)", command=self.takePhoto)
    self.photoButton.pack(side="left", padx=4)
    ttk.Button(row, text="Load result...", command=self.chooseResult).pack(side="left")
    self.resultVar = tk.StringVar(value="No stamp positions yet.")
    ttk.Label(box, textvariable=self.resultVar, wraplength=330, justify="left").pack(
        fill="x", pady=(6, 0))

def _buildMotionPanel(self, side):
    notebook = self.notebook = ttk.Notebook(side)
    notebook.pack(fill="x", pady=6)
    box = ttk.Frame(notebook, padding=6)
    notebook.add(box, text="Pick & place (tool point, mm)")
    job = self.jobTab = ttk.Frame(notebook, padding=6)
    notebook.add(job, text="Stamp job")
    self._buildJobPanel(job)
    self.gripVar = tk.StringVar(value=f"{self.args.grip:g}")
    self.slideVar = tk.StringVar(value=f"{self.args.slide:g}")
    self.slideRaiseVar = tk.StringVar(value=f"{self.args.slide_raise:g}")
    self.safeVar = tk.StringVar(value=f"{defaultSafeHeight(self.holder):g}")
    self.retreatVar = tk.StringVar(value=f"{self.args.retreat:g}")
    for row, (label, var) in enumerate((("Grip: EE below stamp top", self.gripVar),
                                        ("Slide back to release", self.slideVar),
                                        ("Slide above grip height", self.slideRaiseVar),
                                        ("Safe travel height W", self.safeVar),
                                        ("Retreat reach", self.retreatVar))):
        ttk.Label(box, text=label).grid(row=row, column=0, sticky="w")
        entry = ttk.Entry(box, textvariable=var, width=8)
        entry.grid(row=row, column=1, sticky="w", padx=4, pady=1)
        entry.bind("<Return>", lambda _e: self.redraw())
        entry.bind("<FocusOut>", lambda _e: self.redraw())
    self.carryVar = tk.StringVar()
    ttk.Label(box, textvariable=self.carryVar).grid(row=5, column=0, columnspan=2, sticky="w")
    self.holdingVar = tk.BooleanVar(value=False)
    ttk.Checkbutton(box, text="Carrying a stamp (untick if it fell off)",
                    variable=self.holdingVar).grid(row=6, column=0, columnspan=2,
```

```
                                        sticky="w", pady=(4, 0))
        self.moveOnClickVar = tk.BooleanVar(value=True)
        ttk.Checkbutton(box, text="Move as soon as a stamp is clicked",
                        variable=self.moveOnClickVar).grid(row=7, column=0, columnspan=2,
                                                           sticky="w")
        buttons = ttk.Frame(box)
        buttons.grid(row=8, column=0, columnspan=2, sticky="w", pady=(6, 0))
        ttk.Button(buttons, text="Pick & return", command=self.pickAndReturn).pack(side="left")
        ttk.Button(buttons, text="Pick up", command=self.pickUp).pack(side="left", padx=4)
        ttk.Button(buttons, text="Put down", command=self.putDown).pack(side="left")
        buttons = ttk.Frame(box)
        buttons.grid(row=9, column=0, columnspan=2, sticky="w", pady=(4, 0))
        ttk.Button(buttons, text="Retreat", command=self.retreat).pack(side="left")
        ttk.Button(buttons, text="Park (home)", command=self.park).pack(side="left", padx=4)
        self._buildBaseCheck(box)
        self.selectedVar = tk.StringVar(value="Click a stamp on the photo.")
        ttk.Label(box, textvariable=self.selectedVar, wraplength=330, justify="left",
                  font=("Consolas", 9)).grid(row=10, column=0, columnspan=2, sticky="w",
                                             pady=(6, 0))
        self.updateCarryText()

    def _buildBaseCheck(self, box):
        frame = ttk.LabelFrame(box, text="Position check", padding=4)
        frame.grid(row=11, column=0, columnspan=2, sticky="we", pady=(8, 0))
        row = ttk.Frame(frame)
        row.pack(fill="x")
        ttk.Label(row, text="Hover above top").pack(side="left")
        self.hoverVar = tk.StringVar(value=f"{self.args.hover:g}")
        ttk.Entry(row, textvariable=self.hoverVar, width=6).pack(side="left", padx=4)
        ttk.Button(row, text="Hover over", command=self.hoverOver).pack(side="left")
        row = ttk.Frame(frame)
        row.pack(fill="x", pady=(4, 0))
        ttk.Label(row, text="Base servo").pack(side="left")
        for step in JOG_STEPS_deg:
            ttk.Button(row, text=f"{step:+g}", width=5,
                       command=lambda d=step: self.jogBase(d)).pack(side="left", padx=1)
        row = ttk.Frame(frame)
        row.pack(fill="x", pady=(4, 0))
        ttk.Label(row, text="Reach (mm)").pack(side="left")
        for step in REACH_STEPS_mm:
            ttk.Button(row, text=f"{step:+g}", width=5,
                       command=lambda d=step: self.jogReach(d)).pack(side="left", padx=1)
        ttk.Button(frame, text="Record offset", command=self.recordOffset).pack(
            anchor="w", pady=(4, 0))
```

```
    self.jogVar = tk.StringVar()
    ttk.Label(frame, textvariable=self.jogVar, wraplength=330, justify="left",
              font=("Consolas", 9)).pack(fill="x", pady=(4, 0))
    self.hoverAt = None
    self.updateJogText()

def _buildJobPanel(self, box):
    job = self.job
    corner = paperCorner(job)
    inset = job["paperSpotInset_mm"]
    self.jobVars = {}
    fields = (("inkDepth", "Ink: press face below pad", job["inkPressDepth_mm"]),
              ("dabs", "Ink: presses", job["inkDabs"]),
              ("paperDepth", "Paper: press face below paper", job["paperPressDepth_mm"]),
              ("rock", "Rock out/in along arm", job.get("rock_mm", 0)),
              ("rocks", "Rocks per press", job.get("rocks", 0)),
              ("approach", "Slow from this above contact", job["approach_mm"]),
              ("dwell", "Dwell at the bottom (s)", job["dwell_s"]),
              ("pressSpeed", "Press speed (deg/s)", job["pressSpeed_degps"]),
              ("spotU", "Paper spot U", round(corner[0] - inset[0], 1)),
              ("spotV", "Paper spot V", round(corner[1] + inset[1], 1)),
              ("advance", "Letter spacing (spot +V by)", job["advance_mm"]))
    for row, (key, label, value) in enumerate(fields):
        var = self.jobVars[key] = tk.StringVar(value=f"{value:g}")
        ttk.Label(box, text=label).grid(row=row, column=0, sticky="w")
        entry = ttk.Entry(box, textvariable=var, width=8)
        entry.grid(row=row, column=1, sticky="w", padx=4, pady=1)
        entry.bind("<Return>", lambda _e: self.redraw())
        entry.bind("<FocusOut>", lambda _e: self.redraw())
    row = len(fields)
    word = ttk.Frame(box)
    word.grid(row=row, column=0, columnspan=2, sticky="we", pady=(4, 0))
    ttk.Label(word, text="Word (stamps in order)").pack(side="left")
    self.wordVar = tk.StringVar()
    entry = ttk.Entry(word, textvariable=self.wordVar, width=24)
    entry.pack(side="left", padx=4, fill="x", expand=True)
    entry.bind("<Return>", lambda _e: self.redraw())
    entry.bind("<FocusOut>", lambda _e: self.redraw())
    ttk.Button(word, text="Clear", width=6,
               command=lambda: (self.wordVar.set(""), self.redraw())).pack(side="left")
    pick = ttk.Frame(box)
    pick.grid(row=row + 1, column=0, columnspan=2, sticky="w", pady=(4, 0))
    ttk.Label(pick, text="Click on photo picks:").pack(side="left")
    self.clickModeVar = tk.StringVar(value="stamp")
```

```
    for value, text in (("stamp", "stamp"), ("paper", "paper spot"), ("word", "add to word")):
        ttk.Radiobutton(pick, text=text, value=value,
                        variable=self.clickModeVar).pack(side="left", padx=2)
    buttons = ttk.Frame(box)
    buttons.grid(row=row + 2, column=0, columnspan=2, sticky="w", pady=(6, 0))
    ttk.Button(buttons, text="Run stamp job", command=self.runStampJob).pack(side="left")
    ttk.Button(buttons, text="Stamp word", command=self.runStampWord).pack(side="left",
                                                                            padx=4)
    ttk.Button(buttons, text="Hover ink", command=self.hoverInk).pack(side="left")
    ttk.Button(buttons, text="Hover paper", command=self.hoverPaper).pack(side="left",
                                                                          padx=4)
    self.jobInfoVar = tk.StringVar()
    ttk.Label(box, textvariable=self.jobInfoVar, wraplength=330, justify="left",
              font=("Consolas", 9)).grid(row=row + 3, column=0, columnspan=2, sticky="w",
                                         pady=(6, 0))

def _buildLog(self, side):
    box = ttk.LabelFrame(side, text="Log", padding=4)
    box.pack(fill="both", expand=True)
    self.logText = tk.Text(box, width=58, height=16, font=("Consolas", 8), state="disabled")
    scroll = ttk.Scrollbar(box, command=self.logText.yview)
    self.logText.configure(yscrollcommand=scroll.set)
    scroll.pack(side="right", fill="y")
    self.logText.pack(fill="both", expand=True)

# ---------------------------------------------------------------- photo / live view
def openCamera(self):
    cap = cv2.VideoCapture(self.args.camera, cv2.CAP_DSHOW)
    if not cap.isOpened():
        self.log(f"Could not open camera {self.args.camera}. Try another --camera index.")
        return None
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, WEBCAM_SIZE_px[0])
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, WEBCAM_SIZE_px[1])
    for _ in range(WEBCAM_WARMUP_FRAMES):          # let auto exposure settle
        cap.read()
    return cap

def startLive(self):
    if self.cap is None:
        self.cap = self.openCamera()
        if self.cap is None:
            return
    if not self.live:
        self.live = True
```

```
        self.canvas.delete("overlay")
        self.updateLive()

    def updateLive(self):
        if not self.live:
            return
        ok, frame = self.cap.read()
        if not ok:
            self.log("Camera read failed.")
            self.live = False
            return
        if frame.shape[1::-1] != WEBCAM_SIZE_px:
            self.log(f"Camera gives {frame.shape[1]}x{frame.shape[0]}, the calibration needs "
                     f"{WEBCAM_SIZE_px[0]}x{WEBCAM_SIZE_px[1]}.")
            self.live = False
            return
        self.lastFrame = frame
        view = frame.copy()
        corners, ids, _ = self.detector.detectMarkers(frame)
        seen = set() if ids is None else set(ids.ravel().tolist())
        if ids is not None:
            cv2.aruco.drawDetectedMarkers(view, corners, ids)
        for row, (tagId, name) in enumerate(self.neededTags):
            found = tagId in seen
            cv2.putText(view, f"{name}: {'seen' if found else 'NOT SEEN'}", (10, 25 + 25 * row),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 200, 0) if found else (0, 0, 255), 2)
        self.showImage(view)
        self.root.after(LIVE_PERIOD_ms, self.updateLive)

    def onSpace(self, event):
        # Space also presses a focused button and types into entries; only take over otherwise.
        if self.live and event.widget.winfo_class() not in ("TEntry", "TButton", "Button",
                                                            "TCheckbutton", "TCombobox"):
            self.takePhoto()

    def takePhoto(self):
        if self.matlabBusy:
            self.log("MATLAB is still locating the last photo.")
            return
        if not self.live or self.lastFrame is None:
            self.log("Start the live view first.")
            return
        self.live = False
        PHOTO_DIR.mkdir(exist_ok=True)
```

```
    stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    photoPath = PHOTO_DIR / f"webcam_{stamp}.png"
    cv2.imwrite(str(photoPath), self.lastFrame)
    self.showImage(self.lastFrame)
    self.log(f"Saved {photoPath.name}. Locating the stamp holder in MATLAB (about 20 s)...")
    self.matlabBusy = True
    self.photoButton.state(["disabled"])

    def work():
        jsonPath, output = locateWithMatlab(self.args.matlab, self.args.calibration, photoPath)
        self.post(self.matlabDone, jsonPath, output)
    threading.Thread(target=work, daemon=True).start()

def matlabDone(self, jsonPath, output):
    self.matlabBusy = False
    self.photoButton.state(["!disabled"])
    if jsonPath is None:
        self.log(output)
        self.log("MATLAB did not produce stamp positions. Check the messages above, then "
                 "Live view and try again.")
        return
    for line in output.splitlines():
        if line.startswith(("WARNING", "Holder tags", "Base tag")):
            self.log(line)
    self.loadResult(jsonPath)

def chooseResult(self):
    path = filedialog.askopenfilename(initialdir=LOG_DIR, title="Stamp holder result",
                                      filetypes=[("Stamp holder", "stampholder_*.json")])
    if path:
        self.loadResult(Path(path))

def loadResult(self, path):
    try:
        result = json.loads(path.read_text())
        stamps = result["stamps"]
    except (OSError, ValueError, KeyError) as err:
        messagebox.showerror("Load result", f"Could not read {path}:\n{err}")
        return
    if isinstance(stamps, dict):                # jsonencode of a single struct
        result["stamps"] = [stamps]
    image = None
    for candidate in (path.with_suffix(".jpg"), Path(result.get("photoPath", ""))):
        if candidate.is_file():
```

```
            image = cv2.imread(str(candidate))
            if image is not None:
                break
        if image is None:
            messagebox.showerror("Load result", f"No image found for {path.name}.")
            return
        self.live = False
        self.result = result
        self.resultName = path.name
        self.pixels = stampPixels(result)
        self.background = image
        self.selected = self.armAt = self.hoverAt = None
        holder = result.get("holder", {})
        text = f"{path.name}"
        if "baselineError_mm" in holder:
            text += f"\nHolder tags {holder['baseline_mm']:.1f} mm apart " \
                    f"(CAD {holder['baselineError_mm']:+.1f} mm)"
        if any("topPx" not in s for s in result["stamps"]):
            text += "\n(older result without topPx: click targets ignore lens distortion)"
        self.inkPad = inkPadPoint(result, self.job)
        if self.inkPad is None:
            text += f"\nInk pad tag {self.job['inkTagId']} not in this result: no stamp job"
        self.resultVar.set(text)
        if isinstance(result.get("warnings"), str):
            result["warnings"] = [result["warnings"]]
        for warning in result.get("warnings") or []:
            self.log(f"WARNING: {warning}")
        self.log(f"Loaded {len(self.pixels)} stamps from {path.name}.")
        self.redraw()

    def showImage(self, bgr):
        view = cv2.resize(bgr, None, fx=self.scale, fy=self.scale,
                          interpolation=cv2.INTER_LINEAR)
        ok, buf = cv2.imencode(".ppm", view)
        if ok:
            self.photo = tk.PhotoImage(data=buf.tobytes())
            self.canvas.itemconfigure(self.imageItem, image=self.photo)

    # ------------------------------------------------------------------ stamp overlay
    def motionSettings(self):
        """Settings from the entries; None (and a log line) if one is bad."""
        try:
            grip, slide, slideRaise, safeW, retreatR = (float(v.get()) for v in (
                self.gripVar, self.slideVar, self.slideRaiseVar, self.safeVar, self.retreatVar))
```

```
    except ValueError:
        self.log("Grip, slide, slide height, safe height and retreat reach must be numbers.")
        return None
    if grip < 0 or slide < 0:
        self.log("Grip and slide must be >= 0 mm.")
        return None
    return Settings(grip, slide, slideRaise, safeW, retreatR, self.holder["stampHeight_mm"])

def updateCarryText(self):
    st = self.motionSettings()
    self.carryVar.set("" if st is None else
                      f"Carry height W {st.carryW:g} (stamp hangs "
                      f"{st.carryW - st.safeW:g} mm below the EE)")

def reachProblem(self, stamp, st):
    """None if every pose of picking up / putting down this stamp is reachable, else why not."""
    U, V, Wtop = stamp["top_UVW_mm"]
    if st.safeW < Wtop:
        return f"safe height {st.safeW:g} is below the stamp top ({Wtop:.1f})"
    try:
        _, _, _, R, theta0 = stampPoint(self.model, stamp)
        for W in (Wtop - st.grip, Wtop - st.grip + st.slideRaise, st.safeW, st.carryW):
            self.model.solve(U, V, W)
        for W in (st.safeW, st.carryW):
            armPose(self.model, theta0, st.retreatR, W)
        for W in (Wtop - st.grip + st.slideRaise, st.safeW):
            armPose(self.model, theta0, R - st.slide, W)
    except ReachError as err:
        return str(err)
    return None

def redraw(self):
    self.updateCarryText()
    if self.result is None or self.live:
        return
    self.showImage(self.background)
    self.canvas.delete("overlay")
    settings = self.motionSettings()
    for i, (stamp, (x, y)) in enumerate(zip(self.result["stamps"], self.pixels)):
        x, y = x * self.scale, y * self.scale
        bad = settings is None or self.reachProblem(stamp, settings) is not None
        color = COLORS["bad"] if bad else COLORS.get(stamp.get("row"), COLORS["lower"])
        r = 9
        self.canvas.create_oval(x - r, y - r, x + r, y + r, outline=color, width=2,
```

```
                        tags="overlay")
        if bad:
            self.canvas.create_line(x - r, y - r, x + r, y + r, fill=color, width=2,
                            tags="overlay")
        if i == self.armAt:
            self.canvas.create_oval(x - 5, y - 5, x + 5, y + 5, fill=COLORS["arm"],
                            outline="", tags="overlay")
        if i == self.selected:
            r = 15
            self.canvas.create_oval(x - r, y - r, x + r, y + r, outline=COLORS["selected"],
                            width=3, tags="overlay")
    self.drawJobOverlay()
    self.updateSelectedText()

def drawJobOverlay(self):
    """Paper edge (dashed, from its corner along +V), paper spot (cross) and ink pad."""
    def toCanvas(X):
        return [v * self.scale for v in worldToPixel(self.result, X)]
    Wp = self.job["paperHeight_mm"]
    cU, cV = paperCorner(self.job)
    edge = toCanvas([cU - 30, cV, Wp]) + toCanvas([cU, cV, Wp]) + toCanvas([cU, cV + 200, Wp])
    self.canvas.create_line(*edge, fill=COLORS["paper"], width=2, dash=(6, 3), tags="overlay")
    spot = self.paperSpot()
    if spot is not None:
        x, y = toCanvas(spot)
        self.canvas.create_line(x - 10, y, x + 10, y, fill=COLORS["paper"], width=2,
                        tags="overlay")
        self.canvas.create_line(x, y - 10, x, y + 10, fill=COLORS["paper"], width=2,
                        tags="overlay")
        self.canvas.create_text(x + 12, y - 12, text="paper", fill=COLORS["paper"],
                        anchor="sw", tags="overlay")
    plan = self.wordPlan(log=lambda _text: None)
    if plan is not None:
        for stamp, X in zip(*plan):
            x, y = toCanvas(X)
            self.canvas.create_rectangle(x - 4, y - 4, x + 4, y + 4, outline=COLORS["paper"],
                                width=2, tags="overlay")
            self.canvas.create_text(x - 6, y + 6, text=stamp["name"], fill=COLORS["paper"],
                            anchor="ne", tags="overlay")
    if self.inkPad is not None:
        x, y = toCanvas(self.inkPad)
        self.canvas.create_oval(x - 12, y - 12, x + 12, y + 12, outline=COLORS["ink"],
                        width=2, tags="overlay")
        self.canvas.create_oval(x - 3, y - 3, x + 3, y + 3, fill=COLORS["ink"], outline="",
```

```
                        tags="overlay")
        self.canvas.create_text(x + 14, y - 14, text="ink", fill=COLORS["ink"], anchor="sw",
                        tags="overlay")
    self.updateJobText()

def paperSpot(self):
    """(U, V, paper W) of the stamp spot from the entries; None if they aren't numbers."""
    try:
        return (float(self.jobVars["spotU"].get()), float(self.jobVars["spotV"].get()),
                self.job["paperHeight_mm"])
    except ValueError:
        return None

def wordPlan(self, log=None):
    """(stamps, paper spots) of the word in the entry; None if it's empty or something is
    wrong, after passing why to log (default: the GUI log)."""
    log = log or self.log
    if self.result is None:
        log("Load a stamp holder result first.")
        return None
    try:
        stamps = parseWord(self.wordVar.get(), self.result["stamps"])
    except ValueError as err:
        log(f"Word: {err}.")
        return None
    if not stamps:
        log("Word: type the stamps in order (e.g. L3 U7 L3), or pick 'add to word' and "
            "click them on the photo.")
        return None
    spot = self.paperSpot()
    try:
        advance = float(self.jobVars["advance"].get())
    except ValueError:
        spot = None
    if spot is None:
        log("Word: paper spot U, V and the letter spacing must be numbers.")
        return None
    if len(stamps) > 1 and advance == 0:
        log("Word: set the letter spacing, or every letter lands on the same spot.")
        return None
    return stamps, wordSpots(spot, advance, len(stamps))

def jobSettings(self, quiet=False):
    """JobSettings from the entries; None (and a log line unless quiet) if one is bad."""
```

```
    log = (lambda _text: None) if quiet else self.log
    v = {key: var.get() for key, var in self.jobVars.items()}
    try:
        job = JobSettings(float(v["inkDepth"]), int(v["dabs"]), float(v["paperDepth"]),
                          float(v["approach"]), float(v["dwell"]), float(v["pressSpeed"]),
                          float(v["rock"]), int(v["rocks"]))
    except ValueError:
        log("Stamp job: depths, approach, dwell, speed and rock must be numbers, presses "
            "and rocks whole numbers.")
        return None
    if (job.dabs < 1 or job.approach <= 0 or job.dwell < 0 or job.pressSpeed <= 0
            or job.rock < 0 or job.rocks < 0):
        log("Stamp job: need presses >= 1, approach > 0, dwell >= 0, press speed > 0, "
            "rock and rocks >= 0.")
        return None
    return job

def updateJobText(self):
    lines = []
    table = self.model.baseTable
    st = self.motionSettings()
    job = self.jobSettings(quiet=True)
    depths = {"Ink pad": job and job.inkDepth, "Paper spot": job and job.paperDepth}
    for name, target in (("Ink pad", self.inkPad), ("Paper spot", self.paperSpot())):
        if target is None:
            lines.append(f"{name}: -")
            continue
        U, V, W = target
        theta0 = self.model.baseAngleOf(U, V)
        if theta0 is None:
            lines.append(f"{name}: [{U:.1f}, {V:.1f}, {W:.1f}] too close to the base")
            continue
        lines.append(f"{name}: [{U:.1f}, {V:.1f}, {W:.1f}]\n"
                     f"  reach {reachOf(self.model, U, V):.1f}, theta0 {theta0 % 360:.1f} deg")
        problem = st and job and pressProblem(self.model, target, depths[name], st, job)
        if problem:
            lines.append(f"  CAN'T REACH the press: {problem}")
        if table and not table[0][0] <= theta0 % 360 <= table[-1][0]:
            lines.append(f"  outside the base calibration (theta0 {table[0][0]:g}-"
                         f"{table[-1][0]:g}): may land off by a few mm")
    spot = self.paperSpot()
    cU, cV = paperCorner(self.job)
    if spot is not None and (spot[0] > cU or spot[1] < cV):
        lines.append("Paper spot is OFF the paper (paper is U < "
```

```
                f"{cU:g}, V > {cV:g})")
    why = []
    plan = self.wordPlan(log=why.append) if self.wordVar.get().strip() else None
    if plan is not None:
        stamps, spots = plan
        U, V, _ = spots[-1]
        lines.append(f"Word: {' '.join(s['name'] for s in stamps)} ({len(stamps)} letters), "
                f"last at [{U:.1f}, {V:.1f}]")
        for stamp, target in zip(stamps, spots):
            problem = ((st and self.reachProblem(stamp, st))
                    or (st and job and pressProblem(self.model, target, job.paperDepth,
                                                    st, job)))
            if problem:
                lines.append(f"  CAN'T DO {stamp['name']} at V {target[1]:.1f}: {problem}")
                break
            if target[0] > cU or target[1] < cV:
                lines.append(f"  {stamp['name']} at V {target[1]:.1f} is OFF the paper")
                break
    elif why:
        lines.append(why[0])
    self.jobInfoVar.set("\n".join(lines))

def updateSelectedText(self):
    if self.selected is None:
        self.selectedVar.set("Click a stamp on the photo.")
        return
    stamp = self.result["stamps"][self.selected]
    U, V, W = stamp["top_UVW_mm"]
    text = (f"{stamp['name']}: top [{U:.1f}, {V:.1f}, {W:.1f}] mm\n"
            f"reach {reachOf(self.model, U, V):.1f} mm, "
            f"base angle {self.model.baseAngleOf(U, V):.1f} deg")
    settings = self.motionSettings()
    problem = settings and self.reachProblem(stamp, settings)
    if problem:
        text += f"\nCAN'T REACH: {problem}"
    self.selectedVar.set(text)

def onClick(self, event):
    if self.result is None or self.live or self.matlabBusy:
        return
    x, y = event.x / self.scale, event.y / self.scale
    if self.clickModeVar.get() == "paper":
        U, V = pixelToWorld(self.result, x, y, self.job["paperHeight_mm"])
        self.jobVars["spotU"].set(f"{U:.1f}")
```

```
            self.jobVars["spotV"].set(f"{V:.1f}")
            self.log(f"Paper spot set to [{U:.1f}, {V:.1f}] mm.")
            self.redraw()
            return
        dist =[math.hypot(px - x, py - y) for px, py in self.pixels]
        nearest = min(range(len(dist)), key=dist.__getitem__)
        if dist[nearest] * self.scale > PICK_RADIUS_px:
            return
        if self.clickModeVar.get() == "word":
            self.wordVar.set(f"{self.wordVar.get().strip()} "
                             f"{self.result['stamps'][nearest]['name']}".strip())
            self.redraw()
            return
        self.selected = nearest
        self.redraw()
        if not self.moveOnClickVar.get():
            return
        if self.holdingVar.get():
            self.putDown()
        elif self.notebook.select() == str(self.jobTab):
            self.runStampJob()              # Stamp job tab open: a click stamps that letter
        else:
            self.pickAndReturn()

    # ------------------------------------------------------------------ motion
    def selectedStamp(self):
        if self.selected is None:
            self.log("Select a stamp first.")
            return None
        return self.result["stamps"][self.selected]

    def stampEvents(self, stamp):
        """Callbacks for the route's events: keep "Carrying a stamp" in step with the arm."""
        def gripped():
            self.holdingVar.set(True)
            self.log(f"   picked up {stamp['name']}")

        def released():
            self.holdingVar.set(False)
            self.log(f"   put down {stamp['name']}")
        return {"gripped": gripped, "released": released}

    def pickAndReturn(self):
        stamp, index = self.selectedStamp(), self.selected
```

```
    if stamp is None:
        return
    if self.holdingVar.get():
        self.log("Already carrying a stamp: Put down first (or untick Carrying).")
        return
    self.runMotion(f"pick up and return {stamp['name']}",
                   lambda s, st: pickAndReturnLegs(self.model, s, stamp, st),
                   lambda: setattr(self, "armAt", index), self.stampEvents(stamp))

def pickUp(self):
    stamp, index = self.selectedStamp(), self.selected
    if stamp is None:
        return
    if self.holdingVar.get():
        self.log("Already carrying a stamp: Put down first (or untick Carrying).")
        return
    self.runMotion(f"pick up {stamp['name']}",
                   lambda s, st: pickLegs(self.model, s, stamp, st),
                   lambda: setattr(self, "armAt", index), self.stampEvents(stamp))

def putDown(self):
    stamp, index = self.selectedStamp(), self.selected
    if stamp is None:
        return
    if not self.holdingVar.get():
        self.log("Not carrying a stamp: Pick up first (or tick Carrying).")
        return
    self.runMotion(f"put down in {stamp['name']}",
                   lambda s, st: putLegs(self.model, s, stamp, st),
                   lambda: setattr(self, "armAt", index), self.stampEvents(stamp))

def retreat(self):
    holding = self.holdingVar.get()       # read here: the legs are made on the worker thread
    self.runMotion("retreat", lambda s, st: retreatLegs(
        self.model, s, st.carryW if holding else st.safeW, st.retreatR))

def hoverOver(self):
    stamp, index = self.selectedStamp(), self.selected
    if stamp is None:
        return
    if self.holdingVar.get():
        self.log("Carrying a stamp: Put down before hovering (or untick Carrying).")
        return
    try:
```

```
            hover = float(self.hoverVar.get())
        except ValueError:
            self.log("Hover must be a number.")
            return
        U, V, Wtop = stamp["top_UVW_mm"]
        try:
            target = atStamp(self.model, U, V, Wtop + hover)
        except ReachError as err:
            self.log(f"Can't hover over {stamp['name']}: {err}")
            return

        def arrived():
            self.armAt = index
            self.hoverAt = {"stamp": stamp, "pose": target, "s1": target["s"][BASE],
                            "dR": 0.0, "s23": tuple(target["s"][1:])}
            self.updateJogText()
        self.runMotion(f"hover over {stamp['name']}",
                       lambda s, st: hoverLegs(self.model, s, stamp, hover, st), arrived)

    def jogBase(self, step):
        if self.hoverAt is None:
            self.log("Hover over a stamp first: the jog starts from there.")
            return
        if self.armBusy:
            self.log("The arm is still moving.")
            return
        self.hoverAt["s1"] += step
        self.updateJogText()
        if self.arm is None:
            return
        # Like the travel moves: always come down onto the new angle from BASE_OVERSHOOT_deg
above.
        s1, (s2, s3) = self.hoverAt["s1"], self.hoverAt["s23"]
        self.jogMoves([[s1 + BASE_OVERSHOOT_deg, s2, s3], [s1, s2, s3]])

    def jogReach(self, step):
        """Move the EE step mm out (+) or in along the arm at the hover height, base unchanged."""
        h = self.hoverAt
        if h is None:
            self.log("Hover over a stamp first: the jog starts from there.")
            return
        if self.armBusy:
            self.log("The arm is still moving.")
            return
```

```
    pose = h["pose"]
    try:
        moved = armPose(self.model, pose["theta0"], pose["R"] + h["dR"] + step, pose["W"])
    except ReachError as err:
        self.log(f"Can't jog the reach: {err}")
        return
    h["dR"] += step
    h["s23"] = tuple(moved["s"][1:])
    self.updateJogText()
    if self.arm is not None:
        self.jogMoves([[h["s1"], *h["s23"]]])

def jogMoves(self, targets):
    """Run the jog gotos in a worker thread."""
    arm = self.arm
    self.armBusy = True
    self.abort.clear()

    def work():
        try:
            for target in targets:
                arm.moveTo(target, abort=self.abort)
        except ReachError as err:
            if str(err) != "stopped":
                arm.send("stop")
            self.post(self.log, f"-- jog: {err}")
        finally:
            self.post(setattr, self, "armBusy", False)
    threading.Thread(target=work, daemon=True).start()

def updateJogText(self):
    h = self.hoverAt
    if h is None:
        self.jogVar.set("Hover over a stamp, then jog base and reach until the EE is centred.")
        return
    delta = h["s1"] - h["pose"]["s"][BASE]
    self.jogVar.set(f"{h['stamp']['name']} at W {h['pose']['W']:.1f}: base servo {h['s1']:.2f} "
                    f"(commanded {h['pose']['s'][BASE]:.2f}, jog {delta:+.2f} = "
                    f"{delta / self.model.baseA:+.2f} deg of theta0)\n"
                    f"reach jog {h['dR']:+.1f} mm (+ = out)")

def recordOffset(self):
    h = self.hoverAt
    if h is None:
```

```
            self.log("Hover over a stamp and jog it centred first.")
            return
        pose, cmd = h["pose"], h["pose"]["s"][BASE]
        new = not OFFSETS_FILE.exists()
        with open(OFFSETS_FILE, "a", newline="") as f:
            w = csv.writer(f)
            if new:
                w.writerow(["time", "result", "stamp", "U_mm", "V_mm", "W_mm", "R_mm", "theta0_deg",
                            "s1_cmd_deg", "s1_centred_deg", "jog_deg", "reachJog_mm",
                            "reachCorrection_mm", "baseA", "baseB"])
            w.writerow([datetime.datetime.now().isoformat(timespec="seconds"), self.resultName,
                        h["stamp"]["name"], f"{pose['U']:.2f}", f"{pose['V']:.2f}",
                        f"{pose['W']:.2f}", f"{pose['R']:.2f}", f"{pose['theta0']:.3f}",
                        f"{cmd:.2f}", f"{h['s1']:.2f}", f"{h['s1'] - cmd:+.2f}", f"{h['dR']:+.1f}",
                        f"{self.model.reachCorrectionAt(pose['W']) + self.model.reachTableAt(pose['theta0'],
pose['R']):.2f}",
                        self.model.baseA, self.model.baseB])
        self.log(f"Recorded {h['stamp']['name']} at W {pose['W']:.1f}: base servo "
                 f"{h['s1'] - cmd:+.2f}, reach {h['dR']:+.1f} mm in {OFFSETS_FILE.name}")

    def park(self):
        """Retreat, turn the base to its home angle, then go to the firmware's home pose."""
        if self.holdingVar.get():
            self.log("Carrying a stamp: Put down before parking (or untick Carrying).")
            return
        self.runMotion("park", lambda s, st: parkLegs(self.model, s, st, self.homePose))

    # ------------------------------------------------------------------ stamp job
    def runStampJob(self):
        """Selected stamp: pick up, ink, press on the paper spot, put back, park."""
        stamp = self.selectedStamp()
        if stamp is None:
            return
        if self.holdingVar.get():
            self.log("Already carrying a stamp: Put down first (or untick Carrying).")
            return
        if self.inkPad is None:
            self.log(f"No ink pad: tag {self.job['inkTagId']} isn't in this result. Take a "
                     "photo with it in view.")
            return
        job, spot, ink = self.jobSettings(), self.paperSpot(), self.inkPad
        if job is None or spot is None:
            if spot is None:
                self.log("Paper spot U and V must be numbers.")
```

```
            return
        events = self.stampEvents(stamp)
        events["inked"] = lambda: self.log(f"   inked {stamp['name']}")

        def stamped():
            self.log(f"   stamped {stamp['name']} at [{spot[0]:.1f}, {spot[1]:.1f}]")
            try:
                advance = float(self.jobVars["advance"].get())
            except ValueError:
                return
            if advance:
                self.jobVars["spotV"].set(f"{spot[1] + advance:.1f}")
                self.redraw()
        events["stamped"] = stamped
        self.runMotion(f"stamp job {stamp['name']}",
                       lambda s, st: stampJobLegs(self.model, s, stamp, ink, spot, st, job,
                                                  self.homePose),
                       events=events, pressSpeed=job.pressSpeed)

    def runStampWord(self):
        """The stamps in the Word entry, in order: each one picked up, inked, pressed on the
        paper (the first at the paper spot, each next one the letter spacing along +V) and
        put back, without parking in between; then park. The whole route is planned (and
        every pose checked) before the arm moves."""
        if self.holdingVar.get():
            self.log("Already carrying a stamp: Put down first (or untick Carrying).")
            return
        if self.inkPad is None:
            self.log(f"No ink pad: tag {self.job['inkTagId']} isn't in this result. Take a "
                     "photo with it in view.")
            return
        job, plan, ink = self.jobSettings(), self.wordPlan(), self.inkPad
        if job is None or plan is None:
            return
        stamps, spots = plan
        names = [s["name"] for s in stamps]
        advance = float(self.jobVars["advance"].get())   # a number: wordPlan checked it
        events = {}
        for k, (stamp, spot) in enumerate(zip(stamps, spots)):
            for name, fn in self.stampEvents(stamp).items():
                events[f"{name} {k}"] = fn
            events[f"inked {k}"] = lambda s=stamp: self.log(f"   inked {s['name']}")

            def stamped(k=k, spot=spot):
```

```
            self.log(f"   stamped {names[k]} ({k + 1}/{len(names)}) at "
                     f"[{spot[0]:.1f}, {spot[1]:.1f}]")
            # Spot and word = what's left, so after a STOP, Stamp word carries on.
            self.jobVars["spotV"].set(f"{spot[1] + advance:.1f}")
            self.wordVar.set(" ".join(names[k + 1:]))
            self.redraw()
        events[f"stamped {k}"] = stamped
    word = " ".join(names)
    self.runMotion(f"stamp word {word}",
                   lambda s, st: stampWordLegs(self.model, s, stamps, ink, spots, st, job,
                                               self.homePose),
                   lambda: (self.wordVar.set(word), self.redraw()),
                   events=events, pressSpeed=job.pressSpeed)

def hoverInk(self):
    if self.inkPad is None:
        self.log(f"No ink pad: tag {self.job['inkTagId']} isn't in this result.")
        return
    self.hoverTarget("ink pad", self.inkPad)

def hoverPaper(self):
    spot = self.paperSpot()
    if spot is None:
        self.log("Paper spot U and V must be numbers.")
        return
    self.hoverTarget("paper spot", spot)

def hoverTarget(self, name, target):
    """EE (or, when carrying, the stamp face) Hover-above mm over target = (U, V, W)."""
    try:
        hover = float(self.hoverVar.get())
    except ValueError:
        self.log("Hover must be a number.")
        return
    holding = self.holdingVar.get()
    U, V, W = target

    def legs(s, st):
        hang = st.stampHeight - st.grip if holding else 0.0
        return travelLegs(self.model, s, U, V, st.carryW if holding else st.safeW,
                          st.retreatR) + [("descend", atStamp(self.model, U, V, W + hang + hover))]
    self.runMotion(f"hover {hover:g} mm over the {name}"
                   + (" (stamp face)" if holding else ""), legs)
```

```
def runMotion(self, what, makeLegs, onDone=None, events=None, pressSpeed=None):
    """Plan from the arm's current servo angles (home if not connected) and run it in a
    worker thread. makeLegs(startServos, Settings) -> [(label, pose)]. events maps a
    pose's "event" to a callback run (on the Tk thread) once the arm reaches that pose.
    Poses marked "slow" run at pressSpeed deg/s (if given); a pose's "dwell" (s) waits
    there after arriving."""
    if self.armBusy:
        self.log("The arm is still moving. STOP first to change target.")
        return
    settings = self.motionSettings()
    if settings is None:
        return
    try:
        normalSpeed = float(self.speedVar.get())
    except ValueError:
        normalSpeed = self.args.speed
    self.armBusy = True
    self.armAt = self.hoverAt = None
    self.abort.clear()
    arm = self.arm
    events = events or {}
    slow = [False]                  # press speed currently set on the firmware
    at = [""]                       # which move is running, for error messages

    def work():
        try:
            start = [pos for pos, _ in arm.status()] if arm else list(self.homePose)
            route = planRoute(self.model, start, makeLegs(start, settings))
            self.post(self.log, f"-- {what}: {len(route)} moves"
                      + ("" if arm else " (not connected, plan from home only)"))
            for pose in route:
                self.post(self.log, "   " + describe(pose))
            if arm is None:
                return
            previous = None
            for step, pose in enumerate(route, 1):
                at[0] = f" (at move {step}/{len(route)}, {pose['label']})"
                if self.abort.is_set():
                    raise ReachError("stopped")
                if pose["label"] in SETTLE_BEFORE and pose["label"] != previous:
                    time.sleep(SETTLE_s)        # let the arm stop swinging before going down
                previous = pose["label"]
                wantSlow = bool(pressSpeed and pose.get("slow"))
                if wantSlow != slow[0]:
```

```
                arm.send(f"speed {pressSpeed if wantSlow else normalSpeed:g}")
                slow[0] = wantSlow
            arm.moveTo(pose["s"], abort=self.abort)
            if pose.get("dwell"):
                time.sleep(pose["dwell"])
            if pose.get("event") in events:
                self.post(events[pose["event"]])
        self.post(self.log, f"-- done: {what}")
        if onDone:
            self.post(onDone)
    except ReachError as err:
        if arm and str(err) != "stopped":
            try:
                arm.send("stop")
            except ReachError:
                pass
        self.post(self.log, f"-- {what}{at[0]}: {err}")
    finally:
        if arm and slow[0]:
            try:
                arm.send(f"speed {normalSpeed:g}")
            except ReachError:
                pass
        self.post(self.motionDone)
threading.Thread(target=work, daemon=True).start()

def motionDone(self):
    self.armBusy = False
    self.redraw()

def stop(self):
    self.abort.set()
    if self.arm:
        self.arm.send("stop")
        self.log(">> stop")

def setSpeed(self):
    try:
        speed = float(self.speedVar.get())
    except ValueError:
        return
    if speed > 0 and self.arm:
        self.arm.send(f"speed {speed:g}")
        self.log(f">> speed {speed:g}")
```

```
# ------------------------------------------------------------------ serial
def refreshPorts(self):
    ports = list(serial.tools.list_ports.comports())
    labels = [f"{p.device} - {p.description}" for p in ports]
    self.portBox["values"] = labels
    if labels and not self.portVar.get():
        preferred = [l for l in labels if "USB" in l or "JTAG" in l]
        self.portVar.set((preferred or labels)[0])

def toggleConnection(self):
    if self.arm:
        if self.armBusy:
            self.log("STOP the arm before disconnecting.")
            return
        self.arm.close()
        self.arm = None
        self.connectButton.configure(text="Connect")
        self.connStatus.configure(text="Disconnected (plan only)", foreground="#b00020")
        return
    device = self.portVar.get().split(" - ")[0]
    if not device:
        messagebox.showwarning("No port", "Pick a serial port first.")
        return
    try:
        self.arm = Arm(device)
    except serial.SerialException as err:
        messagebox.showerror("Connect failed",
                             f"{err}\n\nIs the PlatformIO monitor or another GUI open?")
        return
    self.connectButton.configure(text="Disconnect")
    self.connStatus.configure(text=f"Connected ({device})", foreground="#1b7f3b")
    self.setSpeed()

# ------------------------------------------------------------------ misc
def post(self, fn, *args):
    """Run fn(*args) on the Tk thread (worker threads must not touch widgets)."""
    self.events.put((fn, args))

def pollEvents(self):
    try:
        while True:
            fn, args = self.events.get_nowait()
            fn(*args)
```

```
        except queue.Empty:
            pass
        self.root.after(50, self.pollEvents)

    def log(self, text):
        self.logText.configure(state="normal")
        self.logText.insert("end", text + "\n")
        self.logText.see("end")
        self.logText.configure(state="disabled")

    def onClose(self):
        self.stop()
        self.live = False
        if self.cap is not None:
            self.cap.release()
        if self.arm:
            self.arm.close()
        self.root.destroy()


def main():
    parser = argparse.ArgumentParser(description=__doc__.split("\n")[0])
    parser.add_argument("result", nargs="?", help="start from this logs/stampholder_*.json")
    parser.add_argument("--arm", default=DEFAULT_ARM_CONFIG, help="per-arm config JSON")
    parser.add_argument("--port", help="serial port, e.g. COM6")
    parser.add_argument("--camera", type=int, default=DEFAULT_CAMERA,
                        help=f"webcam index (default {DEFAULT_CAMERA}; 0 is the laptop's own camera)")
    parser.add_argument("--calibration", default=DEFAULT_WEBCAM_CALIBRATION,
                        help=f"camera calibration (default {DEFAULT_WEBCAM_CALIBRATION})")
    parser.add_argument("--matlab", default=DEFAULT_MATLAB, help="matlab.exe to run the vision")
    parser.add_argument("--hover", type=float, default=DEFAULT_HOVER_mm,
                        help=f"Hover over: tool point above the stamp top, mm (default
{DEFAULT_HOVER_mm:g})")
    parser.add_argument("--grip", type=float, default=GRIP_DEPTH_mm,
                        help=f"tool point below the stamp top to grip it, mm (default {GRIP_DEPTH_mm:g})")
    parser.add_argument("--slide", type=float, default=DEFAULT_SLIDE_mm,
                        help=f"slide in along the arm to release, mm (default {DEFAULT_SLIDE_mm:g})")
    parser.add_argument("--slide-raise", type=float, default=SLIDE_RAISE_mm,
                        help=f"slide this far above the grip height, mm (default {SLIDE_RAISE_mm:g})")
    parser.add_argument("--retreat", type=float, default=DEFAULT_RETREAT_mm,
                        help=f"reach to pull in to before turning, mm (default {DEFAULT_RETREAT_mm:g})")
    parser.add_argument("--speed", type=float, default=DEFAULT_SPEED_degps,
                        help=f"move speed, deg/s (default {DEFAULT_SPEED_degps:g})")
    args = parser.parse_args()
```

```
    cfg = loadArmConfig(args.arm)
    joints, _ = loadJointConfig()
    model = ReachModel(cfg, Kinematics(joints, loadCalibration(cfg["servoCalibration"])))
    root = tk.Tk()
    StampHoverGui(root, args, model)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Chat history with Claude

Conversation
##########################
conversationId: c5418dec-1290-5ecc-a5e0-8c150f6b588a
endpoint: bedrock
title: Opening Python GUI Files
exportAt: 16:56:42 GMT-0400 (Eastern Daylight Time)

Options
##########################
presetId: null
modelLabel: Anthropic Claude Haiku 4.5
iconURL: https://dgddob53drbcb.cloudfront.net/claude.svg
spec: Bedrock Anthropic Claude Haiku 4.5
model: us.anthropic.claude-haiku-4-5-20251001-v1:0
endpoint: bedrock
endpointType: bedrock
title: Opening Python GUI Files

History
##########################
>> User:
How do I open a GUI that was created in python if I have the .py file?

>> Anthropic Claude Sonnet 5.5:
Thinking:
This is a simple question about running a Python GUI script, so I'll give concise steps covering the common operating systems.

>> Anthropic Claude Sonnet 5.5:
Running a Python GUI from a .py file

A Python GUI (Tkinter, PyQt, PySide, Kivy, wxPython, etc.) is just a Python script, so you run it like any other.

1. Make sure Python is installed
Open a terminal (Command Prompt/PowerShell on Windows, Terminal on macOS/Linux) and check:
```
python --version
```
On macOS/Linux you may need `python3 --version`. If it isn't installed, get it from python.org. On Windows, check "Add Python to PATH" during install.

2. Install the GUI library (if needed)
- **Tkinter** is usually bundled with Python. On some Linux systems: `sudo apt install python3-tk`
- **PyQt/PySide/Kivy/wxPython** need installing:
  ```
  pip install PyQt5
  ```
  (or `PyQt6`, `PySide6`, `kivy`, `wxPython`)

To see which one it uses, open the file in a text editor and look at the `import` lines at the top. If there's a `requirements.txt`, run:
```
pip install -r requirements.txt
```

3. Run the file
Navigate to the folder and run it:
```
cd path/to/folder
python yourfile.py
```
(Use `python3` on macOS/Linux if needed.) The window should appear.

Other ways to run it
- **Double-click** the .py file. This works on Windows if Python is associated with .py files, but the window may close immediately if there's an error, so the terminal is better for troubleshooting.
- **IDE:** Open it in VS Code, PyCharm, or IDLE and press the Run button (or F5 in IDLE).

Common problems
Issue	Fix

`ModuleNotFoundError: No module named 'X'`	`pip install X`
`'python' is not recognized`	Reinstall Python with PATH checked, or try `py yourfile.py` on Windows
Window flashes and closes	Run from the terminal to see the error message
Nothing appears, no error	The script may be missing `root.mainloop()` (Tkinter) or `app.exec_()` (Qt) at the end
Multiple Python versions	Use `python -m pip install ...` so the package goes to the right one

Making a standalone app (optional)
If you want something that opens without Python installed, you can package it with PyInstaller:
```
pip install pyinstaller
pyinstaller --onefile --windowed yourfile.py
```
The executable will be in the `dist` folder.

If it doesn't work, tell me your operating system and paste the error message, and I can help narrow it down.

>> User:
Now, I have a matlab program and the python GUI. How can I make them work together in one cohesive program?

>> Anthropic Claude Sonnet 5.5:
Thinking:
I'm thinking through several ways to connect MATLAB with a Python GUI: using the MATLAB Engine API to call MATLAB directly from Python, compiling MATLAB code into a Python package via MATLAB Compiler SDK, running MATLAB as a subprocess or compiled executable, or reimplementing the logic in Python/NumPy/SciPy, with Oct2Py as an Octave-based alternative. I want to give concise code examples for each and flag that the Engine API requires a licensed MATLAB install.

I should also mention Python version compatibility concerns and suggest using threading to keep the GUI responsive while MATLAB runs, then wrap up with a few clarifying questions.

>> Anthropic Claude Sonnet 5.5:
Combining a MATLAB program with a Python GUI

The usual setup is for the Python GUI to be the front end and MATLAB to do the computation behind it. There are several ways to connect them, depending on your situation.

Option 1: MATLAB Engine API for Python (most common)

Python starts MATLAB in the background and calls your functions directly. It requires MATLAB installed on the same machine.

Install (from your MATLAB install folder):
```
cd "matlabroot/extern/engines/python"
python -m pip install .
```
Or, in recent versions: `pip install matlabengine` (the version must match your MATLAB release).

Check the [MATLAB/Python version compatibility table](https://www.mathworks.com/support/requirements/python-compatibility.html), since each MATLAB release supports only certain Python versions.

Use it:
```python
import matlab.engine

eng = matlab.engine.start_matlab()
eng.cd(r"C:\path\to\your\matlab\code", nargout=0)

# Call a MATLAB function: result = myfunc(3, 4)
result = eng.myfunc(3.0, 4.0)
print(result)

eng.quit()
```

Tips:
- Your MATLAB code should be a **function** (`function out = myfunc(a, b)`), not a script, so you can pass inputs and get outputs.
- If a function returns nothing, use `nargout=0`.
- Convert arrays with `matlab.double(my_list)`, or `np.array(result)` to go back to NumPy.
- Start the engine once when the GUI launches, not on every button click, because startup takes several seconds.

Option 2: Compile MATLAB into a Python package
With **MATLAB Compiler SDK**, you package your MATLAB code as a Python library. Users only need the free **MATLAB Runtime**, not a full MATLAB license. This is the best choice if you want to distribute the program to others.

```python
import myMatlabPackage
m = myMatlabPackage.initialize()
```

```
result = m.myfunc(3.0, 4.0)
m.terminate()
```
It requires the Compiler SDK toolbox to build.

Option 3: Run MATLAB from the command line
Simple but slower. Python launches MATLAB in batch mode and exchanges data through files.

```python
import subprocess
subprocess.run(
   ["matlab", "-batch", "run_analysis('input.csv','output.csv')"],
   check=True
)
```
Python writes the input file, MATLAB processes it, and Python reads the output. It works with any version and is easy to debug, but each call starts MATLAB, which is slow.

Option 4: Compiled MATLAB executable
Compile your MATLAB program to a standalone `.exe` (with MATLAB Compiler) and call it with `subprocess`, as in Option 3 but without a MATLAB license or startup cost on the user's machine.

Option 5: Port the MATLAB code to Python
If the MATLAB code is mostly math, NumPy, SciPy, and Matplotlib can often replace it. This gives you one language, no license dependency, and the easiest distribution. **Oct2Py** (GNU Octave) is another free route for simple MATLAB code.

Keep the GUI responsive
MATLAB calls that take a while will freeze the window. Run them in a background thread. A Tkinter example:

```python
import threading, tkinter as tk
import matlab.engine

eng = matlab.engine.start_matlab()

def run_matlab():
   result = eng.myfunc(3.0, 4.0)
   label.config(text=f"Result: {result}")

def on_click():
   label.config(text="Running...")
   threading.Thread(target=run_matlab, daemon=True).start()
```

```
root = tk.Tk()
label = tk.Label(root, text="Ready")
tk.Button(root, text="Run", command=on_click).pack()
label.pack()
root.mainloop()
```

(Strictly, updating widgets from a non-main thread is risky in Tkinter, so for larger apps use `root.after()` or a queue. Qt has signals and slots for this.)

Which should you choose?

Situation	Best option
Just you, MATLAB installed	**Engine API** (Option 1)
Sharing with people without MATLAB	**Compiler SDK** (Option 2 or 4)
Quick prototype, any version	**Command line + files** (Option 3)
Math is simple, want independence	**Port to Python** (Option 5)

To give you more specific code, it would help to know:
1. Which GUI library it uses (Tkinter, PyQt, etc.)
2. Whether your MATLAB code is a script or functions, and what its inputs and outputs are
3. Whether others will need to run this without MATLAB
4. Your MATLAB and Python versions

>> User:
Great. Now I have a question. I have to program a robot to pick up stamps one at a time, dip them in an inkpad, press them onto a piece of paper to spell a name, and put them back (including a wiping motion because the stamps are picked up by a permanent magnet). I have the CAD and measurements of the stamp holder and stamps. I have the geometry of the robot arm. This program needs to be customizable because it needs to spell different words with different stamps. How should I start?

>> Anthropic Claude Sonnet 5.5:
Thinking:
I should give a practical, structured plan without overloading it—covering coordinate frames, kinematics/calibration, a stamp library format, task decomposition into pick-dip-stamp-return steps, motion planning with waypoints, a word-to-job compiler, and simulation/testing, then wrap up with clarifying questions about the user's specific setup.

Since the end effector uses a permanent magnet, release likely requires a mechanical stripping motion—sliding the stamp against a fixed ledge or wiper to detach it, rather than switching the magnet off. I should also flag details like mirrored letter orientation, spacing/pitch, pressing force and ink dip

depth, and re-dipping for repeated letters, then ask the user to confirm their robot type and setup before finalizing the answer.

>> Anthropic Claude Sonnet 5.5:
Where to start

Treat this as a **data-driven pipeline**: a word goes in, a list of stamp jobs comes out, each job expands into a standard sequence of motions, and those motions become joint commands. Build it bottom-up so you can test each layer.

Step 1: Define frames and a single source of truth
Pick one **world frame** (e.g., the robot base) and define fixed positions relative to it:
- Stamp holder (and each slot's position and orientation)
- Inkpad
- Paper (origin and orientation)
- Wipe/release feature
- Home position

Put everything in a config file (YAML or JSON) rather than in code, so you can change words, layouts, and calibration without editing the program:

```yaml
stamps:
  A: {slot_xyz: [0.10, 0.02, 0.05], height: 0.030, face_width: 0.015, pickup_z: 0.052}
  B: {slot_xyz: [0.10, 0.04, 0.05], ...}
inkpad: {center: [0.25, 0.10, 0.02], dip_depth: 0.002}
paper:  {origin: [0.30, -0.10, 0.00], letter_pitch: 0.025, baseline_angle: 0}
```

Your CAD gives you the stamp and slot geometry, so extract those numbers once into this file.

Step 2: Get the robot kinematics working alone
Before anything about stamps, make the arm go where you tell it:
- **Forward kinematics:** joint angles → end-effector pose
- **Inverse kinematics:** desired pose → joint angles, with joint limits and a check for multiple solutions or unreachable points
- Verify against a few known poses, ideally on the real robot

MATLAB's Robotics System Toolbox (`rigidBodyTree`, `inverseKinematics`) can do this from DH parameters or a URDF. Python options include `roboticstoolbox-python` or `ikpy`. If the robot manufacturer provides an SDK or controller with its own IK, use that and just send Cartesian targets.

Step 3: Break the task into a reusable sequence

For each letter, the robot does the same sequence with different parameters:

1. Move above the stamp's slot (**approach pose**)
2. Descend and contact the stamp with the magnet (**pickup**)
3. Lift straight up (**retreat**)
4. Move above the inkpad, lower to the dip depth, lift
5. Move above the target position on the paper, lower, press, lift
6. Return above the slot, lower, and **release**
7. Return home or go to the next letter

Write this as one function, `stamp_letter(letter, target_position)`, that outputs a list of waypoints. Use the approach/retreat pattern (hover above, move vertically) for every interaction, since it avoids collisions with neighboring stamps.

Step 4: Design the release/wipe carefully
A permanent magnet can't be switched off, so releasing a stamp has to be mechanical. The usual approach is to slide the stamp sideways against a fixed ledge or edge so the magnet slides off, or to set the stamp in its slot and pull the magnet away along a path that the slot geometry resists. Think this through with your CAD first, because it determines the end of every letter's sequence. Also check:
- Is the magnetic force enough to hold the stamp during the dip and press but weak enough to wipe off?
- Will the wipe motion smear ink onto the holder? Maybe the stamp should be wiped *after* it's back in the slot, or the ink side should face away.

Step 5: Handle the word-to-job logic
This is where customization comes in:
- Input: a string like `"ANNA"`
- For each character, look up its stamp in the config and compute its paper position: `x = origin + i * letter_pitch`
- Validate: is every letter available? Is the word too long for the paper?
- **Repeated letters** just reuse the same stamp, and it must go through the full dip → press → return cycle each time
- Decide whether spaces are skipped or add extra spacing

Remember that stamps are usually mirrored on the face, and the orientation when pressed must read correctly. Also account for the **rotation** of the stamp as it's held by the magnet, since the pickup orientation determines the print orientation.

Step 6: Simulate before touching hardware
Plot the planned path in MATLAB or Python (matplotlib, or the Robotics Toolbox visualizer). Check for:
- Unreachable poses and joint limit violations
- Collisions with the holder, pad, or table
- Reasonable speeds

Then run the real robot **slowly**, one letter at a time, with the first tests at a height above the paper before actually pressing.

Step 7: Calibrate
Expect your CAD to differ from reality by a few millimeters, which matters when picking up a magnetic stamp from a slot. Do a calibration routine where you jog the robot to the actual slot, pad, and paper positions and record the offsets, and store them in the config. For pressing, a small compliance (a foam backing, a spring-loaded stamp mount, or force limiting) forgives height errors.

Suggested build order
1. Config file and data structures
2. FK/IK verified on the robot
3. Move to a single stamp slot, hover, and return
4. Pick up and put back one stamp (including the wipe/release)
5. Add the dip
6. Add the press on paper
7. Loop over a word
8. Wrap it in the Python GUI (text box for the word, Start/Stop button, status display)

Keep an emergency stop and speed limit in place from step 3 onward.

Questions that would let me get specific
1. What robot is it (brand/model, number of joints), and how do you send it commands (SDK, G-code, ROS, serial, MATLAB toolbox)?
2. How does the wipe work physically? Does the stamp slide off the magnet against a ledge, or something else?
3. Is the stamp holder a grid of slots, a rack, or a rotating carousel?
4. Do you need to optimize for speed, or just reliability?
5. Is there any force or position sensing, or is it open-loop?

>> User:
I should've specified that this project also uses computer vision. The position of the robot in relation to the stamp holder, ink pad, and paper is not predefined, so AprilTags will be used. I already have a program that reads the AprilTags to translate their pixel locations into coordinates. The issue is the locations of the stamp holder, ink pad, and paper are not constant, so I can't put positions into a json. Also, the holder is the shape of a quarter circle, so describing its geometry is not very simple. However, it is exactly circular with constant angle offsets for the stamps. The whole setup, including the robot arm, 2 stamps, and stamp holder are included in a CAD file I have. What file format can you analyze for CAD files?

>> Anthropic Claude Sonnet 5.5:
Thinking:

I don't have file-reading tools here, so I can't open native CAD binaries like SolidWorks files directly, and no attachment is visible. But text-based formats such as STEP, STL (ASCII), OBJ, URDF, or DXF could be pasted and analyzed, though large files would be impractical that way—better to have the user extract relevant data first.

The real design question is the pose-relative approach: define the stamp holder parametrically in its own local frame (radius, angle offset, center), use an AprilTag to get the holder's world pose, then compute each slot's pose via a chained transform. The same logic applies to the pad and paper, calibrated once with a camera-to-robot hand-eye transform, with AprilTag pose estimation (solvePnP) giving full 6D pose given intrinsics and tag size, or a 2D homography if they just need table-plane coordinates. I'd store the local geometry in a JSON config, filled in at runtime with live object poses, and sketch out example code for this.

For the file itself, STEP is the best neutral format for geometry, URDF for kinematics, STL/OBJ for meshes, or just a dimensioned drawing with raw measurements -- native CAD formats like SLDPRT/F3D/IPT/CATPart aren't readable so they'd need to export. I should be upfront that I can't tell if file uploads are supported in this interface, so I'll suggest STEP if attachments work, or pasting text/measurements otherwise, without overclaiming what I can actually access.

>> Anthropic Claude Sonnet 5.5:
CAD formats, and how to handle the moving setup

What I can work with
I can only read **text-based** formats, and I don't see a file attached to this conversation. If your interface lets you upload one, or you paste the contents, these are the options:

Format	Usefulness
STEP (.step/.stp)	Best neutral CAD export. It's ASCII text, but large for assemblies, so I'd only skim it for circles, cylinders, and dimensions.
URDF (.urdf)	Best for the robot arm. It's short XML with joint axes, link offsets, and limits, and it feeds straight into MATLAB or Python IK.
DXF	Good for a 2D top view of the quarter-circle holder, since arcs and radii are explicit.
STL/OBJ	Just triangles. Fine for visualization or collision checks, but not for extracting "radius = 80 mm".
Native files (.sldprt, .sldasm, .f3d, .ipt, .CATPart)	I can't read these. Export to STEP first.

Honestly, you probably don't need to send me the CAD at all. For this project you only need about a dozen numbers, which you can read off in your CAD program with the measure tool:

- Holder arc radius (center to stamp-pickup point)
- Angular spacing between slots, and the angle of the first slot
- Slot height (pickup z) relative to the holder base

- Stamp height and face dimensions
- Offset from the holder's AprilTag to the holder's arc center
- Same for the ink pad and paper tags: tag-to-surface offsets
- Robot DH parameters or a URDF export

If you'd rather extract them automatically, Python's `cadquery` or FreeCAD scripting can load a STEP file and report cylinder radii and positions.

The key idea: define everything in local frames
The moving layout doesn't break the JSON approach. It just changes what goes in it. Store geometry **relative to each object's own AprilTag**, and let vision supply where each object currently is.

```
T_robot_slot(i) = T_robot_camera · T_camera_holderTag · T_holderTag_arcCenter · T_arc_slot(i)
```

- `T_robot_camera`: from a one-time hand-eye calibration (fixed camera) or the current arm pose (camera on the arm)
- `T_camera_holderTag`: measured live by the AprilTag detection
- `T_holderTag_arcCenter`: fixed, from CAD, stored in the config
- `T_arc_slot(i)`: computed from the circle parameters

The quarter-circle holder is easy
Since it's exactly circular with constant angular offsets, you don't describe it as a shape. You describe it with 3 numbers:

```yaml
holder:
  tag_id: 3
  tag_to_arc_center: {xyz: [0.05, 0.00, 0.01], rpy: [0, 0, 0]}   # from CAD
  radius: 0.120
  first_slot_angle_deg: 10
  slot_spacing_deg: 15
  pickup_z: 0.052          # above holder base
stamps:
  A: {slot_index: 0}
  B: {slot_index: 1}
inkpad: {tag_id: 5, tag_to_center: [...], dip_depth: 0.002}
paper:  {tag_id: 7, tag_to_origin: [...], letter_pitch: 0.025}
```

In code, slot *i* is:

```python
```

```
import numpy as np

def slot_pose_in_arc_frame(i, cfg):
    th = np.deg2rad(cfg["first_slot_angle_deg"] + i * cfg["slot_spacing_deg"])
    r = cfg["radius"]
    T = np.eye(4)
    T[:3, :3] = Rz(th)                    # stamp faces along the radial direction
    T[:3, 3] = [r*np.cos(th), r*np.sin(th), cfg["pickup_z"]]
    return T
```

Practical vision points
1. **Use full 6-DoF tag poses** if you can. With camera intrinsics and the tag's physical size, `solvePnP` (or the AprilTag library's pose output) gives position *and* orientation. A pixel-to-table homography only gives 2D position, which is enough for the paper and pad if the table is flat, but the holder's rotation matters for slot angles, so you need the tag's heading at minimum.
2. **Detect once, then freeze.** Capture the tag poses at the start of a run, and again only if something moves. Don't chase live detections mid-motion, since noise can jitter the arm.
3. **Average several frames** to reduce tag pose noise, and consider re-checking the holder tag before each pickup, since that's the tightest-tolerance step.
4. **Tag size and flatness matter.** Pose error grows with distance and shallow viewing angles, so a larger tag, closer camera, or two tags per object improves accuracy.
5. **Final alignment.** AprilTags typically give a few millimeters of error. A slightly compliant magnet pickup and a spring-loaded or foam-backed press forgive most of that.

Next steps
1. Is the camera fixed in the workspace or mounted on the arm?
2. Do you have the robot as a URDF, or just DH parameters?
3. Can you tell me where each AprilTag is mounted relative to its object (holder, pad, paper)?

If you can paste the holder and stamp dimensions (or a STEP/DXF excerpt), I can write the full frame-transform code for you.

>> User:
The camera is fixed in the workspace. I do not have the robot as a URDF. Attached is a bird's eye view of the stamp holder (purple), paper (grey), and ink pad (orange). The stamp holder is actually made up of two, concentric quarter circles. The two squares on the stamp holder's edges are where the april tags will go.

>> Anthropic Claude Sonnet 5.5:
Thinking:

Looking at the bird's eye view, I can see a gray paper rectangle on the left, a robot base in the center with yellow mounting markers, and an arm extending toward the paper with a small gripper near its edge, plus what looks like a stamp holder arranged in concentric rows above.

Though position isn't predefined per the user, the arc's apparent centering on the robot base hints the holder might attach there. The two tag squares at the arc ends could give redundant pose estimates, with their center computed independently and the line between them yielding better orientation than a single tag. The paper and ink pad lack visible tags though—need to ask where tags should go on those, likely corners for the paper and the box base (not the lid) for the ink pad. The robot itself looks like it could be a SCARA-style or planar linkage arm.

The arm appears to be a multi-link mechanism, possibly a pantograph or parallelogram linkage with a vertical stack suggesting Z-lift capability, so I should ask about degrees of freedom rather than guess the mechanism type. Without scale reference I can't determine absolute dimensions, and the two silver stamp-like objects seem to sit in pockets within the grid, though I'm unsure which row they belong to. Counting the pocket grid itself is tricky—it looks like two concentric rows with roughly a dozen or so pockets each, but I can't be confident in an exact count.

I'm thinking the configuration needs to support each row having its own radius, starting angle, spacing, and count independently, since the rows may be staggered or have different pocket counts. The robot base sits centrally with a revolute arm sweeping toward the paper on the left, which seems workable for reach. Since this is a CAD top-down view rather than an actual camera feed, I need to consider that a real fixed camera would need to detect tags before the arm moves to avoid occlusion, and that top-down viewing angles can create pose ambiguity for flat AprilTags due to limited tilt information.

For practical fix, constraining via the known coplanar tag arrangement (homography) resolves flip ambiguity better than tilting the camera. I'm also considering adding a tag directly on the robot base or end-effector to simplify hand-eye calibration by defining the robot frame straight from that tag's pose, and wondering if the yellow corner markers on the base plate are actually tags rather than mounting holes. I want to structure my answer around what I observe, implications, a config sketch, the frame pipeline, measurement priorities, and clarifying questions, plus a short code snippet for estimating the holder's center frame from two tag poses.

I'm settling on a Kabsch-based rigid fit using all eight corner points from both tags rather than just averaging two tag poses, since it's more robust—comparing known CAD corner positions in the holder frame against measured corner positions in the camera frame. Writing fit function...

Since the user's existing pipeline likely returns only 2D table coordinates, I should simplify to a 2D version of the fit (rotation about z only), matching their actual data rather than assuming full 3D poses.

For slot positions within the holder frame, each row's angle and radius determine the pocket location, and the yaw comes from the slot's angular offset; transforming into world coordinates then just applies the holder-to-world rotation and translation. I still need to figure out how specific stamps map to specific slots

-- whether that's a fixed configuration or something that needs detection, given there are only two stamps right now.

>> Anthropic Claude Sonnet 5.5:
What I can see, and how it changes the plan

What the image shows
This is a top view of your CAD, so I'm reading it by eye. Correct me where I'm wrong.

- **Robot:** a base plate in the middle with an arm reaching left toward the paper, ending in a small magnet/end-effector near the paper's edge. The stacked parallel bars look like a linkage arm with a separate vertical axis, but I can't tell the type from this view.
- **Stamp holder:** two concentric rows of square pockets forming a quarter-ring around the top and right of the base. The arc looks roughly centered near the robot base, which would simplify things if it's true by design.
- **Two lighter purple squares** at the ends of the arc, which I take to be the AprilTag plates.
- **Two small silver parts** sitting in pockets, which I assume are your 2 stamps.
- **Ink pad:** an orange clamshell shown opened flat, with a hinge between the two halves.
- **Paper:** a large grey portrait rectangle on the left, with no tag marked yet.

Why the two holder tags help
Two tags at the arc ends give you better data than one:

- The **line between them** gives holder heading much more accurately than a single tag's orientation, because a small tag's rotation estimate is noisy.
- The arc center and every pocket are then fixed by CAD, so you don't measure the pockets at all.

The most robust way to use them is to fit all 8 tag corners at once. Compare the corner positions in the holder frame (from CAD) with the positions your existing program reports in the robot frame, and solve for the best rigid transform. Since the holder sits flat on the table, a 2D fit (rotation about z plus an xy translation) is enough:

```python
import numpy as np

def fit_rigid_2d(P_holder, Q_robot):
    """Find R (2x2), t (2,) so that R @ p + t ≈ q. Inputs are Nx2 arrays."""
    Pc, Qc = P_holder.mean(0), Q_robot.mean(0)
    H = (P_holder - Pc).T @ (Q_robot - Qc)
    U, _, Vt = np.linalg.svd(H)
    d = np.sign(np.linalg.det(Vt.T @ U.T))
    R = Vt.T @ np.diag([1, d]) @ U.T
    return R, Qc - R @ Pc
```

```

Also print the residual error. If it's above a couple of millimeters, a tag was misread, the tag size in the config is wrong, or the CAD tag position is off.

## Describing the two-row holder
Each row is its own arc, and the rows may have different pocket counts, spacing, or staggering. Give each row its own parameters, all measured from the **holder's arc center**, which is the origin of the holder frame:

```yaml
holder:
  tags:                    # from CAD, in holder frame (arc center = origin)
    - {id: 3, center: [0.150, 0.020], yaw_deg: 100, size: 0.040}
    - {id: 4, center: [-0.020, 0.150], yaw_deg: -10, size: 0.040}
  rows:
    outer: {radius: 0.140, first_angle_deg: 5,  spacing_deg: 6.2, count: 14, pickup_z: 0.052}
    inner: {radius: 0.115, first_angle_deg: 8,  spacing_deg: 7.0, count: 12, pickup_z: 0.052}
stamps:
  A: {row: outer, index: 4}
  B: {row: inner, index: 6}
```

(All numbers here are placeholders. Take the real ones from CAD.)

```python
def slot_pose_holder(row, k):
    th = np.deg2rad(row["first_angle_deg"] + k * row["spacing_deg"])
    xy = row["radius"] * np.array([np.cos(th), np.sin(th)])
    return xy, th        # position and radial heading in the holder frame

def slot_pose_robot(row, k, R, t):
    xy, th = slot_pose_holder(row, k)
    yaw_robot = th + np.arctan2(R[1, 0], R[0, 0])
    return R @ xy + t, yaw_robot
```

Stamp-to-pocket assignment lives in the config. With only 2 stamps now, you can simply list their pockets. Adding stamps later is then just adding lines.

The ink pad and paper need tags too
I don't see tag locations on either one. Some suggestions:
```

- **Paper:** two tags near opposite corners (or along one edge) give position and rotation, and the text baseline derives from them. Tags that could be covered by the paper itself should sit on the table beside it, with a fixed offset from the paper corner. That also means paper placement needs to be repeatable relative to the tags.
- **Ink pad:** put the tag on the **base half**, not the lid, since the lid moves. If the lid is open flat, make sure the open angle is repeatable, or use a tag on each half. Also, which half holds the ink? The dip target depends on it.

Vision cautions for this layout
1. **Occlusion:** the arm passes over the holder and ink pad. Detect all tags at the start of a run with the arm at a home pose that clears the camera's view, then freeze the result.
2. **Top-down planar-tag ambiguity:** a camera looking straight down at a flat tag can flip between two pose solutions. Since you're only using the table-plane position and heading, a plane-constrained homography avoids this. Otherwise tilt the camera slightly.
3. **Robot frame:** with a fixed camera, the simplest calibration is a tag on the **robot base plate** (the yellow corner features might even be usable). Then everything is expressed relative to the robot with no separate hand-eye step. Otherwise, jog the end-effector to 3 or more known tag points and fit the transform.
4. **Reach check:** compute whether every slot, the pad, and the paper positions are reachable for the actual detected layout *before* moving, and show a clear GUI error if not.

What I need to continue
1. **Robot type and degrees of freedom.** Is it SCARA-like, a 3-axis planar arm plus Z, or a 6-axis arm? Link lengths and joint limits too. With those I can write the FK/IK in MATLAB or Python, no URDF needed.
2. **How is it commanded?** Joint angles over serial, G-code, a vendor SDK, or something else?
3. **Pocket counts and angles** for each row, plus the pocket pitch. I can't count them reliably from the image.
4. **Tag locations and sizes** on the holder, pad, and paper.
5. **How the magnet releases a stamp.** I see no obvious ledge in the top view, so this is the main thing I'd like to understand.

If you can read off the dimensions from CAD, a quick measurement list would be: arc center to each tag center, tag size, each row's radius, first-pocket angle, pocket spacing, and pocket depth.

>> User:
Arc center to farther ring of stamps: 145.66 mm; Arc center to closer ring of stamps: 125 mm; tag size: 50 mm; arc center to each tag center: 84.75 mm (same for both tags due to symmetry of stamp holder); first-pocket angle: -48.0395 degrees, pocket spacing: 8.007 degrees (same for both inner and outer row); height of tops of stamps once they are placed in outer arc: 48.087 mm, height of tops of stamps once they are placed in inner arc: 34.087 mm; angle of left apriltag on stamp holder: -37.8405 deg; angle of right apriltag on stamp holder: 37.8405 deg. Each arc in the stamp holder has 13 pockets. Unlike in the picture, the stamp holder will be centered so it's symmetric about the world's y axis (assuming in the image I

previously gave you, x is to the right, y is up, and z is out of the page). The robot arm has two joints that rotate in the same plane, and a base joint that can yaw around the world's z axis. A screenshot of the robot is attached.

>> Anthropic Claude Sonnet 5.5:
Thinking:
Looking at the screenshot, I can see this is a base with a servo, vertical plate, and a parallelogram linkage keeping the end effector oriented consistently as the arm moves, with an end plate holding a stamp magnet.

Working through the arc geometry: the pocket angles span roughly symmetric about a center at index 6, with 13 pockets spaced about 8.007° apart totaling near 96°, matching the tag positions at ±37.84°. The angle measurement axis and sign convention (clockwise vs counterclockwise, measured from the symmetry axis) are ambiguous from the data alone, so I should flag this assumption rather than guess silently.

I shouldn't assume a fixed world frame tied to the robot base — the actual positions will come from the AprilTag fit, so I'll treat the symmetric layout as nominal only. For heights, the outer stamp tops sit at 48.087 and inner at 34.087, meaning the magnet's pickup z-height should match whichever row's top surface it's grabbing from.

Looking at the radii, 125 and 145.66 differ by about 20.66mm, confirming the outer row is both farther out and raised higher, matching the stepped appearance in the screenshot. Now I'm computing the tag's x,y position in the holder frame using the 84.75mm radius and ±37.84° angle, working out sin(37.8405°) ≈ 0.6157 via interpolation from the 38° reference value.

Finishing that calculation, I get tag centers at roughly (±52.0, 66.9) mm. But I'm now concerned the tags might sit on a tilted ramp rather than a flat plane — the plates in the screenshot look angled like wedges, which would complicate a simple 2D pose assumption and may require the actual tilt/height from CAD or a full 3D treatment instead.

With just two tag centers, I realize I can still get a 2D rigid fit for the holder's origin and heading: define the holder origin as the midpoint of the two centers offset by 66.93mm along the holder's y-axis, with x_hat along the baseline between right and left tags and y_hat perpendicular to it. The baseline distance should check out to about 103.98mm as a validity sanity check, and tag IDs resolve any left/right ambiguity.

Now shifting to the robot's kinematics — it looks like a parallelogram linkage with shoulder and elbow joints keeping the end plate vertical, plus a base yaw. I'd need the link lengths (shoulder-to-elbow, elbow-to-wrist), the tool offset to the magnet face, shoulder height, base position, joint limits, and servo specs (the blue servos look like SG90s) to set up the IK properly.

Second servo's angle may represent the forearm's absolute orientation rather than relative to the first link, so I'll need calibration. Also the magnet traces a circular arc of radius r around the base yaw axis due to the parallelogram linkage, and I need to account for the wrist's tool offset accordingly.

There's a bigger problem: without a wrist-rotation DOF, the stamp's orientation is tied directly to the base yaw angle, meaning the printed letters would rotate along with φ relative to the paper. This means the pickup heading from the pocket plus base rotation determines final stamp orientation, which could misalign the stamped text unless the pocket geometry is arranged carefully relative to the arc center.

Since each pocket aligns the stamp radially with the arm, the orientation relative to the arm is constant across all pockets, so the stamp's world orientation when pressed only depends on the arm's azimuth at the moment of pressing, plus some fixed offset.

This means for straight horizontal text, letters pressed at different radial positions would each have slightly different azimuths, causing noticeable rotation per letter — I'm estimating roughly 7 degrees per letter at typical paper distances and spacing, which adds up to a significant fan-like tilt across a word. One way around this is to orient the paper so the word follows the arc naturally, letting the rotation become part of the design rather than a flaw.

This is important to flag: without wrist yaw, each letter's printed orientation depends on the base yaw, so options are to accept the rotation on an arc, lay text radially, add a wrist rotation, or choose paper distance to minimize spread. I'm thinking the planner should actually compute the predicted letter rotation and display it in the GUI, and I also need to consider the stamp's held orientation when picking from the pocket.

If the arc center coincides with the robot base, pickup orientation stays consistent across pockets; if offset, each pocket gives a different stamp-to-arm orientation, causing inconsistency. For drop-off to work cleanly, the stamp must be oriented radially relative to the arc center, meaning the arm's azimuth should match the pocket direction -- which only works smoothly if the base is positioned at the arc center. This supports my earlier guess that the arc is roughly centered on the robot base, especially now that the user mentioned the holder is symmetric about world y, implying the base sits at the arc center by design -- though I still need to think through how this interacts with the AprilTag approach.

The holder isn't physically attached to the base, so I need to account for a possible offset: if arc center differs from the base axis, the planner must compute the resulting rotation for both pickup and placement. Since the stamp orientation in the gripper equals pocket heading minus pickup azimuth, returning to the same pocket naturally uses the same azimuth, so that part is self-consistent. What matters for the final letter orientation on paper is the formula combining press azimuth with that relative pickup rotation.

I also need to verify the arm's radial reach against the pocket ring geometry — inner and outer rows sit at specific distances from the arc center — and double check the pickup heights for the stamp tops at those two row heights.

Checking pocket spacing: at r=125mm with 8° steps the arc length between pockets is about 17.5mm, and at r=145.66mm it's about 20.4mm. The magnet tool block, roughly 15mm wide, needs to clear neighboring stamps when descending into a pocket, so I should confirm there's no collision risk at the tighter inner-row spacing.

Now I'm organizing what to deliver: assumptions on angle convention that need confirmation, computed pocket angles and tag positions, a config file, and code for building the holder frame from two tags and verifying pocket placement.

I still need to cover a yaw plus parallelogram two-link planar robot model, IK code with placeholders for link lengths and tool offsets flagged for measurement, servo mapping caveats, the lack of wrist yaw as a key issue, and outstanding questions about tilt/ramp tag points and world axis orientation relative to the holder and robot base. I'm now working out the actual pocket angle values across the 13 positions.

The angles come out nearly symmetric around zero, so I'm checking whether they fit a clean formula of (k-6) times 8.007 degrees, with any residual being negligible given the resulting sub-millimeter error. I also check whether the tag angle lines up with any pocket position (it doesn't), then recompute the tag coordinates in millimeters with more decimal precision using trig identities.

I'm reasoning about which angular convention makes sense: measuring from the +y axis toward +x, since that keeps the left and right tags symmetric about ±37.84°, matching the symmetric layout observed in the image. This means the left tag sits on the -x side, which I should flag for the user to confirm against their own image.

Now I'm writing a Python helper to compute the holder's pose from the two tag positions in the robot frame, computing the vector between the tags, the baseline distance, and deriving the orientation and origin from that.

Still writing pocket geometry function...

Still writing IK function...

I'm solving the planar two-link IK using the law of cosines to get the elbow angle, choosing the elbow-up solution typical for parallelogram linkages, then deriving the shoulder angle and the absolute forearm angle, and considering how the servo angle maps onto these joint angles for this linkage design.

I should note that the actual servo-to-angle mapping will likely need calibration, and if the end-effector has a sideways offset from the arm's plane, an azimuth correction would also be necessary. Since I'm inferring the linkage geometry from the screenshot description rather than verified specs, I'll keep assumptions explicit and flag the rotation point calculation as something worth highlighting prominently in the response.

For the printed heading, if the stamp holder sits centered on the base axis with the stamp placed radially, the offset term vanishes and the printed direction simply tracks the arm's azimuth at the moment of

pressing. For text with a baseline running perpendicular to the arm, the rotation across a word's width can be estimated using the arctangent of lateral offset over radial distance -- for example, a 100mm wide word at a 250mm radius gives roughly ±11° of letter rotation across the word.

Options to handle this include orienting text radially, designing the stamp's letter facing to compensate, adding a wrist rotation servo, or keeping the paper distance large and words short. I also need to check how stamps sit in their pockets -- since they likely rest with inked face down and are picked from the top, pressing works without flipping, so I should verify the stamp footprint matches the pocket width.

I'd suggest a quick offline check computing reachability across all 26 pockets, keeping units in millimeters, and organizing the answer with clear headers plus follow-up questions. I also want to flag a tag-tilt issue: if an AprilTag sits on an inclined ramp, using a planar homography assuming flat table height instead of the true 3D pose would introduce a parallax error proportional to the height difference times the camera's offset-to-height ratio, so the pose should come from a proper 3D solve rather than a flat-plane assumption.

>> Anthropic Claude Sonnet 5.5:
Holder geometry, robot model, and one problem to resolve first

1. Numbers from your measurements

I assumed that pocket and tag angles are measured **from the holder's symmetry axis (+y), positive toward +x**. That is the only reading where ±37.84° is symmetric about y. If your CAD measures them differently, tell me and I'll adjust.

Item	Value
Tag centers (holder frame, arc center = origin)	(±51.99, 66.93) mm
Tag-to-tag distance (built-in sanity check)	**103.98 mm**
Inner / outer row radius	125.00 / 145.66 mm
Pocket angles	(k − 6) × 8.007°, k = 0…12
Pickup height (stamp tops)	inner 34.087 mm, outer 48.087 mm

Your first-pocket angle of −48.0395° implies the last pocket is at +48.0445°, which differs from a perfectly symmetric layout by 0.005°. That is about 0.01 mm, so I used the symmetric form.

Pocket pitch is about 17.5 mm on the inner row and 20.4 mm on the outer row. Your end effector must be narrower than that, with room to spare.

2. Locating the holder from the two tags

Two tag centers give you everything: the line between them is the holder's x-axis, and the arc center sits 66.93 mm "behind" their midpoint.

```python
import numpy as np

# --- holder config (mm, degrees) ---
TAG_R, TAG_ANG = 84.75, 37.8405
TAG_Y = TAG_R * np.cos(np.radians(TAG_ANG))         # 66.93
TAG_BASELINE = 2 * TAG_R * np.sin(np.radians(TAG_ANG))   # 103.98
ROW_R   = {"inner": 125.0, "outer": 145.66}
TOP_Z   = {"inner": 34.087, "outer": 48.087}         # above holder base
SPACING, N_POCKETS = 8.007, 13

def holder_frame(p_left, p_right, tol=2.0):
    """p_left/p_right: tag centers (x, y) in the robot frame, mm."""
    p_left, p_right = np.asarray(p_left, float), np.asarray(p_right, float)
    v = p_right - p_left
    d = np.linalg.norm(v)
    if abs(d - TAG_BASELINE) > tol:
        raise ValueError(f"Tag spacing {d:.1f} mm, expected {TAG_BASELINE:.1f}. "
                         "Check tag IDs, tag size, or camera scale calibration.")
    x_hat = v / d
    y_hat = np.array([-x_hat[1], x_hat[0]])
    origin = (p_left + p_right) / 2 - TAG_Y * y_hat   # arc center
    return origin, x_hat, y_hat

def pocket(row, k, origin, x_hat, y_hat):
    """Returns pocket xy, radial unit vector, and pickup z (above holder base)."""
    th = np.radians((k - 6) * SPACING)
    radial_local = np.array([np.sin(th), np.cos(th)])
    radial = radial_local[0] * x_hat + radial_local[1] * y_hat
    return origin + ROW_R[row] * radial, radial, TOP_Z[row]
```

3. Vision caveats specific to your holder

- **The tag plates look like inclined ramps in your screenshot.** If your pixel-to-coordinate program assumes the tags lie at table height, a tag 15 to 25 mm higher gets shifted by parallax, which can reach several mm toward the image edges. Either calibrate the pixel-to-coordinate mapping at the tag's plane height, or use full 6-DoF tag poses. Please check the tag height and tilt in CAD.
- **Tag yaw:** I'm using only tag *centers*, so this doesn't matter for the holder pose. If you later use tag orientation or corners, I need to know how each tag is rotated on its plate.
- The 103.98 mm check above catches swapped IDs and scale errors immediately.

4. Robot model: yaw plus a planar two-link arm

From the screenshot, this looks like a servo-driven **parallelogram linkage**, similar to a MeArm. If so, the end plate stays vertical regardless of joint angles, and the problem reduces to:

1. **Base yaw** = `atan2(dy, dx)` toward the target.
2. **Planar IK** in the (reach, height) plane, with the tool offset subtracted.

```python
def ik(target, base_xy, z_shoulder, L1, L2, tool_dr, tool_dz, elbow_up=True):
    """
    target: (x, y, z) of magnet face contact point, robot frame, mm.
    tool_dr: horizontal offset from wrist pivot to magnet face (outward +).
    tool_dz: vertical drop from wrist pivot to magnet face (positive = tool below).
    Returns yaw, shoulder angle (from horizontal), forearm angle (relative to upper arm).
    """
    dx, dy = target[0] - base_xy[0], target[1] - base_xy[1]
    yaw = np.arctan2(dy, dx)
    r = np.hypot(dx, dy) - tool_dr
    z = target[2] + tool_dz - z_shoulder
    D = (r*r + z*z - L1*L1 - L2*L2) / (2*L1*L2)
    if abs(D) > 1:
        raise ValueError("Target out of reach")
    q2 = np.arccos(D) * (-1 if elbow_up else 1)
    q1 = np.arctan2(z, r) - np.arctan2(L2*np.sin(q2), L1 + L2*np.cos(q2))
    return yaw, q1, q2
```

Servo mapping: on parallelogram arms, the second servo often commands the forearm's *absolute* angle rather than the relative elbow angle (`q1 + q2`), and servo zero points rarely match your CAD zero. Plan a calibration table of servo degrees against measured geometric angles.

5. Check this before writing more code: letter rotation

With only a base yaw and no wrist rotation, **the stamp's heading on the paper equals the arm's azimuth at the moment of pressing**. If the holder's arc center sits on the base axis, a stamp picked from any pocket is aligned with the arm. Then:

> printed heading = arm azimuth at press (+ constant)

So letters in a straight horizontal line at distance D from the base rotate by `atan(s/D)`, where `s` is the sideways offset. A 100 mm word at D = 250 mm gives about **±11° of rotation across the word**.

Options:

1. **Orient the paper so the text runs radially** (along the arm). Every letter then shares the same azimuth. You'd need to choose how the letters face on the stamps to match.
2. **Accept a fan or arc layout** and compute the rotation, then show it in the GUI.
3. **Add a wrist-rotation servo** (the most flexible, but a mechanical change).
4. **Keep words short and the paper far away** to shrink the angle.

If the holder's arc center is *not* on the base axis, each stamp is also held at a different angle depending on its pocket. It must still go back to the same pocket, but its print angle changes. So the holder's placement relative to the base matters too. Your AprilTag fit will tell you its actual offset.

6. What I need next

1. **Robot dimensions:** L1, L2, shoulder height above the table, base axis location, and the magnet's offset from the wrist pivot (horizontal and vertical, plus any sideways offset).
2. **Servo/controller:** hobby servos via Arduino or similar? What command format and angle range?
3. **Joint limits** and whether the second servo drives relative or absolute forearm angle.
4. **Stamp print orientation:** which way does the letter read relative to the pocket's radial direction?
5. **Your intended paper orientation:** is the text direction radial or sideways relative to the arm?
6. **Tag heights** on the holder ramps, and how you plan to locate the robot base in the camera frame (a tag on the base plate is the simplest).
7. **Release/wipe feature:** I still don't see a ledge in either view. How does the stamp come off the magnet when it's returned?

Once I have items 1 to 3, I can write the full pipeline: tag poses to holder frame to waypoints to IK to servo commands, with a reachability check for all 26 pockets.

>> User:
I actually have a program now, but thank you. Here's a conceptual question: How do I predict the error budget mentioned in this prompt? "An error budget. Before the run you predicted a miss from the reprojection error of your
calibration, the tag corner error through H, the parallax at the disc height, and the residual of your servo fit. Show the prediction next to the measured miss, and say where the difference comes from. (You know the pieces, the question is which one dominates.)" The lab manual for this project is attached for context.

>> Anthropic Claude Sonnet 5.5:
Thinking:
For predicting the error budget, I'd outline converting each pixel-level error source to physical units, propagating it to the commanded target miss, then combining the independent sources in quadrature to find the dominant contributor and compare against measured results. I should also be upfront that actual numbers depend on their specific data, and this fits fine as conceptual help for a lab that allows AI assistance with proper attribution.

Breaking down the sources: calibration reprojection error in pixels converts to millimeters via the local scale factor, though I should flag that RMS reprojection error measured at calibration points may underestimate error at extrapolated regions due to distortion. Tag corner detection noise also needs conversion through the homography, accounting for how averaging four corners into a center reduces variance, and noting that a small base tag's homography could amplify errors when applied over much larger distances.

There's a lever-arm effect worth highlighting: small angular errors in the base tag's corner detection translate into larger positional errors at distant points, since rotation error scales with distance from the reference tag. This likely dominates over simple pixel noise, and since both base and target tags carry their own corner uncertainty, these errors combine. I also want to note that parallax effects at different heights above the table are deterministic rather than random, since they stem from the camera's perspective projection rather than measurement noise.

This parallax displacement is proportional to a point's height times its horizontal distance from the camera, divided by the camera's height — so even small height discrepancies (like a tag sitting on a raised plate) can produce substantial directional bias, potentially tens of millimeters depending on geometry. If I know the camera's height, I can correct for this analytically using the homography decomposition to recover camera pose, leaving only a much smaller residual error tied to uncertainty in the actual height value. Critically, this bias is directional (radiating from the camera's nadir point) and should be added as a vector rather than combined in quadrature with other error sources.

I'm also mapping the servo angle-to-position fit residuals through the arm's Jacobian to estimate tip position error in millimeters, treating yaw as tangential displacement and shoulder/elbow errors as combining radially and vertically — a rough example suggests roughly 1 degree of error at 200mm reach translates to about 3.5mm of tip displacement, before accounting for servo resolution, deadband, backlash, and gravity-induced compliance.

Then I think the cleanest path forward is running a Monte Carlo simulation: perturb tag corner detections with Gaussian noise, recompute the homography, propagate through inverse kinematics and servo fit residuals, then forward through the kinematic chain to get the actual landing point distribution across many trials, which sidesteps a lot of the manual error-combination bookkeeping.

I also want to flag that the measured miss includes error sources outside the formal budget — ruler measurement uncertainty, ambiguity in defining the stamp-face landing point versus letter center, reference frame choice for the base tag, plus unmodeled effects like backlash, servo deadband, link flex under gravity, and CAD-to-actual link length mismatches from fabrication.

To separate systematic from random error, I'd compute residual vectors between measured and predicted positions, decompose them into radial and tangential components, and check whether they show consistent directional bias (pointing to calibration/parallax issues) versus scattered noise (pointing to random effects), while also checking dependence on radius, angle, and which arm was used. My hypothesis is that uncorrected parallax likely dominates, followed by servo calibration residuals and

lever-arm extrapolation effects, with pixel/reprojection noise being smallest — though I should present any numbers as illustrative rather than definitive.

I want to defend the "dominates" claim with concrete evidence: running a Monte Carlo simulation toggling each error source on/off, or empirically rerunning with parallax correction enabled versus disabled and comparing the actual measured miss distances. I'm now working through a worked numerical example — picking a camera height, converting tag size in mm to pixels to get an mm/px scale, then estimating how reprojection noise and corner-detection noise propagate into position error in millimeters, to illustrate roughly how large each contributing term might be in practice.

>> Anthropic Claude Sonnet 5.5:
Predicting the error budget

Convert every error source to the same units (mm at the pen, in the table plane), classify each as random or systematic, combine them, and compare against the measured miss in the same statistic.

Step 1: Put each source in mm at the table

1. Calibration (reprojection) error

- **Input:** RMS reprojection error from your calibration, in px.
- **Conversion:** multiply by the local scale, mm per px at the table. Get it from `H`, or approximately from tag size ÷ tag width in pixels. For example, a 50 mm tag that appears 80 px wide gives 0.625 mm/px.
- **Prediction:** `σ_rep,mm ≈ e_rep,px × mmPerPx`
- **Caveat:** the RMS is over the calibration board's positions. Error is usually worse at the image edges and where the board never went, so treat this as a lower bound.

2. Tag corner error through H

This is more than pixel noise, because `H` is built from the base tag. That tag is only 50 mm wide, but you use `H` hundreds of mm away. A corner error of σ_c px tilts the estimated frame by about `√2·σ_c·mmPerPx / 50 mm` radians. That error is multiplied by the lever arm to the target.

Illustrative numbers: σ_c = 0.5 px gives 0.31 mm per corner, which gives a frame rotation error of about 0.006 to 0.009 rad. At a target 250 mm from the base, that is **about 1.5 to 2 mm**. Averaging the four corners helps the target tag's center (σ/2), but not the lever-arm effect from the base tag.

Also add the target tag's own center noise (small) and the tag-yaw error multiplied by the tag-to-disc offset from Problem 1(a) (small if the offset is short).

3. Parallax at the disc height

This one is **deterministic**, not noise. If `H` is the plane-`W=0` homography and the disc is at height `h`, then a point at horizontal distance `d` from the camera's nadir (the point on the table directly below the camera) appears shifted by:

```
parallax ≈ d · h / (Hcam − h)        (directed radially away from the camera nadir)
```

You can get `Hcam` and the nadir by decomposing `H` with `K`, or by measuring the camera height with a ruler.

- Illustrative: h = 35 mm, d = 250 mm, Hcam = 600 mm gives **about 15 mm**. This is likely your largest term if uncorrected.
- If you use `H_h`, the remaining error is the uncertainty in h: `σ_h · d / Hcam`. That is about 1 mm for σ_h = 2 mm.
- Use the disc height for the press target, since the magnet lands where the letter prints. A tag at a different height than the disc creates a differential parallax, so account for each separately.

4. Servo-fit residual

- **Input:** RMS residual of your θ→s fit, converted to degrees of joint error (σ_θ0, σ_θ1, σ_θ2).
- **Conversion:** propagate through the forward kinematics Jacobian, `δp = J δθ`.
  - Yaw error is tangential: `ρ · σ_θ0` (in rad). 1° at ρ = 200 mm is about 3.5 mm.
  - Shoulder and elbow errors are mostly radial and vertical: `σ_r = sqrt(Σ (J_rj σ_θj)²)`.
- **Caveat:** the fit residual is computed on your calibration points (in-sample), so it understates the real error. It also excludes backlash, deadband, and gravity sag.

Step 2: Combine them correctly

- **Random, independent terms** (corner noise, reprojection, servo residual) combine as root-sum-square: `σ = sqrt(σ₁² + σ₂² + …)`.
- **Systematic terms** (parallax, a known offset) add as vectors, with sign. If you've corrected them, add only the leftover uncertainty.
- Keep **radial and tangential components separate**. Yaw and base-tag rotation errors show up tangentially, while shoulder/elbow and parallax show up mostly radially. This also makes the later diagnosis much easier.
- **Match statistics.** If each in-plane axis has σ, the RMS miss is `σ√2` and the mean miss magnitude is about `1.25σ`. Compare predicted and measured values using the same statistic.

Step 3: A Monte Carlo is the cleanest way to do this

Rather than hand-propagating, simulate the pipeline with noise injected, which also handles the nonlinear parts:

```
for trial in 1..N:
   perturb base-tag and target-tag corners by N(0, cornerSigma_px)
   recompute H, then targetUV_mm
   add parallax (or the residual from heightSigma_mm)
   run IK, then add N(0, fitResidual_deg) to each joint angle
   run FK to get the landed position
   missVec_mm = landed − intended target
report mean/RMS of |missVec|, plus radial and tangential parts, per letter
```

Run it once per letter, since ρ and ϕ change the lever arms. Then **turn each source off in turn**. The one whose removal drops the total most is your dominant term, and you can state that with evidence.

Step 4: Explain the difference between predicted and measured

Your measured miss also includes things the four-term budget doesn't:

- **Kinematic model error:** link lengths or pivot offsets that differ from CAD, which show up as a ρ-dependent radial bias.
- **Backlash, deadband, and gravity sag** in the servos and linkage.
- **Magnet pickup offset:** the stamp may hang a few mm off-center or rotate on the magnet, and the wipe can shift it.
- **Ruler error** (about 0.5 to 1 mm) and how you define "where it landed" on the stamp face.
- **Paper or base-tag placement** if the paper slid.

Diagnostics to say where the gap comes from:

1. **Plot the miss vectors** in (radial, tangential) axes. A consistent direction across letters means a bias (parallax, calibration, model error). Scatter means random noise.
2. **Plot miss vs ρ and ϕ.** Growth with ρ points to a frame rotation or servo angle error. A dependence on position relative to the camera nadir points to parallax.
3. **Compare the two arms.** Same vision and same layout, so any difference in miss is arm-specific: servo fit and link tolerances. That also feeds your section (e).
4. **Do an ablation run.** Run once with `H` and once with `H_h` and compare the measured misses. This is real experimental evidence for the parallax term.

What typically dominates

With the illustrative numbers above:

Source	Uncorrected	Corrected
Reprojection	~0.3 mm	~0.3 mm

Tag corners via H	~1.5–2 mm	~1.5–2 mm
Parallax	~15 mm	~1 mm
Servo residual	~4–5 mm	~4–5 mm
Total	**~16 mm**	**~5–6 mm**

These are placeholders. Use your own numbers. The pattern is that uncorrected parallax tends to dominate, then servo error (via the lever arm), then the base-tag rotation. Pixel-level terms are usually small. If your measured miss matches the corrected total, great. If it's larger, the leftover is usually kinematic model error or servo backlash, which you can support with the radial/tangential plots.

If you paste your reprojection error, tag pixel width, camera height, and servo-fit residuals, I can help you compute the actual numbers. Also, since the lab policy requires you to explain every line, make sure you can justify each term's conversion yourself.

>> User:
I have this program. I'm expecting it to access my plugin webcam but it says no camera available. Can you make it recognize my camera?

>> Anthropic Claude Sonnet 5.5:
Thinking:
I'm looking at the openCamera function which calls cv2.VideoCapture with a DSHOW backend, and the failure message suggests trying a different --camera index. Since the default camera index might be set to 1 (reserving 0 for a laptop's built-in camera), that could fail on a desktop or machine without that specific enumeration.

Other likely culprits include another app already holding the camera (Zoom, Teams, browser), Windows privacy settings blocking desktop app access, driver/USB issues, or a resolution mismatch since the code expects 640x480 specifically. The error could also just be OpenCV's own DSHOW or MSMF warning about an invalid index, and since CAP_DSHOW is Windows-specific, I'm assuming this is a Windows setup given the COM6 reference in the code.

I'm planning to suggest a small diagnostic script that enumerates working camera indexes across backends (DSHOW, MSMF, ANY) and verifies an actual frame read succeeds, plus a patched version of openCamera with that same fallback logic built in.

I'm worried silently falling back to a different camera index could be risky, since the calibration file is tied to a specific physical camera—using the wrong one would give a bad homography despite looking like it works. I'll make the fallback behavior clearly logged and suggest pinning with --camera N, with a note that calibration must match the actual camera in use.

On a desktop without a built-in camera, index 0 is likely the webcam itself, which is probably the real cause here rather than any backend issue. I'll suggest they try `--camera 0` explicitly and ask for the exact error message if that doesn't resolve it, keeping the response focused and actionable.

>> Anthropic Claude Sonnet 5.5:
Fixing "camera not available"

I can't run this on your machine, so I can't confirm the cause. From the code, the most likely one is the camera index.

`openCamera()` opens exactly one index with one backend (`DSHOW`), and the index comes from `DEFAULT_CAMERA` in `move_to_tag.py`. The `--camera` help text says "0 is the laptop's own camera", so the default is probably 1. If your computer has no built-in camera (a desktop, or a laptop with the camera disabled), your plug-in webcam is index 0 and the default index fails. DirectShow can also occasionally fail on a webcam that Media Foundation opens fine.

Step 1: Quick test

Try this first, since it may already work:
```
python tools/stamp_hover_gui.py --camera 0 --port COM6
```
Then try `--camera 2` if that fails.

Step 2: Find what works

Save this as `tools/find_camera.py` and run it. It tries every index with every backend and reports what delivers real frames:

```python
import cv2

BACKENDS = (("DSHOW", cv2.CAP_DSHOW), ("MSMF", cv2.CAP_MSMF), ("ANY", cv2.CAP_ANY))

for index in range(6):
    for name, backend in BACKENDS:
        cap = cv2.VideoCapture(index, backend)
        ok = False
        if cap.isOpened():
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
            for _ in range(10):                 # first frames are often empty
```

```
            ok, frame = cap.read()
        if ok:
            print(f"index {index}  {name:<5}  OK  {frame.shape[1]}x{frame.shape[0]}")
    if not ok:
        print(f"index {index}  {name:<5}  no frames")
    cap.release()
```

Hold your hand in front of each working camera (or show the frame with `cv2.imshow`) to tell which index is the webcam. Then pass that index with `--camera`.

Step 3: Make the GUI robust

Replace the existing `openCamera` method with the code below. It tries your requested index with each backend, then scans the other indexes. It also checks that a real frame comes back, because `isOpened()` can be True on a camera that never delivers images.

Add this above the `StampHoverGui` class:

```python
CAMERA_BACKENDS = (("DirectShow", cv2.CAP_DSHOW), ("Media Foundation",
cv2.CAP_MSMF),
                   ("default", cv2.CAP_ANY))
CAMERA_SCAN_COUNT = 6          # indexes 0..5 are tried if the requested one fails


def tryCamera(index, backend):
    """An opened VideoCapture that really delivers frames, or None."""
    cap = cv2.VideoCapture(index, backend)
    if not cap.isOpened():
        cap.release()
        return None
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, WEBCAM_SIZE_px[0])
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, WEBCAM_SIZE_px[1])
    ok = False
    for _ in range(max(WEBCAM_WARMUP_FRAMES, 5)):      # let auto exposure settle
        ok, frame = cap.read()
    if not ok or frame is None:
        cap.release()
        return None
    return cap
```

And the method inside the class:

```python
    def openCamera(self):
        requested = self.args.camera
        indexes = [requested] + [i for i in range(CAMERA_SCAN_COUNT) if i != requested]
        for index in indexes:
            for name, backend in CAMERA_BACKENDS:
                cap = tryCamera(index, backend)
                if cap is not None:
                    size = (int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
                            int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)))
                    note = "" if index == requested else (
                        f" (NOT the requested index {requested}: check this is the camera "
                        f"you calibrated, then pass --camera {index})")
                    self.log(f"Camera {index} opened with {name}, {size[0]}x{size[1]}.{note}")
                    return cap
        self.log(f"No camera delivered frames (tried indexes 0-{CAMERA_SCAN_COUNT - 1} with "
                 "DirectShow, Media Foundation and default). See the checklist.")
        return None
```

Important: calibration is per camera

The fallback may open a different camera from the one you calibrated, such as the laptop camera instead of the plug-in webcam. `--calibration` holds the intrinsics (`K`, distortion) for one specific camera. Using another camera still produces a picture, but the homography and every stamp position will be wrong. The log message above is there so this can't happen silently. Once you know the right index, pin it with `--camera N`.

If nothing works

1. **Another program holds the camera.** Only one program can open a webcam at a time. Close Zoom, Teams, the Windows Camera app, browser tabs using the camera, and **other Python or MATLAB sessions**. A GUI instance that was killed but is still running in Task Manager will hold it.
2. **Windows privacy settings.** Go to Settings → Privacy & security → Camera. Turn on "Camera access" and "Let desktop apps access your camera". Python is a desktop app and is blocked when this is off.
3. **Device Manager.** Check that the webcam appears under "Cameras" or "Imaging devices" without a warning icon. Try another USB port (preferably one directly on the computer, not a hub), or replug it.
4. **Test outside your code.** If the Windows Camera app can't see it either, the problem is the driver or hardware, not the program.
5. **Resolution.** If the camera opens but the log says `Camera gives WxH, the calibration needs 640x480`, the camera doesn't support 640×480 on that backend. Try another backend by reordering `CAMERA_BACKENDS`, and don't resize the frame to fit, because that would invalidate the calibration.

If it still fails, send me the **exact message** from the log or console (including any `[ WARN ... cap_dshow / cap_msmf ]` lines) and the output of `find_camera.py`, and I can narrow it down.

>> User:
Great. also with the above program, can you tell what defines the origin of the world's coordinate system (in mm)?

>> Anthropic Claude Sonnet 5.5:
Thinking:
I'm checking what defines the world coordinate origin -- it looks like Tag 0's center coordinates from the layout config establish the world frame, matching what the docstring and lab manual describe about the base tag anchoring the arm's base frame.

The actual program code doesn't compute this directly; it's set upstream, likely by a MATLAB routine that uses tag 0 to derive the camera pose, with the origin sitting at the base yaw axis on the table (W=0) rather than exactly at the tag center, which is offset according to the layout file.

To verify this without seeing layout.json or the MATLAB source, I'd suggest checking what pixel the origin [0,0,0] maps to using worldToPixel, since the pose data gives world-to-camera transforms where camera center equals -R^T t. The base-angle calibration setup implies that any origin offset relative to the true base axis would introduce measurable error into the reach and angle calculations, so this is worth flagging as a source of uncertainty in the overall error budget rather than treating as settled.

>> Anthropic Claude Sonnet 5.5:
World origin in this program

The code you pasted doesn't set the origin itself. It's set upstream, by tag 0 plus `config/layout.json`, and the origin is meant to be the arm's base axis on the table.

What the code shows

1. **Tag 0 defines the frame.** The docstring of `baseTagCenter()` says so:
   ```python
   def baseTagCenter():
       """Tag 0's centre [U, V] from config/layout.json (it defines the world frame)."""
       return json.loads(LAYOUT_CONFIG.read_text())["baseTag"]["centerUV_mm"]
   ```
   `layout.json` stores where tag 0's center sits in world coordinates, so tag 0 is a known reference point and not necessarily the origin. The paper corner is computed relative to it (`U0 + paperEdgeU_mm`, `V0 + paperCornerV_mm`).

2. **The pose comes from MATLAB.** `locateStampHolder` isn't in this file, but its output is. `result["pose"]` holds `K`, `R` and `t`, a world-to-camera transform. `worldToPixel` and `pixelToWorld` use it, with `xc = R·X + t`. That transform is what fixes the origin and axes, built from tag 0's detected corners and its known position in `layout.json`.

3. **The origin is the base yaw axis.** Several places only make sense if (U, V) = (0, 0) is on the base's vertical rotation axis:
   - `reachOf = sqrt(U² + V² − ds²)`, where `ds` is the tool's sideways offset from the arm plane
   - `phi = atan2(V, U)` for base azimuth, and `model.baseAngleOf(U, V)`
   - The lab manual says the world frame is "the arm's base frame" and the table is W = 0, and its diagram puts the origin at the base axis, with the base tag a bit below it.

Axes and units

- **Units:** mm everywhere in the world frame.
- **U and V:** in the manual's diagram, U points right and V up, in the table plane. Their direction comes from tag 0's orientation, as encoded in `layout.json` and `locateStampHolder`.
- **W:** height above the table (W = 0 is the table). Stamp tops and tag heights (`height_mm`) are measured along it.

What I can't confirm

I can't see `layout.json` or the MATLAB code, so I'm inferring the following rather than reading it:
- The exact value of `baseTag.centerUV_mm`, meaning how far tag 0 is from the base axis
- The exact axis directions

How to check

1. Open `config/layout.json` and read `baseTag.centerUV_mm`. If it is, say, `[0, -80]`, tag 0 is 80 mm behind the base axis along V.
2. Project the origin onto your photo to see where the program thinks it is:
   ```python
   print(worldToPixel(self.result, [0, 0, 0]))
   ```
   It should land on the center of the base's rotation axis. You could also draw it as an overlay in `drawJobOverlay`.
3. Compare against the physical robot. Any offset between the true base axis and (0, 0) turns into error in `baseAngleOf` and `reachOf` for every target. It is a systematic term for your error budget, and it grows as targets get closer to the base.

>> User:

Great, thanks. Now, I've ran the program and taken an image, but at this point my terminal returns the following error. How do I get the program to run?

```
Exception in thread Thread-1 (work):
Traceback (most recent call last):
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\threading.py", line 1218, in
_bootstrap_inner
    self._context.run(self.run)
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\threading.py", line 1160, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\aviva\Documents\2026-27_work\robotics\Robotics_Stamp_Lab\ME416-LAB-2\tools\stamp_h
over_gui.py", line 834, in work
    jsonPath, output = locateWithMatlab(self.args.matlab, self.args.calibration, photoPath)
                       ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\aviva\Documents\2026-27_work\robotics\Robotics_Stamp_Lab\ME416-LAB-2\tools\stamp_h
over_gui.py", line 522, in locateWithMatlab
    run = subprocess.run([matlab, "-batch", matlabCommand(photoPath, calibration)], cwd=REPO_ROOT,
                         capture_output=True, text=True)
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\subprocess.py", line 669, in run
    with Popen(*popenargs, **kwargs) as process:
         ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\subprocess.py", line 1213, in
__init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        pass_fds, cwd, env,
                        ^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
                        gid, gids, uid, umask,
                        ^^^^^^^^^^^^^^^^^^^^^^
                        start_new_session, process_group)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\subprocess.py", line 1719, in
_execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
                           # no special security
                           ^^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
                           cwd,
                           ^^^^
```

startupinfo)

>> Anthropic Claude Sonnet 5.5:
Thinking:
The traceback cuts off before the actual error, but given CreateProcess failing, this is almost certainly a FileNotFoundError (WinError 2) because the MATLAB executable path doesn't exist. I should suggest the user pass the correct path with the --matlab flag pointing to their actual MATLAB installation.

Beyond the path fix, the thread exception means matlabBusy never resets, so the GUI will appear stuck — I should recommend wrapping work() in a try/except so matlabDone always fires even on failure, with locateWithMatlab checking shutil.which or Path.exists first to give a clearer error message. I'll also note that MATLAB's -batch flag requires R2019a or later, and that running with cwd=REPO_ROOT should keep things on the right path, but I'll keep the explanation concise.

>> Anthropic Claude Sonnet 5.5:
Fixing the MATLAB launch error

Your traceback is cut off. The last line, which names the actual exception, is missing. The failure is inside `_winapi.CreateProcess` in `locateWithMatlab`, so Windows couldn't start the program. That is almost always `FileNotFoundError: [WinError 2]`, meaning **the `matlab.exe` path the program is using doesn't exist on your computer**.

The camera and photo worked, since the failure comes after "Saved webcam_....png". The GUI then runs `DEFAULT_MATLAB` from `move_to_tag.py`, which is probably a path from whoever wrote it (a different MATLAB version or install folder).

Step 1: Find your MATLAB

In Command Prompt or PowerShell:
```
where matlab
```
If that prints nothing, look in the usual place:
```
dir "C:\Program Files\MATLAB"
```
You should see a folder such as `R2025a` or `R2025b`. Your executable is then:
```
C:\Program Files\MATLAB\R2025b\bin\matlab.exe
```
Use the folder name you actually have, and check that the file exists.

Step 2: Pass it to the program

```
python tools/stamp_hover_gui.py --matlab "C:\Program Files\MATLAB\R2025b\bin\matlab.exe" --port COM6 --camera 1
```

To avoid typing it every time, open `move_to_tag.py`, find `DEFAULT_MATLAB`, and change it to your path (use a raw string, `r"C:\Program Files\..."`).

Step 3: Make the failure readable (optional but recommended)

As written, any exception in the worker thread kills the thread silently, and `matlabBusy` stays `True`. After that, the GUI says "MATLAB is still locating the last photo" until you restart. Replace the `work` function inside `takePhoto` with:

```python
        def work():
            try:
                jsonPath, output = locateWithMatlab(self.args.matlab, self.args.calibration,
                                                    photoPath)
            except OSError as err:
                jsonPath, output = None, (f"Could not start MATLAB ({err}).\n"
                                          f"Path used: {self.args.matlab}\n"
                                          "Pass the right one with --matlab \"...\\bin\\matlab.exe\".")
            self.post(self.matlabDone, jsonPath, output)
```

Now a bad path shows up in the GUI log, and the Take photo button re-enables.

You can skip retaking photos

The photo is already saved in `photos/webcam_<time>.png`. After fixing the path, you don't have to re-capture, but the GUI only reads results through MATLAB, so just retake it (it takes a few seconds), or run `locateStampHolder` by hand in MATLAB on that file and use **Load result...** on the JSON it produces in `logs/`.

If the error is something else

- **`WinError 2` but the path is right:** `cwd=REPO_ROOT` may be wrong. This is unlikely, since `REPO_ROOT` comes from `servo_angle_gui`.
- **MATLAB starts but no JSON appears:** the GUI then logs MATLAB's own output ("MATLAB did not produce stamp positions"). That points to `locateStampHolder`, the calibration file, or a tag not being seen. Check the log, and make sure tags 0, 1, 3 and the ink pad tag were all shown as "seen" in the live view before you took the photo.

- **`-batch` not recognized:** this needs MATLAB R2019a or newer.

If it still fails, paste the **last 2 or 3 lines** of the traceback (the part after `startupinfo)`), and the log text in the GUI.

>> User:
Does this program utilize a servo_angle_points excel file?

>> Anthropic Claude Sonnet 5.5:
Thinking:
Looking at the imports, I don't see a direct reference to a servo_angle_points Excel file—calibration seems to be loaded through cfg["servoCalibration"] via loadArmConfig and loadCalibration functions instead.

To verify, I'd want to open config/arm_bryson.json and check the servoCalibration field directly, or search the codebase for any references to "servo_angle_points" or Excel-related imports like openpyxl or pandas. I should suggest a grep or findstr command so the user can check for themselves.

>> Anthropic Claude Sonnet 5.5:
Short answer: not directly

Nothing in `stamp_hover_gui.py` mentions `servo_angle_points`, and it has no Excel handling at all: no `.xlsx`, no `openpyxl` or `pandas` import. The only files it reads or writes itself are:

File	Used for
`config/stamp_holder.json`	Holder CAD data
`config/stamp_job.json`	Ink pad and paper offsets, press settings
`config/layout.json`	Tag 0 position
`logs/stampholder_*.json`	MATLAB vision result
`logs/stamp_offsets.csv`	Written by Record offset
the arm config (`config/arm_bryson.json`)	Arm geometry

Where a servo calibration could come in indirectly

The servo calibration is loaded in `main()` by code in other files:

```python
cfg = loadArmConfig(args.arm)
joints, _ = loadJointConfig()
model = ReachModel(cfg, Kinematics(joints, loadCalibration(cfg["servoCalibration"])))
```

```

- `cfg["servoCalibration"]` is a path (or name) stored in the arm config JSON.
- `loadCalibration` comes from `ik_gui.py`, and `loadJointConfig` comes from `servo_angle_gui.py`. I can't see either file.

So if a `servo_angle_points` spreadsheet exists, it would be read, or turned into a calibration file, by one of those helper modules. It probably holds the measured pairs of servo command and joint angle that become your servo fit. I can't tell from what I've seen whether it's an Excel file, a CSV, or a JSON, or whether the program uses it directly or only a fit made from it.

## How to check

From the repo root, in PowerShell:

```powershell
Select-String -Path *.py,*.m,*.json,*.md -Pattern "servo_angle_points" -Recurse
Get-ChildItem -Recurse -Include *servo_angle*,*.xlsx | Select-Object FullName
```

Then open `config/arm_bryson.json` and read the `servoCalibration` value. That tells you which file the model uses for the servo fit, and `loadCalibration` in `ik_gui.py` shows how it's parsed.

Why it matters for your lab

The servo-fit residual is one of the four terms in your error budget. Once you find the calibration file, you can:

- Compute the residual from its data points, which are the measured angles against the fitted ones.
- Check that the calibration matches your physical arm. If the repo's file came from another arm (the config name suggests "bryson"), your servo error would be a systematic offset and not a small residual. The manual also says to compare the two team arms, so each arm needs its own calibration.

If you paste `loadCalibration` and the `servoCalibration` entry from the arm config, I can tell you exactly what it reads and how to compute the residual from it.

>> User:
Here are the two files you mentioned. The issue is the original program was created for someone else's robot arm so the servo calibration is different. I need to load in my own calibration. Where should I put my personal servo_angle_points excel file?

>> Anthropic Claude Sonnet 5.5:
```

Thinking:
I see the calibration file is actually a CSV, not Excel, with a default path under logs/servo_angle_points_corrected.csv. The arm config JSON (config/arm_bryson.json) loads calibration via loadCalibration from a servoCalibration key, so I just need to put the file somewhere and update that config path accordingly.

The CSV needs columns joint, ref_deg, s1_deg, s2_deg, s3_deg, time, with joints 2 and 3 requiring at least two distinct reference angles -- the easiest way to generate it correctly is through servo_angle_gui.py, which autosaves timestamped CSVs into logs/. For running two arms, I'd duplicate arm_bryson.json into arm_aviva.json, point its servoCalibration at the right file, and launch with --arm config/arm_aviva.json.

I should flag that the fit's RMS error is in servo degrees, not joint angle degrees, and this is in-sample fit quality rather than held-out validation. I also need to be upfront that joint 1's base calibration (baseA, baseB, baseTable, reach correction) and the hardcoded link lengths A1=A2=80 aren't captured by this CSV fitting process at all -- those live separately in the arm config and firmware, and I haven't actually seen their config file to confirm specifics.

>> Anthropic Claude Sonnet 5.5:
Where to put your calibration

It has to be a CSV, not an Excel file. `loadCalibration` opens the file with `csv.DictReader`, so an `.xlsx` will fail. In Excel, use File → Save As → CSV.

Where it lives

The code doesn't force a location. The path is only used in two places:

Program	Where the path comes from
`ik_gui.py`	A command-line argument, or the default `logs/servo_angle_points_corrected.csv`
`stamp_hover_gui.py`	`cfg["servoCalibration"]` in the arm config JSON (`config/arm_bryson.json` by default)

`stamp_hover_gui.py` is the one you're running, so the arm config matters most.

Steps

1. **Put your CSV in `logs/`**, for example `logs/servo_angle_points_aviva.csv`. `servo_angle_gui.py` already saves there, and `ik_gui.py` looks there by default.
2. **Copy the arm config** `config/arm_bryson.json` to `config/arm_aviva.json`. Don't edit Bryson's file, since you need a separate one per arm anyway.
3. **Change its `servoCalibration`** to your file:

   ```json
   "servoCalibration": "logs/servo_angle_points_aviva.csv"
   ```
4. **Run from the repo root** with your config:
   ```
   python tools/stamp_hover_gui.py --arm config/arm_aviva.json --port COM6 --camera 1 --matlab "..."
   ```

I can't see `loadArmConfig`, so I don't know whether it resolves that path relative to the repo or to your current directory. `loadCalibration` just calls `open(path)`. If you get `FileNotFoundError`, run from the repo root or use an absolute path in the JSON.

To test the file by itself first:
```
python tools/ik_gui.py logs/servo_angle_points_aviva.csv
```
The "Servo fit" panel shows `a`, `b` and `rms` for joints 2 and 3. If it loads, the stamp GUI will accept it too.

Required CSV format

```
joint,ref_deg,s1_deg,s2_deg,s3_deg,time
2,0,150.0,92.5,60.0,2026-10-14T18:00:00
2,90,150.0,3.0,60.0,...
3,0,...
```

- `joint` must be 2 or 3, because only those get fit (the base isn't in this file).
- `ref_deg` is the link angle measured from +r. Joint 2 is θ1, and joint 3 is the absolute θ2.
- Joints 2 and 3 each need points at **at least two different angles**. More angles (0, 90, 45, and so on) give a better fit and a meaningful residual.
- `s2_deg` and `s3_deg` are the servo angles. `s1_deg` and `time` aren't read by `loadCalibration`, but keep them for consistency.

The easiest way to get this file is to record it with `servo_angle_gui.py`. It writes the correct format automatically to `logs/servo_angle_points_<timestamp>.csv`. If you already have the data in Excel, match those column headers exactly.

Other things that are probably still Bryson's

The CSV only covers the shoulder and elbow fits. Other arm-specific values live elsewhere:

- **Base yaw calibration** (`baseA`, `baseB`, `baseTable`, the reach corrections) comes from the arm config JSON. These need your arm's values too. I haven't seen that file, so check it.
- **Joint limits and home angles** are parsed from `firmware/src/main.cpp`. If your servos' limits differ, that file must match the firmware you flashed.
- **Link lengths** `A1_mm = A2_mm = 80` are hardcoded in `ik_gui.py`. Check them against your build. The two arms may differ slightly, which is relevant to your lab's arm comparison.

For your error budget

Each fit's `rms` is in **servo degrees** and is computed on your calibration points. Divide by `|a|` to get joint-angle error in degrees. As noted before, that is an in-sample residual and understates the true error.

If you paste your arm config JSON, I can tell you which other values need to change.

>> User:
In your earlier error budget analysis, how can I obtain sigma_h?

>> Anthropic Claude Sonnet 5.5:
Thinking:
σ_h comes from the uncertainty in the disc or tag's height above the table, as used in the H_h term. I need to trace this back to CAD values—the stamp height plus seat z-offset from the holder config, or the tag's height_mm—combined with manufacturing tolerances on those dimensions.

Practically, I'd measure several physical stamps with calipers and take the standard deviation, then combine that with assembly tolerances (how the stamp sits in its well, holder flatness, 3D print tolerance, paper thickness) using a uniform-distribution assumption of ±t/√3 where needed. In the actual pipeline, since tag heights feed into the pixel-to-world projection via the pose from MATLAB, this height uncertainty propagates into horizontal position error as σ_h times the ratio of horizontal distance to camera height.

For the measurement procedure, I'd correct any CAD-to-measured bias rather than treat it as noise, then combine tolerance, caliper resolution, and seating variation in quadrature, also checking for holder tilt by measuring at both ends — typical σ_h values land around 0.5–2 mm. Propagating that through gives σ_xy = σ_h·d/(Hcam−h), e.g. 1 mm height error at d=250, Hcam=600 yields about 0.42 mm positional error, consistent with my earlier estimate.

An empirical alternative is to deliberately vary h and observe how the miss distance changes, or better, estimate camera height directly from the pose: since centre = -R.T @ t, I can pull Hcam straight from the pose's centre[2] value rather than measuring it by hand — worth including that as a code snippet, kept concise.

>> Anthropic Claude Sonnet 5.5:
Getting σ_h

σ_h is the **uncertainty in the height you assume for a point** when you convert a pixel into a world position. It's not a camera quantity. Nothing in the vision pipeline measures it. You estimate it from how well you know each object's height.

1. Where h enters in your program

The camera sees a ray, and a point's (U, V) depends on where that ray meets the assumed height. A wrong height shifts the point sideways by about `σ_h · d / (Hcam − h)`.

Object	Height the code assumes	Source
Stamp tops	`top_UVW_mm[2]`	Holder CAD (`stamp_holder.json`) plus holder tag fit
Holder / base / ink tags	`tag["height_mm"]`	Saved in the MATLAB result
Paper spot	`paperHeight_mm`	`stamp_job.json`
Ink pad surface	`height_mm + inkOffsetUVW_mm[2]`	Tag height plus the job offset

Each one gets its own σ_h. Compute the parallax term per object and combine them, rather than using a single number.

2. Estimating σ_h for one height

Combine independent contributions in quadrature:

1. **Manufacturing tolerance.** If a part is "±t" (3D-print or laser-cut tolerance), treat it as uniform: `σ = t/√3`. A ±1 mm tolerance gives 0.58 mm.
2. **Seating variation.** Does the stamp sit at the same height each time, or can it rock in its well? Measure it.
3. **Holder flatness and tilt.** If the plate or holder isn't level, the height varies across it. Measure at both ends of the arc.
4. **Measurement resolution.** If you check the height with calipers or a ruler, add `resolution/√12`. A 1 mm ruler graduation gives 0.29 mm.
5. **Paper and pad compression.** The paper has a thickness, and the ink pad is foam, so its surface height changes under load.

If CAD and reality differ by a consistent amount, that's a **bias** and not a σ. Measure it and correct the config. Only the leftover scatter goes into σ_h.

3. Measure it, which is better than guessing

For the stamps, which matter most:

1. Put 5 to 10 different stamps in their wells and measure each top height above the plate with calipers or a height gauge.
2. Take the mean and compare it with the CAD value. Fix any offset in the config.
3. The sample standard deviation `std(heights, ddof=1)` is your σ_h for stamp-top height. Combine it with the caliper reading error in quadrature.
4. Repeat for the paper surface (measure at the spot where you press) and the ink pad surface.

Realistic values are **0.5 to 2 mm**.

4. Convert it to a miss in mm

You need `d`, the horizontal distance from the point to the camera nadir (the spot on the table directly below the camera), and `Hcam`. Your pose is already in `result["pose"]`, and `pixelToWorld` computes the camera center:

```python
pose = result["pose"]
R, t = np.array(pose["R"]), np.ravel(pose["t"])
centre = -R.T @ t                    # camera position in world frame
Hcam, nadirUV = centre[2], centre[:2]

def parallaxSigma_mm(UV, h, sigmaH):
    d = np.hypot(*(np.asarray(UV) - nadirUV))
    return sigmaH * d / (Hcam - h)
```

Example: σ_h = 1 mm, d = 250 mm, Hcam = 600 mm, h = 35 mm gives about 0.44 mm.

This error is **radial from the nadir**, so keep it as a vector in your radial/tangential budget instead of adding it to the isotropic terms.

5. Two related points

- **Vertical error is a separate effect.** A wrong stamp height also changes how deep the stamp presses (the commanded W). That affects contact and ink pressure, not the (U, V) landing position.
- **The parallax itself isn't σ_h.** If your program uses the right height for each point (via `pixelToWorld(..., W)`, or by projecting with the tag height), the parallax is already corrected. Only the uncertainty in h remains. If a point is treated as table height, the whole `d·h/(Hcam−h)` term is a systematic error, not a σ.

6. Checking it experimentally

Run the same press with the assumed height changed on purpose, for example `paperHeight_mm` ± 3 mm. The landing shift divided by 3 mm should match `d/(Hcam − h)`. This confirms your Hcam and nadir, and gives a measured sensitivity you can cite in the report.

Report the σ_h values you used, how you obtained them, and the resulting mm.

>> User:
I ran python tools/stamp_hover_gui.py --arm config/arm_aviva.json --port COM6 --camera 0 --matlab "..." and got the following error, how do I fix it?

```
Exception in thread Thread-1 (work):
Traceback (most recent call last):
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\threading.py", line 1218, in
_bootstrap_inner
    self._context.run(self.run)
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\threading.py", line 1160, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\aviva\Documents\2026-27_work\robotics\Robotics_Stamp_Lab\ME416-LAB-2\tools\stamp_h
over_gui.py", line 834, in work
    jsonPath, output = locateWithMatlab(self.args.matlab, self.args.calibration, photoPath)
                       ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"C:\Users\aviva\Documents\2026-27_work\robotics\Robotics_Stamp_Lab\ME416-LAB-2\tools\stamp_h
over_gui.py", line 522, in locateWithMatlab
    run = subprocess.run([matlab, "-batch", matlabCommand(photoPath, calibration)], cwd=REPO_ROOT,
                         capture_output=True, text=True)
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\subprocess.py", line 669, in run
    with Popen(*popenargs, **kwargs) as process:
         ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\subprocess.py", line 1213, in
__init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        pass_fds, cwd, env,
                        ^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
                        gid, gids, uid, umask,
                        ^^^^^^^^^^^^^^^^^^^^^^
                        start_new_session, process_group)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "C:\Users\aviva\AppData\Local\Python\pythoncore-3.15-64\Lib\subprocess.py", line 1719, in
_execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
                           # no special security
                           ^^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
                           cwd,
                           ^^^^
                           startupinfo)
                           ^^^^^^^^^^^^
PermissionError: [WinError 5] Access is denied
```

>> Anthropic Claude Sonnet 5.5:
Thinking:
That access denied error suggests the --matlab path given is wrong, likely pointing to a directory or a literal placeholder like "..." rather than the actual matlab.exe executable.

I should recommend verifying the path with Test-Path and running matlab.exe directly in PowerShell to confirm it works, checking there's no trailing backslash before the closing quote that could break the string, and wrapping the subprocess call in error handling so the thread doesn't crash outright.

>> Anthropic Claude Sonnet 5.5:
Fixing `PermissionError: [WinError 5] Access is denied`

This is a different error from the last one. `WinError 2` means "file not found". `WinError 5` means Windows found something at that path but **couldn't run it**. The usual cause is that the `--matlab` path points to a **folder**, not to `matlab.exe`. Windows reports "Access is denied" when you try to execute a directory.

Check what you passed

Your command shows `--matlab "..."`. If you typed `...` literally, that's the problem, because `...` isn't a path to MATLAB. If you replaced it with a real path for the post, check that it matches one of these:

What you passed	Result
`C:\Program Files\MATLAB\R2025b`	Folder, so WinError 5
`C:\Program Files\MATLAB\R2025b\bin`	Folder, so WinError 5
`C:\Program Files\MATLAB\R2025b\bin\matlab.exe`	Correct
`...\bin\matlab.exe\` (trailing backslash)	In cmd, `\"` escapes the quote and mangles the argument

Verify the path in PowerShell

```powershell
where.exe matlab
Test-Path "C:\Program Files\MATLAB\R2025b\bin\matlab.exe"
```
`Test-Path` should print `True`. To confirm it actually runs:
```powershell
& "C:\Program Files\MATLAB\R2025b\bin\matlab.exe" -batch "disp(1)"
```
This should start MATLAB, print `1`, and exit. Replace `R2025b` with the folder name you actually have (`dir "C:\Program Files\MATLAB"`).

Run it with the verified path

```powershell
python tools/stamp_hover_gui.py --arm config/arm_aviva.json --port COM6 --camera 0 --matlab
"C:\Program Files\MATLAB\R2025b\bin\matlab.exe"
```
Use the exact path that worked above, ending in `matlab.exe`.

If the path is right and it still says access denied

1. **The file isn't executable by your account.** Right-click `matlab.exe`, then Properties, then Security, and check that your user has "Read & execute". This is rare on a normal install.
2. **Antivirus or Windows security is blocking it.** Controlled folder access or corporate endpoint software can block child processes launched from Python. Check the Windows Security protection history, or try the same command from a normal PowerShell window, as in the test above.
3. **`matlab.exe` is a different file.** Some installs have a `matlab.exe` stub in `C:\Windows\System32` or a WindowsApps alias. Use the full path from `bin`, not a bare `matlab`.
4. **Pass the path through the config instead.** Set `DEFAULT_MATLAB` in `move_to_tag.py` to `r"C:\Program Files\MATLAB\R2025b\bin\matlab.exe"` so you can't mistype it.

Make this failure visible in the GUI

The thread dies without telling the GUI, which also leaves **Take photo** disabled. Replace `work` inside `takePhoto` with:

```python
    def work():
        try:
            jsonPath, output = locateWithMatlab(self.args.matlab, self.args.calibration,
                                                photoPath)
        except OSError as err:          # includes PermissionError and FileNotFoundError
```

```
            jsonPath, output = None, (f"Could not start MATLAB: {err}\n"
                                      f"Path used: {self.args.matlab!r}\n"
                                      "It must be the full path ending in matlab.exe.")
        self.post(self.matlabDone, jsonPath, output)
```

If MATLAB starts but fails afterward

The GUI then logs "MATLAB did not produce stamp positions" along with MATLAB's own output. Common causes are a missing calibration file, a tag not being seen, or the layout config. Paste that log and I can help.

>> User:
Alright, now the matlab part works but the gui says all the stamp positions are out of reach. How do I fix this? Are there any files it'd be helpful to share with you?

>> Anthropic Claude Sonnet 5.5:
Thinking:
I need to figure out why every stamp position is being flagged unreachable, which means digging into how reachability gets checked—probably the solver call across grip heights and arm pose constraints, or possibly an issue with how stamp coordinates are being transformed into the world frame.

My next suspicion is servo calibration: if Aviva's calibration constants differ from the limits assumed in main.cpp, computed joint angles could fall outside valid servo ranges for every pose, triggering a limit error universally. I'm also considering whether the arm's physical reach (shoulder segment lengths, holder radius, and required safe/carry heights) simply can't cover the needed workspace at all, which would make every position fail regardless of calibration.

The GUI's "CAN'T REACH" message when clicking a stamp is my best diagnostic tool here, since it should surface the actual rejection reason along with computed reach and base angle. I also noticed the earlier joint RMS values (0.0001, 0.0002) look suspiciously tiny, possibly from fitting with only two calibration points — which would make the fit exact but risky to extrapolate — and the memory timestamps seem to be oddly ahead of the current date, which is strange and worth flagging.

My plan is to first check the
(This is an unfinished message)

>> User:
I know the reachability function works, so something must be wrong with the servo calibration.

>> Anthropic Claude Sonnet 5.5:
Thinking:
If all stamp positions are unreachable, that points to the inverse kinematics checks rather than just servo tuning—reach failures could stem from the arm being too far out, too close, or hitting joint limits, and calibration would mainly affect those limit boundaries.

Checking the firmware's limit calculation, if the calibration slope or offset is wrong, that would cause limit violations across the board regardless of actual position. I'm also considering other culprits: arm configuration offsets, the holder's physical placement relative to the base, or an incorrect tag location in the layout file.

Something catches my eye in memory about suspiciously tiny servo fit residuals for joints 2 and 3—numbers that small usually mean either perfectly synthetic data or a fit based on only two calibration points, which would trivially produce near-zero error without actually validating the calibration.

I want to flag this risk without over-attributing it to the user's own past statements, since the memory timestamps are oddly inconsistent with the current date. I think the safest approach is to raise it as a general diagnostic warning—suggesting they check the rms value in the ik_gui panel, since a residual near 0.0001 is a red flag for an underdetermined fit—without claiming it as something they told me before.

I also want to push back gently on the assumption that the reachability function is wrong; it's more likely a calibration or interpretation issue. I should ask for the exact "CAN'T REACH" message text, since the reason string narrows down whether it's a distance, joint limit, or safe-height problem, and suggest they check the ik_gui workspace shading and fit panel to see if the joint angle ranges and sign conventions match the CAD setup.

Beyond that, I want them to verify the home pose actually falls in the elbow-up workspace, double check the calibration CSV's angle convention (absolute vs relative, correct joint column numbers), and confirm firmware joint limits match. Finally, I should suggest printing the actual stamp reach and base angle to compare against the known max/min reach bounds, since the issue could simply be the stamp positions themselves rather than the calibration or reachability logic.

The carry height might exceed what the arm geometry can reach at the shoulder's z-axis, which would explain why every stamp fails the reach check uniformly — it's a height ceiling problem, not a horizontal one. I should look at adjusting the safe travel height entries to fix this.

To diagnose properly I need to request several files: the arm config, calibration CSV header, the ReachModel/solve code in move_to_tag.py, the stamp holder and layout configs, recent stamp logs with top_UVW_mm values, the exact error text, and the ik_gui fit panel output. I'll also write a quick diagnostic script that loads the model like main does and tests solving at different carry heights for each stamp to pinpoint where it breaks.

Writing diagnostic script...

I should also print forward kinematics for the home pose to sanity-check things, and keep the output concise. I want to be careful about overclaiming that the reachability function "works" — it just reports what it computes, which may still be wrong if assumptions are off.

As a calibration check, I'll suggest testing in the IK GUI with a point like (100, 20), and examining whether the fitted theta ranges and scale factor make sense — e.g., for a 180° hobby servo directly driving a link, the scale should be near ±1, and if it's wildly off that signals bad units or swapped data columns.

>> Anthropic Claude Sonnet 5.5:
Diagnosing "all stamps out of reach"

I can't tell yet whether the servo calibration is the cause. The reachability check is probably telling the truth, but "out of reach" can come from at least five different places. The GUI already says which one, so we can narrow it down quickly.

Step 1: Read the exact reason

Click a stamp. The text under the photo says `CAN'T REACH: ...`. The wording points to the cause:

Message	Meaning
`joint 2 limit (servo X)` or `joint 3 limit (servo X)`	The IK found a geometric solution, but your fit maps it outside the firmware's servo range. **This is the calibration signature.**
`out of reach`	The target is farther than the arm's 160 mm (A1 + A2) can extend. Suspect the layout or tag 0 position, not the calibration.
`too close to the shoulder`	The target is inside the inner dead zone.
`safe height ... is below the stamp top`	The Safe travel height field is wrong.

Please paste the exact message for one inner-row and one outer-row stamp.

Step 2: Check the fit itself

Run the fit in isolation:
```
python tools/ik_gui.py logs/servo_angle_points_aviva.csv
```
In the "Servo fit" panel, check:

- **`a` should be close to ±1** for a servo driving a link directly. Values like 0.01 or 57 mean the `ref_deg` column is in the wrong units or the columns are swapped.
- **The θ range** printed for each joint should be physically plausible. A mirrored range means the sign of `a` is wrong.

- **`rms` should not be 0.00.** A fit this perfect usually means you have only two distinct angles, or the data was copied or synthesized. Record at least 4 or 5 angles per joint.
- **The shaded workspace** should be a sensible blob reaching out to about 160 mm. If it is tiny or empty, the calibration is the problem. Also try typing `100` and `20` into "Go to" and see if it solves.

Common calibration mistakes:
- Joint 3's `ref_deg` must be the **absolute** angle from +r, not relative to link 1.
- The `joint` column must contain 2 and 3, not 1 and 2.
- `s2_deg` and `s3_deg` must hold the servo angles for the right joints.

Step 3: Check things that aren't the calibration

1. **Safe and carry height.** `reachProblem` tests the carry height `safeW + (stampHeight − grip)` at each stamp's reach. If that height is higher than the arm can reach, every stamp fails even with a perfect calibration. Test by lowering "Safe travel height W" in the GUI and watching the red crosses.
2. **Where the origin is.** Your stamp positions come from tag 0's location in `config/layout.json`. If that file still holds the other person's values, every stamp is shifted and reach is wrong for all of them. Check that `reachOf` gives about 125–146 mm for your stamps, since the holder radii are 125 and 145.66 mm from the arc center.
3. **Arm config values.** `arm_aviva.json` also holds the shoulder height, tool offsets, and base calibration. If it is a copy of Bryson's, these are still his.
4. **Firmware limits.** Check the first log line when you start `ik_gui.py`. If it says "WARNING: could not read main.cpp", it is using fallback limits.

Step 4: Run this diagnostic

Save it as `tools/diagnose_reach.py` and run it with your latest result:
```
python tools/diagnose_reach.py logs/stampholder_<time>.json
```
```python
import json
import sys
from pathlib import Path

from ik_gui import Kinematics, loadCalibration
from move_to_tag import ReachError, ReachModel, loadArmConfig
from servo_angle_gui import loadJointConfig
from stamp_hover_gui import defaultSafeHeight, loadHolder, reachOf

cfg = loadArmConfig("config/arm_aviva.json")
joints, source = loadJointConfig()
print(source)
for j in joints:
```

```
    print("  ", j)

fits = loadCalibration(cfg["servoCalibration"])
kin = Kinematics(joints, fits)
for idx, fit in fits.items():
    lo = kin.servoToTheta(idx, joints[idx]["min"])
    hi = kin.servoToTheta(idx, joints[idx]["max"])
    print(f"joint {idx + 1}: a={fit['a']:+.4f} b={fit['b']:.2f} rms={fit['rms']:.4f} "
          f"n={fit['n']}  theta range [{min(lo, hi):.1f}, {max(lo, hi):.1f}]")

home = [j["home"] for j in joints]
print("home pose (r, z) from the fit:", kin.fkFromServos(home[1], home[2])[1])

model = ReachModel(cfg, kin)
safeW = defaultSafeHeight(loadHolder())
result = json.loads(Path(sys.argv[1]).read_text())
for stamp in result["stamps"]:
    U, V, Wtop = stamp["top_UVW_mm"]
    out = []
    for label, W in (("grip", Wtop - 15), ("top", Wtop), ("safe", safeW)):
        try:
            model.solve(U, V, W)
            out.append(f"{label} ok")
        except ReachError as err:
            out.append(f"{label}: {err}")
    print(f"{stamp['name']:>4} U={U:7.1f} V={V:7.1f} reach={reachOf(model, U, V):6.1f}  "
          + " | ".join(out))
```

It prints the parsed firmware limits, your fit and its θ range, the home pose, and the exact failure for each stamp at three heights. That should separate a calibration problem from a layout or height problem.

Files to share

In order of usefulness:
1. The output of `diagnose_reach.py` (it may make the rest unnecessary)
2. `config/arm_aviva.json`
3. Your calibration CSV (the first 15 or so lines are enough)
4. `ReachModel`, `loadArmConfig`, and `solve` from `tools/move_to_tag.py`
5. `config/layout.json`
6. The `kJoints` table from `firmware/src/main.cpp`

>> User:
I fixed that. Now the arm goes to the complete wrong place when I tell it to go to a stamp. please fix

>> Anthropic Claude Sonnet 5.5:
Thinking:
Without seeing the actual arm or config, I can't pinpoint the exact bug, but I can walk through the likely sources systematically. The base yaw calibration values from the arm config are probably still set to Bryson's parameters rather than being updated for this setup.

There's also the world origin from layout.json's baseTag position, the physical placement of tag 0 matching that layout, and the camera calibration file — the default is likely for someone else's camera entirely, which would throw off all projected positions. I should also check the stamp holder config and tag spacing, and the servo fit sign/units, then isolate the problem layer by layer: first verify vision overlay accuracy against the photo, then check whether the projected coordinates land correctly.

I'd test kinematics using the GUI by commanding known r,z values and physically measuring with a ruler, then test the base servo rotation against expected azimuth angles. I want to characterize exactly how the error presents — rotated, too far/short, too high/low, or mirrored — and map these symptoms to likely causes in a table, plus request the key config files and logs to pinpoint which stage is broken.

Since camera index was swapped and calibration is per-camera, that's worth flagging again. Without those files I can't diagnose further myself, so I'll write a small script that dumps the full pipeline for a stamp — pixel coordinates, UVW, reach, theta0, and servo pose — using the same model and imports already established.

>> Anthropic Claude Sonnet 5.5:
Arm goes to the wrong place

I can't fix this blind. The program chains vision, the world frame, the base calibration, and the arm model, and a wrong place can come from any of them. The error pattern identifies which one, so I'd start by finding where the chain first goes wrong.

Step 1: Describe how it's wrong

Symptom	Likely cause
Points in the wrong **direction** (rotated about the base), with the right distance	Base yaw calibration (`baseA`, `baseB`, `baseTable`), or the sign of the base servo. These are likely still Bryson's.
Right direction, wrong **distance** (too far or too short)	`layout.json` tag 0 offset, `ds`/`zA`/`dz`/reach corrections in `arm_aviva.json`, or link lengths
Wrong **height** (hits the table or hovers high)	Shoulder height, tool offset `dz`, or the servo fit for joints 2 and 3
Everything **shifted by the same vector**	`baseTag.centerUV_mm` in `layout.json`, or tag 0 isn't where the file says

| **Mirrored** left/right | Base servo sign, or tag 0 or holder tags swapped |
| Wrong by a **different amount per stamp** | Camera calibration (intrinsics) or holder tag fit |

Step 2: Check each link of the chain

A. Vision (no arm needed). Load the result and check:
- The GUI line **"Holder tags X mm apart (CAD ±Y mm)"** should have Y within about 2 mm. If not, scale or intrinsics are wrong.
- The circles on the photo should sit on the stamps.
- **`--calibration` defaults to the original author's camera.** You changed cameras, so you need your own intrinsics, at 640×480. Wrong intrinsics put every stamp in the wrong place by a different amount.
- Measure one stamp top with a ruler from the base axis and compare with the `top [U, V, W]` the GUI prints. This is the most direct test of the world frame.

B. Origin. `config/layout.json` must hold where tag 0's centre sits relative to the base axis on your setup, and tag 0 must be physically there. If the file is Bryson's, everything shifts.

C. Arm kinematics (no vision needed). Run `python tools/ik_gui.py logs/servo_angle_points_aviva.csv`, connect, and use "Go to" with r = 100, z = 20. Measure where the tip lands relative to the shoulder axis. If this is wrong, the servo fit or link lengths are the problem (check the fit's `a` and θ range).

D. Base yaw. Command the base to two servo angles and measure the actual azimuth difference. It should equal Δservo / `baseA`. If not, `baseA`/`baseB` are wrong.

Step 3: Dump the chain for one stamp

This shows every number between the photo and the servos. Save it as `tools/trace_stamp.py` and run `python tools/trace_stamp.py logs/stampholder_<time>.json L7`:

```python
import json, sys
from pathlib import Path
from ik_gui import Kinematics, loadCalibration
from move_to_tag import ReachModel, loadArmConfig
from servo_angle_gui import loadJointConfig
from stamp_hover_gui import atStamp, baseTagCenter, reachOf, stampPixels

cfg = loadArmConfig("config/arm_aviva.json")
joints, _ = loadJointConfig()
model = ReachModel(cfg, Kinematics(joints, loadCalibration(cfg["servoCalibration"])))
result = json.loads(Path(sys.argv[1]).read_text())
stamps = result["stamps"] if isinstance(result["stamps"], list) else [result["stamps"]]
name = sys.argv[2].upper()
```

index, stamp = next((i, s) for i, s in enumerate(stamps) if s["name"].upper() == name)

U, V, Wtop = stamp["top_UVW_mm"]
print("tag 0 centre in world (layout.json):", baseTagCenter())
print("pixel of stamp top:", stampPixels(result)[index])
print(f"world: U={U:.1f} V={V:.1f} W={Wtop:.1f} reach={reachOf(model, U, V):.1f} mm")
pose = atStamp(model, U, V, Wtop + 5)
print(f"theta0={pose['theta0']:.2f} deg, servos (base, shoulder, elbow)={pose['s']}")
print("holder baseline check:", result.get("holder", {}).get("baselineError_mm"))
```

Walk the output with a ruler. If (U, V) matches reality, the problem is downstream (base calibration, arm config, or servo fit). If it doesn't, it's upstream (camera calibration, tag 0 or `layout.json`).

## What to send me

1. A short description of the error: which direction, roughly how far, and whether it's the same for every stamp
2. The output of `trace_stamp.py`, plus the log line "Holder tags ... apart"
3. `config/arm_aviva.json` and `config/layout.json`
4. `ReachModel`, `baseAngleOf`, and `solve` from `move_to_tag.py`
5. The `--calibration` file name you're using

With those I can say which value is wrong and what to put in.

**Safety:** until you've located the error, run **Hover over** on one stamp at a high hover value (for example 40 mm), at low speed, with a hand near STOP (Esc). Don't use Pick & return, since a wrong target can drive the arm into the holder or table.

>> User:
Error is the same for every stamp. here's arm_aviva.json (first pasted text) and layout.json. How do I tell what --calibration file name I'm using?


```
 {
  "_notes": [
    "Shared rack layout, identical for both arms. Lengths in mm, angles in deg.",
    "World frame [U, V, W] = arm base frame: origin on the base axis at plate level, W up.",
    "Tag frame: x along detected corner 0 -> corner 1, y 90 deg counterclockwise from x seen from above.",
    "yaw_deg = direction of the tag's x axis in the world frame, measured from +U toward +V.",
    "offsetTag_mm = [x, y] from the tag centre to the point of interest, in the tag frame.",
    "null = not measured yet. Required values that are null stop the program with a message.",
```
```

```
    "baseTag: measured t_0 (tag 0 centre -> base axis) = [-57.5, 84.5, 0] in tag 0's frame. U, V are taken parallel to tag 0's x, y, so the tag centre is at -t_0 and yaw_deg = 0."
  ],

  "calibrationFile": "calibration/iphone-0.5-cameraparams.mat",
  "photoDir": "photos",
  "logDir": "logs",

  "tagFamily": "tag36h11",
  "tagSize_mm": 50,
  "tagCornerWinding": "ccw",
  "sideLengthTolerance": 0.03,

  "baseTag": {
    "id": 0,
    "centerUV_mm": [57.5, -84.5],
    "yaw_deg": 0,
    "height_mm": 0
  },

  "inkPad": {
    "id": null,
    "height_mm": 0,
    "offsetTag_mm": [null, null],
    "targetHeight_mm": null
  },

  "paper": {
    "id": null,
    "height_mm": 0,
    "offsetTag_mm": [0, 0],
    "targetHeight_mm": null
  },

  "arm": {
    "id": null,
    "height_mm": null,
    "offsetTag_mm": [0, 0]
  },

  "stamp": {
    "tagToDisc_mm": [null, null],
    "discHeight_mm": null
  },
```

```
  "holderSource": "tags",
  "useHolderTagsAsReference": false,

  "stampHolders": [
    { "letter": "A", "tagId": null, "tagHeight_mm": 0, "onshapeCenterUV_mm": [null, null],
"onshapeYaw_deg": null }
  ]
}
```


>> Anthropic Claude Sonnet 5.5:
Thinking:
The user wants to know which calibration file is actually being used, and I'm checking the code's default value—looking at move_to_tag.py, the default --calibration path is calibration/webcamParams.mat, so with no flag passed that's what runs, even though layout.json points elsewhere.

To verify, they should check the actual loaded calibration file's ImageSize and intrinsics match their webcam's resolution, since loading a mismatched camera's calibration (like an iPhone's) would give wrong intrinsics; ideally they should calibrate their own webcam with MATLAB's Camera Calibrator and pass that file explicitly via --calibration.

I also notice arm_aviva.json inherits Bryson's name, baseFit, baseCorrection, and reachTable values — all tuned specifically to his arm and rack, plus his servoCalibration CSV. A constant per-stamp error points to a systematic offset like the base angle b (which changed across Bryson's calibration attempts from 57.81 to 53.52 to 47.35), so Aviva likely needs her own baseFit recalibration via servo_angle_gui, with the Bryson-specific baseCorrection and reachTable cleared out, while checking whether layout.json's shared baseTag centerUV value still applies to her physical setup.

I should also compare the result JSON's K matrix against the calibration file's intrinsics directly—a webcam at 640x480 would have fx around 500-800, while an iPhone calibration would show much larger fx values at higher resolution, which would reveal if the wrong calibration got loaded.

I'm also thinking through diagnostic tests: with the camera pinned to index 0 at 640x480, a constant error across every stamp would suggest a pure base angle offset, since the tangential error would scale with reach distance.

To isolate whether the error is rotational or translational, I'd compare U1 and L13 at opposite ends of the arc—a rotation error would show opposite tangent directions at each end, while a translation error (wrong tag origin or layout shift) would show the same error vector everywhere. For a fix, I'd suggest using the GUI's base check jog: hover over a stamp, jog until centered, and record the offset to get the base correction delta. For a new arm, I'd recommend starting with a stripped-down config that removes baseCorrection and reachTable entirely so they default to neutral values, then re-measuring reach corrections from scratch rather than reusing old calibration data.

I should check whether aviva's calibration file is actually distinct from Bryson's — I'd want to inspect the webcam calibration parameters (focal length, image size) and compare to what's being loaded in the result JSON. There's a tempting but suspicious memory entry suggesting Aviva's servo fit had near-zero residuals, but the date on that memory is actually after today's date, so I'll disregard it as unreliable rather than factor it into my diagnosis.

I'm structuring the answer around: which calibration file is in use, how to verify it's correct, the main finding that arm_aviva.json still has Bryson's values, how to distinguish and fix the constant error, and what additional info I need from the user.

For calibration, I'd note the standard workflow — capturing checkerboard images at 640x480 with MATLAB's Camera Calibrator, saving the parameters, then running with the explicit `--calibration` path relative to the repo root. I also notice layout.json references an iPhone calibration file separately, but since the GUI passes its own calibration path explicitly, I'm not sure that iPhone entry is even in play here — I'd flag this as something to verify against the actual MATLAB command being run rather than assume.

>> Anthropic Claude Sonnet 5.5:
Which `--calibration` file you're using

You didn't pass `--calibration`, so you're using the default: `calibration/webcamParams.mat`. That default comes from `DEFAULT_WEBCAM_CALIBRATION` in `move_to_tag.py`, and `stamp_hover_gui.py` forwards it to MATLAB through `matlabCommand(photoPath, calibration)`. Your MATLAB run is relative to the repo root (`cwd=REPO_ROOT`), so the file is `<repo>/calibration/webcamParams.mat`.

That is almost certainly the camera of whoever wrote the program, not your plug-in webcam. A webcam calibration is only valid for that camera at that resolution. If it came from a different camera, every stamp position comes out wrong in a consistent way, which fits your symptom of the same error for every stamp.

The `calibrationFile` entry in `layout.json` (`iphone-0.5-cameraparams.mat`) is an iPhone calibration. The GUI never reads it. I can't see `matlabCommand` in `check_stamp_holder.py` or the MATLAB code, so check whether MATLAB falls back to that file anywhere.

How to confirm what was used

1. **Compare the intrinsics.** The result JSON stores the camera matrix MATLAB actually used:
   ```python
   import json, numpy as np
   r = json.load(open("logs/stampholder_<time>.json"))
   print(np.array(r["pose"]["K"]), r["pose"].get("radialDistortion"))
   ```
   Then in MATLAB:

```matlab
s = load('calibration/webcamParams.mat'); disp(s)
% look at: ImageSize, Intrinsics.FocalLength, PrincipalPoint, RadialDistortion
```

`K` should match the file. For a 640×480 webcam, the focal length is typically around 500–900 px, and the principal point is near (320, 240). Check `ImageSize = [480 640]` too.
2. **Check the file's date and origin** with `dir calibration`. A file that predates your hardware was not made for your camera.

Fix: calibrate your own webcam

1. Print a checkerboard and measure the square size.
2. In MATLAB, open the Camera Calibrator (Apps → Camera Calibrator).
3. Use 15–20 images from your webcam, captured at **640×480** with the same `--camera 0` and the same focus.
4. Export `cameraParams` to `calibration/avivaWebcam.mat`.
5. Run with `--calibration calibration/avivaWebcam.mat`.

Autofocus matters. If the webcam refocuses between calibration and use, the focal length changes. Lock the focus if you can.

A second likely cause: `arm_aviva.json` is still Bryson's arm

Your `arm_aviva.json` still says `"name": "BRYSON"`. Only `servoCalibration` was changed. Bryson's arm-specific tuning is still there:

- **`baseFit`** (`a = 1.0331`, `b = 47.35`). The notes show this was refit three times to compensate for Bryson's servo and horn mounting. A different arm will have a different `b`, which is a constant angular offset. That produces a miss of about `R · Δθ` for every stamp. At R = 135 mm, a 5° offset is about 12 mm, and it is the same for every stamp.
- **`baseCorrection`** and **`reachTable`**. These are measured on Bryson's arm and rack, so they are meaningless for yours.
- **`reachCorrection_mm`** and **`reachCorrectionPerHeight`** are also Bryson's measured values.
- **`shoulderOffset_mm`** and **`toolOffset_mm`** may be the same if both arms are the same RASTICrane build. Check them against your arm.

Bryson's tuning may not transfer, so neutralize it:

```json
"name": "AVIVA",
"baseFit": { "a": 1.0, "b": 0.0 },
"baseCorrection": null,
"reachTable": null,
"reachCorrection_mm": 0,
```

"reachCorrectionPerHeight": 0
```

The loader accepts a missing or `null` `baseCorrection` and `reachTable`. These values are placeholders until you refit them in the next step.

## Refit the base for your arm

1. Open `servo_angle_gui.py`. Record **joint 1** at three known directions: arm along +U (θ = 0), +V (θ = 90), and −U (θ = 180). Use the base tag's edges as the reference.
2. Fit s = a·θ + b and copy `a` and `b` into `baseFit`. The old notes show this method for Bryson, which gave a slope near 1 and a `b` that depends on the horn position.
3. Refine in the GUI: **Hover over** a stamp at a high hover value, jog the base until the tool is centred, and use **Record offset**. The `jog_deg` column in `logs/stamp_offsets.csv` shows what's left. Fix `b` by that amount.

## Tell a rotation from a shift

Since the error is the same for every stamp, hover over two stamps at opposite ends of the arc (for example L1 and L13) and look at the miss on the table:

| What you see | Likely cause |
|---|---|
| Miss is the **same vector** at both | World frame shifted: tag 0 isn't where `layout.json` says (57.5, −84.5), or tag 0 is rotated |
| Miss is **perpendicular to the arm** at each, and changes direction between them | Base angle error: `baseFit` |
| Miss is **along the arm** at each | Reach: link lengths, offsets, or the servo fit for joints 2 and 3 |
| Miss **grows with distance** from the base tag | Camera intrinsics or scale |

Also check these:
- **Tag 0's orientation.** `layout.json` says U and V run parallel to tag 0's x and y edges, with `yaw_deg = 0`. If your tag 0 is rotated, everything rotates.
- **Tag 0's position.** The tag centre must be 57.5 mm in U and −84.5 mm in V from the base axis. Measure it on your plate. If the rack plate is the same one Bryson used, this will match, but verify it.

## What to send next

1. The `K` printout above and the `ImageSize` from the calibration file
2. For L1 and L13: where the tool landed versus the stamp (direction and mm)
3. The `servo_angle_points_aviva.csv` joint 2 and 3 rows
4. `matlabCommand` from `check_stamp_holder.py`

**Safety:** keep testing with Hover at 30–40 mm and low speed, with a hand on Esc. With a wrong frame, Pick & return can drive the arm into the holder.
```

>> User:
here are the J1 and J2 fits. you can keep the yaw fit the same or try to figure it out from the attached file

>> Anthropic Claude Sonnet 5.5:
Thinking:
I'm examining the joint 1 data points to fit a linear relationship between reference angle and sensor value, computing slope and intercept using the three given angle-sensor pairs.

I calculate the slope at -0.9722 and intercept at 151.67, giving residuals of about -1.67, 0.83, and 0.83, which produces an rms error of roughly 1.18.

Now I'm questioning what reference frame the angle convention actually uses for this base joint -- whether it's measured relative to the servo arm orientation along a particular axis -- since this matters for interpreting whether the negative slope is consistent with the expected geometric model from similar prior setups.

Checking the overshoot logic: since the comparisons and offsets operate purely in servo space rather than physical angle space, the direction convention sign doesn't actually break the comparison, it only affects which inline comment calling it "clockwise" versus "counterclockwise" is technically correct. The rest of the jogging and offset-recording logic appears sign-agnostic and fine.

I'm also checking the base servo's reachable range given the calibration constants — mapping servo limits of roughly 0 to 300 back through theta0 to see what physical angles the rack can actually sweep, and confirming the joint limits are enforced by firmware minimums rather than this calculation.

Now I'm second-guessing the sign convention: whether Bryson and Aviva defined their zero reference the same way, since Bryson's servo values increase going counterclockwise while Aviva's seem to decrease, which could mean a mirrored mount or just a different reference angle in how the measurements were taken.

This checks out: ref = 90 - theta0, a compass-like convention measured clockwise from +V, with theta0=0 giving ref 90, theta0=90 giving ref 0, and theta0=180 giving ref -90, consistent with symmetry about the y-axis.

Working through the algebra, s = 64.17 + 0.9722·θ0, which closely matches Bryson's original fit (a=0.9939, b=57.81) and the given data points (65, 150, 240) versus Bryson's (58.5, 146.3, 237.2) — this strongly supports the same mounting with a slight horn offset, and keeps the positive-slope "clockwise" interpretation valid.

I should recommend a=0.9722, b=64.17 but flag the alternative sign interpretation, suggesting a quick test: command the servo to 64 and see if the arm points toward +U, then to 150 for +V, to confirm against

the user's observed "home" direction. I'm also noting that the ~17 servo-degree gap between Bryson's refit (b=47.35) and this value translates to roughly 16° of angular error, or about 38mm at the 135mm radius — enough to explain the "completely wrong place" symptom either way the sign goes.

Looking at the J2 and J3 calibration fits, both come from only two reference points each, so their rms of 0.00 is meaningless as an error estimate — I'd flagged this concern earlier. The scale factors (-1.05 for J2, 1.133 for J3) aren't close to ±1, which is suspicious since it implies the servos aren't tracking 1:1 with the input angle, though this could just reflect real hardware quirks or imprecise reference readings; I don't have Bryson's equivalent values to compare against.

Checking the joint 3 reference at -90 with s2=78 to verify link 1's angle computes consistently, which it does at 63.8°, so no issue there.

For joint 2/3 the parallelogram linkage keeps θ2 absolute, confirming that's fine too. For the base yaw, I'll deliver a=0.9722, b=64.17 with the reasoning and updated JSON, and I'm now considering whether Bryson's reach correction c(W) = -0.4+0.185W, measured on his own arm, might introduce an 8.9mm offset at W=50 that could matter.

Since that correction likely stems from Bryson's specific servo fit residuals rather than something universal, I should set it to 0 initially for Aviva's different build and let it be re-measured, keeping toolOffset/shoulderOffset unchanged. I'll finalize the JSON notes with this data and suggest a quick verification step — commanding joint 1 to 64.2° and checking the arm points along the correct direction relative to tag 0's axis.

I'm second-guessing my sign convention though — I need the user to confirm what direction they pointed the arm when ref_deg was 0 during the joint 1 recording, since there are two plausible interpretations (direct correspondence to +U/+V versus a clockwise-mirrored convention), which would flip the sign of the coefficient and change the calibration formula entirely.

Checking the alternative mapping where +U is center instead: the rack range would shift to the upper end of the 300 limit, but that would mean the arm is mounted 90° rotated relative to Bryson's setup. Interpretation 1 still seems more probable since it better matches Bryson's numbers (65 vs 58.5).

Confirming this with the plate geometry — the base plate bolts onto the rack the same way the arm mounts, so the mounting direction should be consistent across interpretations, giving good confidence in interpretation 1 pending verification. The quick physical check: at servo 64 the arm should point along +U, at 150 along +V toward the holder's middle, keeping in mind ref=90-θ and that the base servo has only one sample per point with rms 1.18.

I should also flag the servo's ~1 degree deadband and ~3 degree direction-dependent band, and recommend recording the base points approaching from the same clockwise direction. Now I'm updating the JSON file including the `_notes` strings, and I still need to flag the camera calibration issue and the recurring "same for every stamp" error as separate items to check.

I want to flag the joint 2/3 fit concerns too — J2's slope a=-1.05 and J3's a=1.133 are each based on only two points, so an rms of 0.00 doesn't actually confirm good calibration. I should suggest adding more sample points (J2 at 45/90/135, J3 at -90/-45/0/30) to validate the slope, since a bad slope like 1.13 vs the expected 1.0 would mean angle errors — and for J3 specifically, a 5° elbow error translates to roughly 7mm of positional error given the 80mm arm length. I'll also suggest a quick practical check using the IK GUI's "Go to" function at r=100, z=20 and measuring the result.

Running the numbers again: fitting a=-0.97222 and b=151.6667 gives predicted servo values of 64.17 at θ0=0 (measured 65), 151.67 at θ0=90 (measured 150), and 239.17 at θ0=180 (measured 240) — all residuals under 2, which checks out well. I'm also noting that with three points, this a=0.9722 value differs somewhat from an earlier reference figure, which is worth mentioning.

These differences are still within noise. I'll suggest keeping a=0.9722 as-is, note that the base servo range (θ0 41–139° mapping to s=104–199) stays safely within limits, and remind them to check the base servo limits defined in main.cpp (0–300) before finalizing the answer, keeping the response fairly concise.

>> Anthropic Claude Sonnet 5.5:
Using your calibration

The base (yaw) fit can be worked out from your file, but it depends on one assumption you need to confirm.

Base yaw (joint 1)

In `servo_angle_gui` you entered `ref_deg` as 0, -90 and 90, with servo values 150, 240 and 65. The program's `baseFit` is defined differently: `s1 = a·θ0 + b`, where θ0 is measured from **+U toward +V** (CCW from above) and `b` is the servo angle with the arm along +U.

Fitting your numbers directly gives a slope of **-0.97**, which would mean your base servo runs backwards from Bryson's. I think it's more likely your angles use a different zero. Your servo values (65, 150, 240) almost match Bryson's original measurements (58.5 at +U, 146.3 at +V, 237.2 at -U), shifted by one position. That fits `ref = 90 − θ0`:

Your ref_deg	Arm direction	θ0	Servo
90	+U	0°	65
0	+V (straight toward the holder)	90°	150
-90	-U	180°	240

Least squares on those three points gives:

```
a = +0.9722    b = 64.17    rms = 1.18 servo deg
```

```

**Confirm this before moving to a stamp.** In `servo_angle_gui`, send base = 64 and check that the arm points along tag 0's +x edge (+U). Then send base = 150 and check that it points along +V, at the middle of the holder. If it's mirrored, tell me. The fit would then be `a = +0.9722, b = 151.67` or similar, depending on how you defined 0°.

This is probably most of your "completely wrong place" error. Bryson's `b = 47.35` differs from yours by about 17 servo degrees, which is a miss of roughly 40 mm at R = 135 mm.

## Joints 2 and 3

The numbers in your screenshot match what the CSV gives:

- J2: (0°, 145) and (90°, 50.5) give a = -1.05, b = 145.0
- J3: (0°, 141.5) and (-90°, 39.5) give a = +1.1333, b = 141.5

**The `rms 0.00` means nothing.** Two points always fit a line exactly. With only two points, any error in how well you lined up the link gets absorbed into the slope. Your slopes (-1.05 and +1.13) are not 1.00, and a 3° alignment error at the 90° point changes a by about 3%. Elbow angle error is multiplied by the 80 mm link, so 5° is about 7 mm at the tip.

Add at least 3 more points each, over the angles you will use:

- **J2** (valid θ1 range [-33, 138]): θ = 45, 90, 135
- **J3** (valid θ2 range [-125, 34]): θ = -90, -45, 0, 30

If the new points fall on the line, keep the fit. If they don't, the slope was wrong. Approach each point from the same direction, because servo backlash is about 1° or more.

## Update `arm_aviva.json`

Bryson's rack-specific corrections (`baseCorrection`, `reachTable`) and his reach correction were measured on his arm, so neutralize them until you refit them on yours:

```json
"name": "AVIVA",
"servoCalibration": "logs/servo_angle_points_aviva.csv",

"baseFit": { "a": 0.9722, "b": 64.17 },
"baseCorrection": null,
"reachTable": null,
"reachCorrection_mm": 0,
"reachCorrectionPerHeight": 0
```
```

```

Keep `shoulderOffset_mm` and `toolOffset_mm` if your arm is the same build. Remove the old Bryson notes about the base fit, or replace them with the date and the three points above.

## Check order

1. Verify the base direction at servo 64 and 150 as above.
2. Run `ik_gui.py` and "Go to" r = 100, z = 20. Measure the tip against the shoulder axis. This tests the J2/J3 fit.
3. Use **Hover over** at 30 to 40 mm on a middle stamp (e.g. L7), at low speed, with a hand near Esc.
4. Jog to centre and use **Record offset**. The `jog_deg` value in `logs/stamp_offsets.csv` is the remaining error in `b`.
5. Then try a stamp at each end of the arc to see whether `a` needs adjusting.

## Camera calibration

The remaining possible error is the camera calibration. Your `--calibration` is still the default, `calibration/webcamParams.mat`. If the base fit is right and the miss is still the same for every stamp, check that next.

>> User:
the camera is the same for me and Bryson. I think the issue is in the servo calibration. are there any other places where bryson's arm config is still being used? please send me a json file I should use

>> Anthropic Claude Sonnet 5.5:
Thinking:
I'm checking stamp_hover_gui.py for other references to Bryson's arm config, like the default config path and docstring, and tracing where loadJointConf is used.

Other spots carrying Bryson-specific data: the firmware's joint limits/home pose shared across arms, ik_gui.py's hardcoded calibration CSV and A1/A2 lengths, move_to_tag.py's trial and offset logs that mix in Bryson's data unless filtered by date, and possibly stamp_job.json's tuned values. I'm also double-checking the calibration CSV's joint 2/3 rows to verify the linear fit between reference degrees and servo degrees — getting a slope of about -1.05 and intercept of 145 for joint 2.

Now I'm doing the same check for joint 3, getting a slope near 1.133, which looks consistent. But I'm second-guessing my earlier assumption about joint 1's reference convention — I need to confirm whether ref=0 corresponds to the arm pointing toward the holder, since the three data points (0→150, -90→240, 90→65) give a fitted slope around -0.97, and I want to flag this assumption as unconfirmed rather than treat it as settled.
```

I notice joint 2's slope is negative while joint 3's is positive, and without knowing Bryson's reference convention I can't tell if these match expectations. Looking at the elbow angle ranges from the screenshot, joint 2's angle spans roughly -33 to 138 degrees and joint 3 spans about -125 to 34 degrees, which hints at an elbow-up configuration worth checking against the servo data.

One thing that catches my eye: the config file references the servo calibration path as "servo_angle_points_aviva.csv" but the actual filename provided seems to be "servo_angle_points_aviva_csv.csv" — a mismatch that could prevent the file from loading even though the reach issue was supposedly fixed elsewhere.

I'm also checking shoulderOffset_mm and toolOffset_mm, which come from the CAD/OnShape model, so they're likely shared across arm builds while servo horn offsets stay calibration-specific. I should verify base theta zero reference alignment and cross-check the base servo range in main.cpp against the configured limits.

The theta0 range at the holder maps cleanly to the servo range, and the tool offset's s-component checks out at -3.65. Since the camera setup is confirmed shared between the user and Bryson, I can trust the default calibration intrinsics, with extrinsics still anchored to tag 0; other config files like stamp_job.json and stamp_holder.json are likely tuned independently per setup, and I should note the default `--arm` argument defaults to Bryson.

My own scripts, trace_stamp.py and diagnose_reach.py, hardcode arm_aviva, and since the firmware is shared, the home pose and joint limits should carry over correctly for Aviva's arm too. I'm now verifying the JSON config structure — checking that baseCorrection and reachTable being null resolve safely to zero through the loader's fallback logic, and confirming the config loader's required fields like servoCalibration, shoulder, tool, and baseFit coefficients are all present.

For Bryson's reach correction formula, the height-dependent error of about 10.7mm at W=60 is substantial, and I'm unsure whether it stems from a geometric quirk shared by both arms (like an unlevel end plate) or from Bryson's specific servo calibration fit absorbing some of that error. Since Aviva's fit will differ, I think the safest approach is to start her reach correction at zero and recommend she refit empirically using move_to_tag with the fit flag if she observes consistent height-dependent undershoot, rather than guessing at a value I can't verify.

I also want to flag that she should use the since flag or move the old log file when refitting, since the trials CSV still contains Bryson's data mixed in. And I should note that the joint 2/3 slope values I'm giving are inferred and the reference orientation assumptions need to be checked -- particularly the J3 slope of 1.13 which seems off from the expected ~1 if servo degrees should map directly to link degrees, suggesting something unusual with the parallelogram linkage's reference convention that I should call out as uncertain.

I think the most valuable thing I can give her now is a checklist of exactly where Bryson's calibration data still lives across stamp_offsets.csv, base_offsets.csv, and tag_reach_trials.csv, along with a recommendation to change the default arm config to arm_aviva.json so tools like move_to_tag don't

silently fall back to his data, plus an explicit caveat that I haven't inspected stamp_job.json, stamp_holder.json, check_stamp_holder.py, or the firmware, so those could still hold his offsets too.

I'll also walk back my earlier suspicion about the camera calibration being someone else's rig since she's confirmed it's the same camera -- though I'll briefly note that even the same model webcam can have slightly different intrinsics, so a quick K comparison is still cheap insurance.

Then I'll write out the JSON for her with a "_notes" field (since JSON can't have comments) keeping "name": "AVIVA", followed by a short list of concrete next steps.

>> Anthropic Claude Sonnet 5.5:
Where Bryson's arm data is still used, and a config for your arm

Your arm config now points at your own joint 2/3 calibration, but several other things in the repo still come from Bryson's arm.

What is still Bryson's

Where	What	Fix
`arm_aviva.json`	`baseFit`, `baseCorrection`, `reachTable`, reach correction. The notes say these were fitted from his jogs and his plumb-line trials.	Replaced in the JSON below
`move_to_tag.py`	`DEFAULT_ARM_CONFIG = config/arm_bryson.json`. `ik_gui.py`, `move_to_tag.py --fit` and any tool run without `--arm` silently use his arm.	Change the default to `arm_aviva.json`, or always pass `--arm`
`ik_gui.py`	`DEFAULT_CALIBRATION = logs/servo_angle_points_corrected.csv`, probably his file.	Always pass your CSV path
`logs/tag_reach_trials.csv`, `base_offsets.csv`, `stamp_offsets.csv`	His measured misses. Your **Record offset** clicks append to the same files.	Move his files to a subfolder. If you can't, use `--fit --since <time>`, or `--fit` will fit his data into your arm.
`firmware/src/main.cpp`	Joint limits and home angles, read by `loadJointConfig()`.	Fine if both arms run the same firmware. If your servo ranges differ, edit the `kJoints` table.
`A1_mm = A2_mm = 80` in `ik_gui.py`	Link lengths	Fine if the builds are identical. Measure yours.

I haven't seen `stamp_job.json`, `stamp_holder.json`, or the MATLAB code. The first two describe the rack and paper, so they should be shared, but check them for numbers that Bryson tuned.

Check the filename. The file you attached is called `servo_angle_points_aviva_csv.csv`. `arm_aviva.json` points to `logs/servo_angle_points_aviva.csv`. If your file on disk has the longer name, the config is loading a different file or failing, so make them match exactly.

You said the camera is the same, so I'll drop that as a suspect. Comparing `K` in your result JSON against the calibration file is still a cheap check.

The config to use

Save as `config/arm_aviva.json`:

```json
{
  "_notes": [
    "Per-arm config (Aviva's arm). Lengths in mm, angles in deg.",
    "Arm frame: r = outward along the arm's plane, s = 90 deg CCW from r seen from above, z = up.",
    "shoulderOffset_mm and toolOffset_mm: CAD values from the shared RASTICrane OnShape model (same as Bryson's). Not re-measured on this arm.",
    "baseFit: s1 = a * theta0 + b, theta0 = direction of +r from +U toward +V. From the 2026-10-13 joint 1 points: 65 at +U, 150 at +V, 240 at -U, assuming ref_deg = 90 - theta0. UNVERIFIED: check the arm points along +U at servo 64 and along +V at servo 150.",
    "Joints 2 and 3 come from servoCalibration (2 points each so far: slopes -1.05 and +1.13, rms is meaningless with 2 points).",
    "baseCorrection, reachTable and the reach correction are zeroed. Bryson's values were fitted to his arm. Refit on this arm with Hover over + Record offset and move_to_tag.py --fit --since <time>."
  ],

  "name": "AVIVA",
  "servoCalibration": "logs/servo_angle_points_aviva.csv",

  "shoulderOffset_mm": [13.4, -16.15, 55.175],
  "toolOffset_mm": [27.724, 12.5, -32.912],

  "baseFit": { "a": 0.9722, "b": 64.17 },
  "baseCorrection": null,
  "reachTable": null,

  "minToolHeight_mm": 0,

  "reachCorrection_mm": 0,
  "reachCorrectionPerHeight": 0
}
```

I haven't seen your arm, so these three values are my best guess:

- **`baseFit`** assumes my reading of your `ref_deg` convention. Test it before any stamp motion. In `servo_angle_gui`, send base = 64 and check that the arm points along tag 0's +x edge. Then send base = 150 and check that it points along +V, toward the middle of the rack. If it is mirrored, tell me what you see.

- **`reachCorrectionPerHeight: 0`** discards Bryson's 0.185 mm/mm. Bryson's notes show a reach error that grows with height, which could come from the shared mechanism and not only from his servos. If your tool lands short at greater heights, refit it, because 0 may be wrong for your arm too.
- **Tool and shoulder offsets** are only right if your arm matches the CAD. A wrong value gives a constant error, so measure them if the other checks pass and it is still off.

Test order

1. Run `python tools/ik_gui.py logs/servo_angle_points_aviva.csv`, connect, and "Go to" r = 100, z = 20. Measure the tip against the shoulder axis. This tests joints 2 and 3.
2. Use **Hover over** at about 40 mm above L7, at low speed, with a hand on Esc.
3. Hover over L1 and L13 to see whether the miss is a constant shift or grows with angle. A constant shift means `b`, and a miss that changes across the arc means `a`.
4. Jog to centre, **Record offset**, and read `jog_deg` in `stamp_offsets.csv`. That is the correction to `b`.

Also add more points to the joint 2 and 3 calibration. With only 2 points, a small alignment error becomes a wrong slope. Your slopes of -1.05 and +1.13 are suspect, since a direct servo-to-link mapping should give about ±1.

>> User:
base=64 sends to the tag 0's +x edge and base=150 points along +V. the "go to" in the ik_gui is correct but when I run stamp_hover_gui the shoulder axis is not accurate.

>> Anthropic Claude Sonnet 5.5:
{"type":"token_balance","balance":66251.29999999964,"tokenCost":274698,"promptTokens":137349,"prev_count":0,"violation_count":0,"date":"2026-10-16T20:02:44.552Z"}
(This is an error message)

>> User:
base=64 sends to the tag 0's +x edge and base=150 points along +V. the "go to" in the ik_gui is correct but when I run stamp_hover_gui the shoulder axis is not accurate.

>> Anthropic Claude Haiku 4.5:
{"type":"token_balance","balance":66251.29999999964,"tokenCost":137534,"promptTokens":137534,"prev_count":0,"violation_count":0,"date":"2026-10-16T20:05:30.224Z"}
(This is an error message)